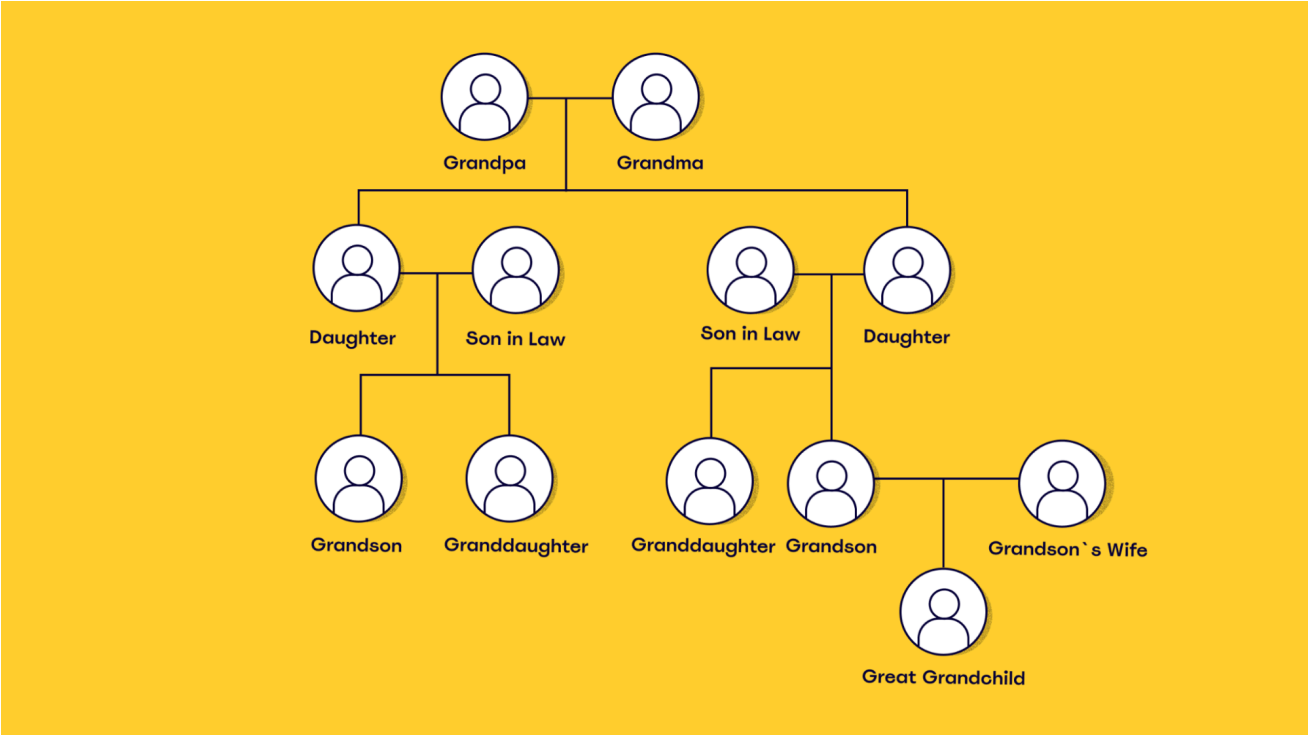
1.1 Foundations of Data Structures

What is a Data Structure?

A **data structure** is a way of organizing and storing data in a computer so that it can be accessed, modified, and processed efficiently. It defines how data is arranged and how different pieces of information relate to one another, influencing the operations that can be performed and how efficiently they can be executed.

What's an example of data structures in our daily life. Take for example, recording family history. In this case, we would use make a family tree such as the following.



We choose a family tree as the data structure because we have information about people we are related to and how they are related, and we want an overview so that we can easily find a specific family member, several generations back. With this, it's easier to see the links and the relations between each individual.

Data structures give us the possibility to manage large amounts of data efficiently for uses such as large databases and internet search engine functions.

Purpose and Utility

The primary goal of a data structure is to minimize computational complexity (both time and space) for common data manipulation tasks. Different data structures are optimized for different types of operations:

- **Arrays and Lists:** Ideal for sequential access and storing collections of items with index-based lookups.
- **Stacks and Queues:** Manage data according to specific processing orders, such as last-in, first-out (LIFO) or first-in, first-out (FIFO).
- **Hash Tables:** Optimized for rapid key-value lookups, insertions, and deletions.
- **Trees and Graphs:** Designed for representing hierarchical relationships and network topologies.

Choosing the correct data structure depends on how the data needs to be accessed, updated, or processed.

Efficiency and Big-O Notation

The efficiency of a data structure is determined by how effectively it supports the operations required by a program. Different structures may perform the same operation at different speeds or require different amounts of memory. These differences become increasingly significant as the volume of data grows. These time and memory differences are notated using Big-O notation.

Big-O notation is a mathematical notation used to describe the asymptotic growth of an algorithm's time or space requirements as the input size increases. It allows programmers to compare algorithmic efficiency without relying on specific hardware or execution times.

The Big-O notation is commonly used to express two things:

- **Time Complexity** describes how the execution time of an algorithm grows relative to the input size. For example, (O(1)) represents constant time growth, which does not depend on the input size, while (O(n)) represents linear growth, where (n) is the input size and the operation's growth is proportional to it.
- **Space Complexity** describes how the memory requirements of an algorithm grow relative to the input size. It accounts for the additional memory needed during execution, such as temporary variables, data structures, and recursive function calls. For example, an algorithm that uses only a fixed number of additional variables has (O(1)) auxiliary space complexity, while an algorithm that creates a separate array containing (n) elements requires (O(n)) auxiliary space.

Selection and Trade-offs

Choosing the appropriate data structure for a given application involves balancing competing priorities:

- **Time Efficiency:** Different data structures can perform the same operation at different speeds. For example, accessing an element by index in an array typically takes constant time, whereas searching an unsorted array may require examining every element. These differences become increasingly important as the amount of data grows.
- **Modification Speed:** Operations such as insertion and deletion also have different costs across data structures. Inserting an element into the middle of an array may require shifting subsequent elements, while inserting into a linked list can be efficient when the insertion position is already known. These differences affect the overall efficiency of algorithms that frequently modify data.

- **Memory Usage:** Data structures vary in their storage requirements. Arrays store elements in a side-by-side sequence, while linked lists require additional memory for references connecting their elements. The choice may therefore involve trading memory overhead for more flexible data manipulation.
- **Problem Requirements:** The most suitable data structure depends on the operations an application performs most frequently. A hash table may be appropriate for retrieving records by key, while a queue is better suited to processing tasks in arrival order.

Ultimately, selecting a data structure requires balancing processing efficiency, memory consumption, and application requirements.

References:

- https://www.w3schools.com/dsa/dsa_intro.php
- https://www.geeksforgeeks.org/dsa/analysis-algorithms-big-o-analysis/
- https://www.w3schools.com/dsa/dsa_timecomplexity_theory.php

1.2 Arrays and Dynamic Arrays

What is an Array?

Imagine a school grading program needs to store the scores of 1000 students. One way we could do it would be to create a separate variable for every score. However, this would make the data difficult to organize, access, and process, especially when the number of students changes.

A better way to solve this would be to group the scores into a single collection of data. This allows the program to manage the values together, access individual elements, and process the entire collection using indexes and algorithms, instead of doing it one by one.

An **array** is a data structure that stores a collection of elements in a linked block of memory, allowing elements to be accessed using their positions, known as indexes. Because the memory is linked together, the computer can instantly calculate the exact memory address of any element using a simple mathematical offset, making retrieval lightning-fast.

Visualisation:



For example we have this array labelled represented as:

```
Numbers = [2, 4, 10, 5, 15, 3]
```

Each array's index starts from 0 and increases based on the order of the elements. Using the example array, Numbers[0] retrieves 2 and Numbers[3] retrieves 5.

Standard (Static) Arrays

When you create a standard array (like in C or Java), you must declare its size upfront. The operating system allocates exactly that amount of linked together memory and nothing more. For example, you can do this by declaring for example int array[n], with n being the size of the array.

Key Characteristics:

- **Fast Access (`O(1)`):** Accessing an element by index (e.g., `array[3]`) takes `O(1)` time. The computer simply takes the base memory address and adds `3 * (element size)`.
- **Cache Friendliness:** Because elements are stored right next to each other, processors can load chunks of the array into the CPU cache simultaneously, making iteration extremely fast.
- **The Limitation:** Static arrays cannot grow. If you declare an array of size 10 and need to add an 11th element, you won't be able to append or add it.

Visualisation:



Dynamic Arrays

Dynamic arrays (like `list` in Python, `std::vector` in C++, or `ArrayList` in Java) solve the size limitation of static arrays while preserving their fast `O(1)` indexing. They achieve this by managing a static array behind the scenes and automatically resizing it when it gets full.

The Resizing Mechanism:

1. **Initialization:** The dynamic array starts with an underlying static array of a small fixed capacity (e.g., capacity of 4).
2. **Insertion:** As you append or insert elements to the end of the list, it fills up the available slots.
3. **Capacity Reached:** When you try to add an element but the capacity is full, the dynamic array creates a *new*, larger static array in memory (typically doubling the size, e.g., from 4 to 8).
4. **Copy and Swap:** It copies all existing elements from the old array into the new one, adds your new element, and then frees the old array's memory.

Because resizing happens infrequently (e.g., at sizes 4, 8, 16, 32...), the heavy cost of copying elements is spread out over many cheap insertions. This makes the **amortized time complexity** for appending an element `O(1)`.

Visualisation:



Operations

Operation	Description
read	Shows a value from an index in the array.
update	Updates the existing value at an array index position.
insert	Inserts a new value in the array, in addition to the existing values.
remove	Removes a value from the array at a given index position.
length	Gives us the number of values in the array. The number of values is the length of an array.
loop	Shows the entire array one-by-one, using a loop.

Example

```
int main(void) {
    int arr[10] = {5, 12, 8, 20};

    // 1
    int val = arr[2];
    printf("%d\n", val);

    // 2
    arr[2] = 21;
    printf("%d\n", arr[2]);
}
```

Try to predict what happens in operation 1 and 2.

- In operation 1, it reads the value of arr[2], which is 8.
- In operation 2, it updates the value of arr[2] to 21.

Performance

Space Complexity

Both static and dynamic arrays have the same space complexity of O(n). The memory required grows proportionally to the number of elements stored.

Time Complexity

Singly Linked List:

Operation	Best Case	Worst Case	Average
Access	`O(1)`	`O(1)`	`O(1)`
Search	`O(1)`	`O(N)`	`O(N)`
Insertion	`O(1)`	`O(N)`	`O(N)`
Deletion	`O(1)`	`O(N)`	`O(N)`

References:

- https://www.geeksforgeeks.org/dsa/introduction-to-arrays-data-structure-and-algorithm-tutorials/
- https://www.w3schools.com/programming/prog_arrays.php
- https://www.geeksforgeeks.org/dsa/difference-between-static-arrays-and-dynamic-arrays/

1.3 Linked Lists

What is a Linked List?

In a treasure hunt, each clue you get leads you directly toward the end goal, the treasure. Each clue tells you what you need to do and what steps you need to take to get to your destination. To reach the end, you must follow the clues from the starting point instead of jumping whichever point you want. This process is similar to how linked lists work.

A linked list is a foundational linear data structure where elements (referred to as nodes) are stored in non-linked memory locations. Unlike arrays, which store elements in a continuous block of memory, the elements of a linked list are connected via pointers (or references). When adding or removing nodes in linked lists, the rest of the nodes in the list do not have to be shifted.

Each node typically consists of two components:

- **Data:** The actual value you want to store (e.g., an integer, string, or object).
- **Next (Pointer):** A reference pointing to the memory address of the next node in the sequence.

Visualisation:



Types of Linked Lists:

- **Singly Linked List:** The most basic type. Each node points only to the next node. Traversal can only move in a single direction (forward from the head to the tail).
- **Doubly Linked List:** Each node contains two pointers: one pointing to the next node and another pointing to the previous node. This allows bidirectional traversal (both forward and backward).
- **Circular Linked List:** The last node points back to the first node instead of pointing to `null`, forming a continuous loop. This can be implemented as either singly or doubly linked.

Visualisation:



Advantages and Disadvantages

Advantages:

- **Dynamic Size:** They can easily grow or shrink in size at runtime, avoiding the fixed-size limitations of static arrays.

- **Efficient Insertions or Deletions:** Adding or removing items doesn't require shifting elements around in memory; you simply update the pointer references.

Disadvantages:

- **No Random Access:** You cannot access the i-th element directly. To find an element, you must start from the head node and traverse through every preceding node
- **Memory Overhead:** Each node requires extra memory to store the pointer(s) in addition to the data itself.
- **Poor Cache Locality:** Because nodes are scattered across memory rather than stored in linked, CPU caching is less efficient compared to arrays.

Operations

Operation	Description
traversal	Goes through the linked list by following the links from one node to the next.
search	Find a node containing a specific value by traversing
insertion	Add a new node (at the head, tail, or specific position) and adjust pointers.
deletion	Remove a node by bypassing it and updating the previous pointer.

Example

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};
void traverseAndPrint(struct Node* head) {
    struct Node* current = head;

    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("null\n");
}

int main() {
    struct Node node1, node2, node3, node4, node5;

    node1.data = 7;
    node2.data = 11;
    node3.data = 3;
    node4.data = 2;
    node5.data = 9;

    node1.next = &node2;
    node2.next = &node3;
    node3.next = &node4;
    node4.next = &node5;
    node5.next = NULL;

    traverseAndPrint(&node1);

    return 0;
}
```

Try to predict the output and execute the code in an IDE.

- The code outputs “7 -> 11 -> 3 -> 2 -> 9 -> null”

Performance

Space Complexity

Linked lists have the space complexity of O(n), as each element requires a separate node containing the data and a pointer/reference to the next node

Time Complexity

Singly Linked List:

Operation	Best Case	Worst Case	Average
Access	`O(1)`	`O(N)`	`O(N)`
Search	`O(1)`	`O(N)`	`O(N)`
Insertion	`O(1)`	`O(N)`	`O(1)`
Deletion	`O(1)`	`O(N)`	`O(1)`

Doubly Linked List: Singly Linked List:

Operation	Best Case	Worst Case	Average
Access	`O(1)`	`O(N)`	`O(N)`
Search	`O(1)`	`O(N)`	`O(N)`
Insertion	`O(1)`	`O(1)`	`O(1)`
Deletion	`O(1)`	`O(1)`	`O(1)`

References:

- https://www.geeksforgeeks.org/dsa/linked-list-data-structure/
- https://www.w3schools.com/dsa/dsa_algo_linkedlists_operations.php

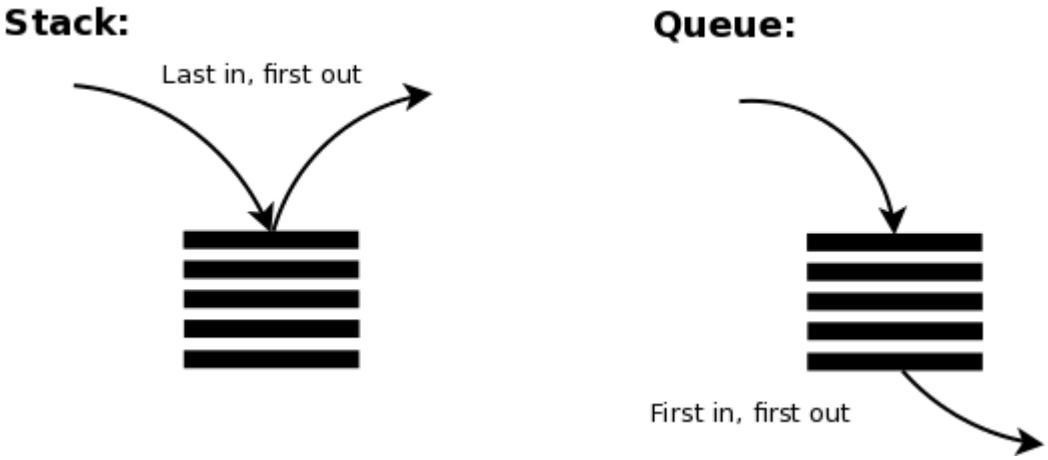
1.4 Stacks and Queues

What are Stacks and Queues?

While piling up plates vertically, You put a clean plate on top, and you also take a plate from the top. The last plate placed is the first one taken. This is similar to a **stack** data structure.

When we are standing in a line at a cashier. The person who arrived first is served first, and new people join at the back. This is similar to a **queue** data structure.

Stacks and **queues** are two ways to store items. They differ in one rule: which item you take out first. A stack removes the newest item first, while a queue removes the oldest item first. Both can use an array, where markers (top for a stack, head and tail for a queue) track where to add and remove, so no items have to be moved.



This process is called “Last in, first out (LIFO)” for stacks and “First in, first out (FIFO)” for queues.

Advantages and Disadvantages

Stacks

Advantages:

- **Time & Memory Efficiency:** Push and pop operations on a stack can be performed in constant time `O(1)`, enabling efficient data access, they are memory-efficient because they only store pushed elements, compared to other data structures.
- **Last-In, First-Out (LIFO):** Stacks follow the LIFO principle, ensuring the last element added is the first one removed. This behavior is useful in scenarios like function calls and expression evaluation.

Disadvantages:

- **Limited access:** Elements in a stack can only be accessed from the top, which makes it difficult to retrieve or modify elements in the middle.
- **Potential for overflow:** Pushing more elements onto a stack than it can hold results in an overflow error, leading to data loss.

Queues

Advantages:

- **Time & Memory Efficiency:** Enqueue and dequeue operations in a properly implemented queue can be performed in constant time `O(1)`, enabling efficient data processing. They can also be memory-efficient because they only store elements that have been added.
- **First-In, First-Out (FIFO):** Queues follow the FIFO principle, ensuring the first element added is the first one removed. This behavior is useful in scenarios like task scheduling and request processing.

Disadvantages:

- **Limited Access:** Elements in a queue can only be directly accessed from the front and rear, making it difficult to retrieve or modify elements in the middle.
- **Potential for Overflow:** In fixed-size queue implementations, adding more elements than the queue can hold results in an overflow error, preventing additional elements from being inserted.

Operations

Stacks:

Operation	Description
push	Add an item to the top of a stacks.
pop	Remove and return the top element from the stack.
peek (top)	Look at the top element of the stack.

Queues:

Operation	Description
enqueue	Add an item to the top of a stacks.
dequeue	Remove and return the top element from the stack.
getFront	Look at the top element of the stack.

Both:

Operation	Description
isEmpty	Checks whether the stack contains any elements.
size	Checks the size of the stack

Example

This is an example of stacks (last in, first out).

Step	Action	Stack (left = bottom, right = top)	Removed
1	Push 5	5	
2	Push 3	53	
3	Push 6	536	
4	Pop	53	6
5	Push 4	534	
6	Pop	53	4
7	Pop	5	3

Performance

Space Complexity

Stacks and queues have a space complexity of `O(n)`, as memory usage grows proportionally to the number of elements stored.

Time Complexity

Stacks:

Operation	Best Case	Worst Case	Average
Access	`O(1)`	`O(N)`	`O(N)`
Search	`O(1)`	`O(N)`	`O(N)`
Insertion	`O(1)`	`O(1)`	`O(1)`
Deletion	`O(1)`	`O(1)`	`O(1)`

Queues:

Operation	Best Case	Worst Case	Average
Access	`O(1)`	`O(N)`	`O(N)`
Search	`O(1)`	`O(N)`	`O(N)`
Insertion	`O(1)`	`O(1)`	`O(1)`
Deletion	`O(1)`	`O(1)`	`O(1)`

References:

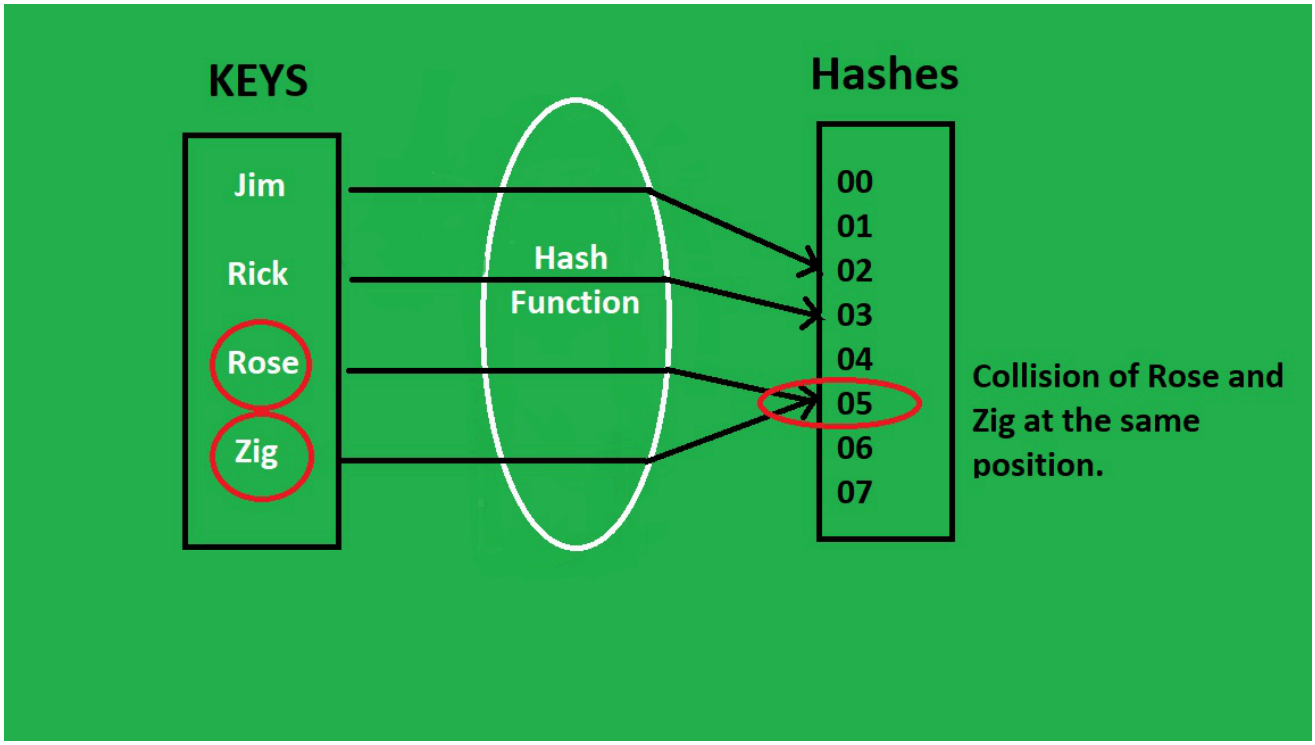
1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. §10.1, pp. 232–236.
2. https://fahadsultan.com/csc122/data/stacks_queues.html

1.5 Hash Tables

What are Hash Tables?

Imagine A shoe rack has shelves labeled A to Z. The rule: each person's shoes go on the shelf matching the first letter of their name. Maria's shoes go on shelf M. Tom's shoes go on shelf T. Mike's shoes also belong on shelf M. Maria is already there, so Mike's shoes go on the same shelf, next to Maria's. To find Mike's shoes, you don't check all 26 shelves. You look at the name, see "M", and go straight to shelf M. On that shelf, you check the few pairs there until you find Mike's. This is an easy analogy of hash tables.

A **hash table** stores items in numbered slots, and a hash function decides which slot each key goes to. One simple hash function is k mod m, which means "the remainder when the key k is divided by the number of slots m". When two keys land in the same slot (a collision), chaining keeps them together in a list in that slot. Searching is usually very fast, because we go straight to one slot, but it is slow in the worst case, when all keys land in the same slot.



In this illustration, it shows four input keys, Jim, Rick, Rose, and Zig. These processed by a hash function to map to index positions ranging from 00 to 07. A collision occurs because both Rose and Zig are incorrectly mapped to the exact same hash position (05). It also shows a hash collision outputting the same hash value. This often results in the function being slower.

Advantages and Disadvantages

Advantages:

- **High Performance:** Provides fast, constant-time `O(1)` lookups and efficient insertions and deletions without requiring element shifting.
- **Space Efficiency & Flexibility:** Uses memory efficiently without extra pointer overhead (unlike trees), supports diverse data types, and suits applications like databases and caches.
- **Built-in Handling:** Has built in features to solve hash collisions, by storing multiple entries in a linked list or tree at the same index position or generating a location for storing or searching the data called probes.

Disadvantages:

- **Collision Vulnerability & Limits:** Collisions are inevitable with large key sets and cause inefficiency. Tables also have limited capacity.
- **Usability Constraints:** Does not allow null values, fails to maintain element order, and can be complex to implement.

Operations

Operation	Description
insertion	Add a new piece of data, usually a key-value pair.
search	Look up the value associated with a given key.
deletion	Remove a key-value pair from the hash table.

Example

Imagine we want to store the keys 10, 22, 31, 4, and 15 in a hash table with 7 slots (0–6), using the division method (h(k) = k mod 7).

Key	Calculation	Slot	What Happens
10	10 ÷ 7 = 1 remainder 3	3	Slot 3 is empty, so store 10.
22	22 ÷ 7 = 3 remainder 1	1	Slot 1 is empty, so store 22.
31	31 ÷ 7 = 4 remainder 3	3	Collision with 10. Add 31 to the front of the list.
4	4 ÷ 7 = 0 remainder 4	4	Slot 4 is empty, so store 4.
15	15 ÷ 7 = 2 remainder 1	1	Collision with 22. Add 15 to the front of the list.

Performance

Space Complexity

Hash tables have a space complexity of `O(N)`, as memory usage grows proportionally to the number of elements stored.

Time Complexity

Stacks:

Operation	Best Case	Worst Case	Average
Access	`O(1)`	`O(N)`	`O(1)`
Search	`O(1)`	`O(N)`	`O(1)`
Insertion	`O(1)`	`O(N)`	`O(1)`
Deletion	`O(1)`	`O(N)`	`O(1)`

Queues:

Operation	Best Case	Worst Case	Average
Access	`O(1)`	`O(N)`	`O(N)`
Search	`O(1)`	`O(N)`	`O(N)`
Insertion	`O(1)`	`O(1)`	`O(1)`
Deletion	`O(1)`	`O(1)`	`O(1)`

References:

1. Cormen et al. Introduction to Algorithms, 3rd ed. Chapter 11 introduction, p. 253; §11.1 “Direct-address tables”, p. 254; §11.2 “Hash tables”, pp. 256-258 (hash function, collisions, chaining, load factor); §11.3.1 “The division method”, p. 263.
2. https://www.geeksforgeeks.org/java/java-program-to-implement-hash-tables-chaining-with-doubly-linked-lists/

1.6 Trees, Binary Search Trees, and Tries

What are Trees?

Imagine a company organization chart. The president sits at the top, with vice presidents and managers positioned below. Each person has a specific position and may oversee other employees. This structure is similar to how trees work.

A **tree** is a set of items called nodes arranged in levels. A tree stores items in levels, like a company chart, with one item at the top and branches below.

A tree has a few components:

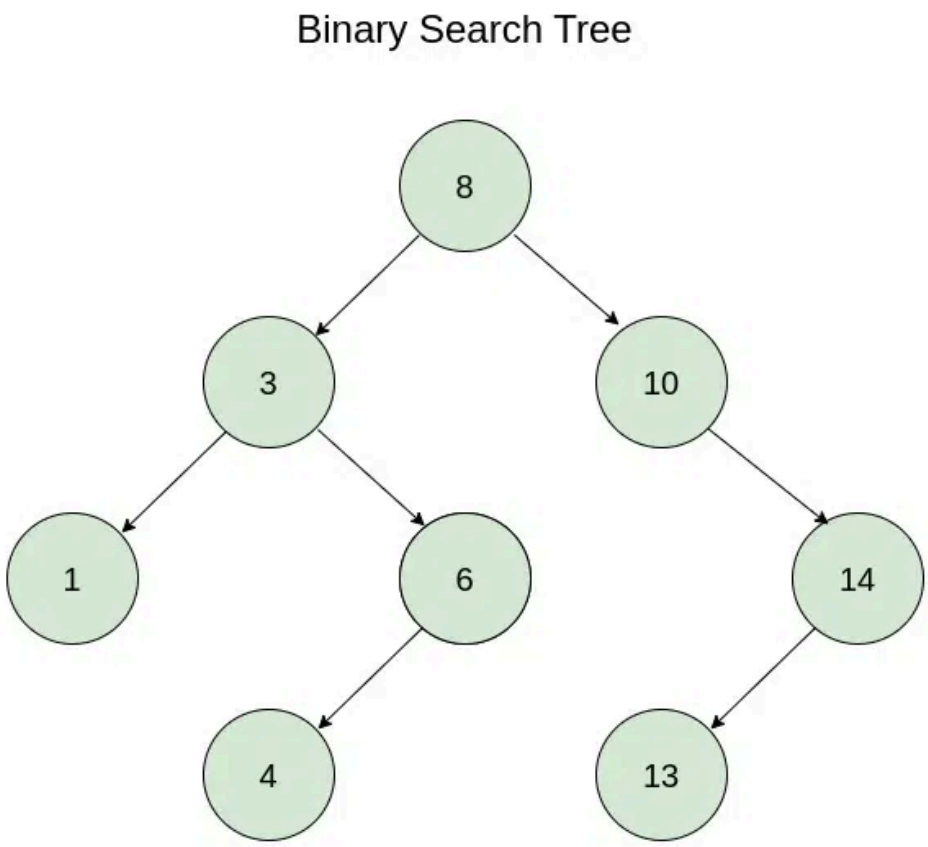
- **Children:** The successor nodes of a node are called its children.
- **Parent:** The predecessor node of a node is called its parent.
- **Root:** The "beginning" node is called the root i.e. a node that has no parent.
- **Leaf:** A node that has no children is called a leaf.



A **binary tree** gives each node at most two children. A **binary search tree (BST)** is a binary tree where, for every node, smaller keys are in its left branch and larger keys are in its right branch. A trie is a tree where each word is stored as a path of letters from the root, and a mark shows where a word ends.

For a tree to be called a binary search tree, it should fulfill the following conditions:

- All the nodes in the left subtree of any node contain smaller values and all the nodes in the right subtree of any node contain larger values.
- Both the left and right subtrees of any node in the tree should themselves be a BST. This means that they should follow the BST rule.
- A unique path exists from the root node to every other node.

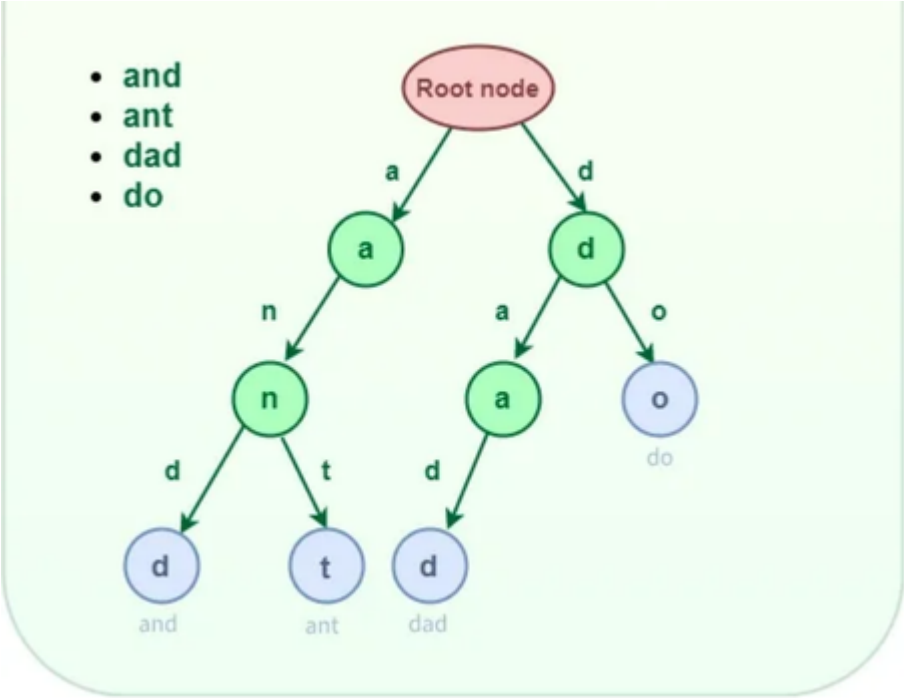


In this example, we can see a binary search tree with the root node as 8. Following the BST rules, all nodes in its left subtree contain smaller values, while all nodes in its right subtree contain larger values. The root's left child is 3, which has a left child of 1 and a right child of 6, with 6 further extending to a left child of 4. On the other side, the root's right child is 10, which has a right child of 14, and 14 has its own left child of 13.

What are Tries?

When using our phones to type words. When you type “c”, then “a”, the phone follows the letters one by one and suggests “car” or “cat”. This is how tries work.

A **trie** is a tree where each word is stored as a path of letters from the root, and a mark shows where a word ends. It is also known as a **prefix tree** or **radix tree**. This makes tries useful for autocomplete and word-search applications.



In this trie, each branch starts from the root node and represents an individual letter. As we move down, the letters are combined to form prefixes and become complete words. Each path represents a sequence of letters, while a marked node indicates the end of a complete word. For example, following the paths from the root allows us to form the words and, ant, dad, and do. Some words share the same letters at the beginning, allowing them to share branches and avoid storing repeated prefixes.

Advantages and Disadvantages

Binary Search Trees

Advantages:

- **Simple Structure:** Easy to implement and understand for basic hierarchical data representation.
- **Flexibility:** No ordering or balancing constraints, allowing nodes to be added freely.

Disadvantages:

- **Inefficient Searching:** Operations take O(n) time in the worst case because it lacks sorted ordering or self-balancing, meaning the heights of the left and right subtrees differ significantly.
- **Unpredictable Growth:** Can easily degrade into a linear linked list if unbalanced.

Tries

Advantages:

- **Fast Searching:** Searches take O(L) time, where L is the length of the word.

- **Prefix Searching:** Efficiently finds words with shared prefixes, making them useful for autocomplete.
- **Shared Prefixes:** Words with common beginnings can share branches, avoiding repeated storage.

Disadvantages:

- **High Memory Usage:** Each node may require multiple pointers, consuming additional memory.
- **Complex Implementation:** More complicated to implement than simpler data structures.
- **Limited Use:** Primarily designed for storing and searching strings.

Operations

Operation	Description
insertion	Insert values in a BST.
search	Search a given key in BST.
deletion	Delete the specified node from a BST.

Example

Build a BST. Insert 8, 3, 10, 1, 6 in this order. The first key becomes the root. Each next key walks down: smaller goes left, bigger goes right, until it finds an empty spot.

Insert	Path	Result
8	Tree is empty	8 becomes the root
3	3 < 8, go left	3 is left child of 8
10	10 > 8, go right	10 is right child of 8
1	1 < 8, left; 1 < 3, left	1 is left child of 3
6	6 < 8, left; 6 > 3, right	6 is right child of 3

Performance

Space Complexity

A BST has a space complexity of $O(N)$, as it stores n nodes, each containing data and pointers to its children. A trie has a space complexity of $O(N \times L)$, where N is the number of words and L is their maximum length, although shared prefixes can reduce memory usage.

Time Complexity

Operation	Best Case	Worst Case	Average
Access	O(logN)	O(N)	O(logN)
Search	O(logN)	O(N)	O(logN)
Insertion	O(logN)	O(N)	O(logN)
Deletion	O(logN)	O(N)	O(logN)

References

1. Cormen et al. Introduction to Algorithms, 3rd ed. Chapter 12 “Binary Search Trees”: introduction p. 286; §12.1 pp. 286-288 (BST property, inorder walk); §12.2 pp. 289-290 (search); §12.3 p. 294 (insertion).
2. Sedgewick, R. and Wayne, K. Algorithms, 4th ed. §5.2 “Tries” (for the trie part).
3. https://www.geeksforgeeks.org/cpp/cpp-binary-search-tree/

1.7 Heaps and Priority Queues

What is a Priority Queue ADT?

Imagine an emergency room. Patients arrive over time, but the nurse always treats the most urgent patient next, not the one who arrived first, and not the one who arrived last.

Three structures give three different answers to "which item do I get next?":

A. The Priority Queue ADT

The problem

Imagine an **emergency room**. Patients arrive over time, but the nurse always treats the *most urgent* patient next, not the one who arrived first, and not the one who arrived last.

Three structures give three different answers to "which item do I get next?":

Structure	Gives you next…	Rule
Queue	the *oldest* item	FIFO (first in, first out)
Stack	the *newest* item	LIFO (last in, first out)
Priority queue	the *most important* item	by priority, regardless of arrival order

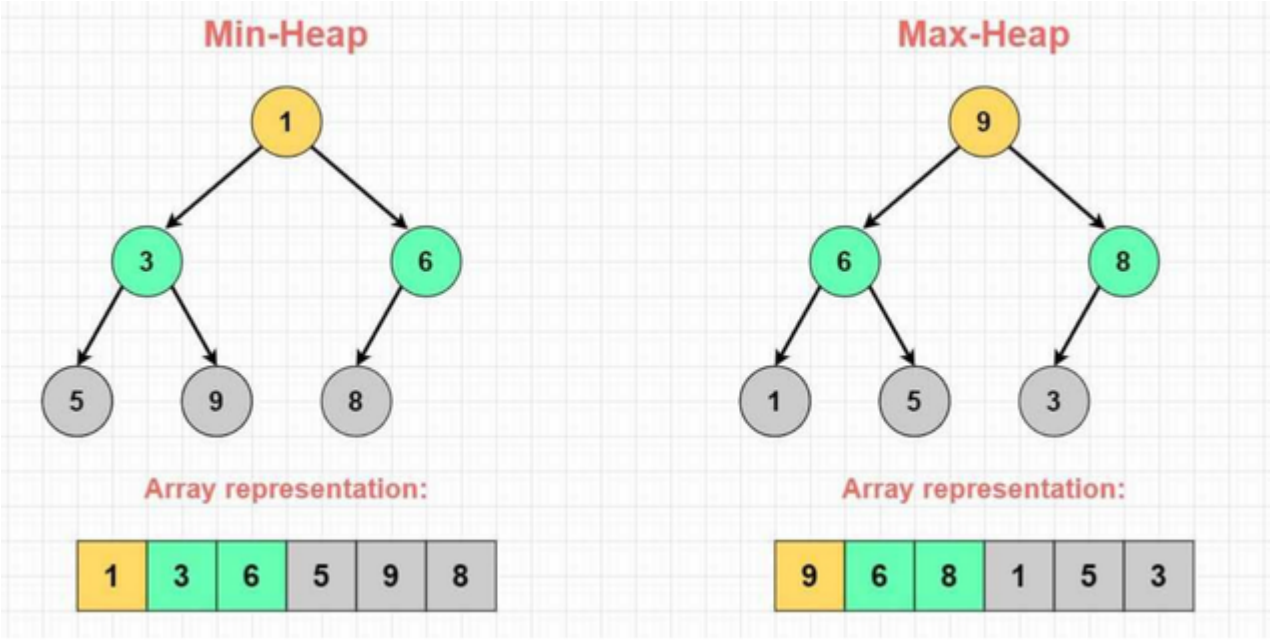
This need appears everywhere: operating-system schedulers (run the most urgent process), Dijkstra's shortest-path algorithm (visit the closest unvisited vertex), event simulations (handle the earliest event), Huffman coding (merge the two rarest symbols) and search engines (keep the best few results).

An **abstract data type (ADT)** is an interface, meaning a set of operations and what they promise. It says nothing about memory. You can think of it as a template for other data structures. For example, a priority queue is an ADT that defines operations like insert, remove, and peek, but it doesn't specify how these operations are implemented.

What are Binary Heaps?

A binary heap is a binary tree that keeps just enough order to answer **which item is the most extreme** (largest or smallest). It has two properties, mainly

- **Shape property** - every level is full, except possibly the last, which is filled from left to right
- **Heap-order property** - Every node’s key is ≤ for the keys of its children (for a min-heap)



These two properties makes binary heaps a *complete binary tree*, which makes it possible to represent it into an **array** form, where the **index** of the array determines the entry's **position** in the heap.

Advantages and Disadvantages

Advantages:

Binary heaps' main advantage is its **efficiency**:

- Retrieving the minimum or maximum element from the root takes `O(1)` time.
- Adding a new element (inserting) or extracting the root takes `O(logN)`
- It is stored in a array rather than node-pointer, which makes it space efficient.
- Using the build-heap pattern (bottom-up), constructing a heap takes linear time `O(N)`.
- Supports heapsort, which is an efficient sorting algorithm with minimal extra space.

Disadvantages:

- Inefficient for **general search**, since the tree is not fully sorted like a binary search tree.
- Because the elements are **partially ordered** (only parent-child order), no natural in-order traversal exists for general queries.
- Unstable sorting, where equal keys may not **preserve** their **initial** order during heapsort.
- Updating the priority of an arbitrary element takes `O(logN)` time since looking up an **arbitrary element** is not efficient in heaps.

Operations

Operation	What It Does	Returns
insert(item, priority)	Adds an item with the given priority	nothing
extract()	Removes and returns the highest priority item	the highest priority item
peek()	Returns the highest priority item without removing it	the item
size()	Counts the item stored	an integer

Example

An emergency room as a min-heap (smaller number = more urgent)

Step	Operation	Queue afterwards (name, priority)	Returned
1	"insert(Alice, 3)"	Alice 3	
2	"insert(Bob, 1)"	"Alice 3, Bob 1"	
3	"insert(Cara, 2)"	"Alice 3, Bob 1, Cara 2"	
4	extract()	"Alice 3, Cara 2"	Bob
5	"insert(Dan, 0)"	"Alice 3, Cara 2, Dan 0"	
6	extract()	"Alice 3, Cara 2"	Dan
7	extract()	Alice 3	Cara
8	extract()	(empty)	Alice

Performance

Space Complexity

A binary heap has a total space complexity of `O(N)`, as it is stored as an **array** and no pointer is needed, since it is a **complete** binary tree (the index of the array tells the position in the heap)

Time Complexity

Operation	Time	Why
Insert	worst `O(logN)`; best `O(1)`	at most $\log_2 n$ swaps (one per level); best case is a new item already its parent
Extract	`O(logN)`	at most $\log_2 n$ iterations, each doing 2 comparisons and 1 swap
Peek	`O(1)`	read a[0]

1.8 Graphs

What is a Graph?

Many real-world situations are really about connections. which cities are joined by a road, which people are friends, which tasks must be finished before others can start, which web pages link to each other. A graph is the simple picture we use to describe all of these.

A **graph** is a collection of dots joined by lines. Programmers say **vertices** (or nodes) for the **dots** and **edges** for the **lines**, and you will see both sets of words in books.

A graph is **not** a drawing. What matters is only which dots are joined to which. You could draw the same graph with the dots in different places, or with longer or shorter lines, and it would still be the same graph.

Graphs are important because they are **everywhere**, maps, friendships, computer networks, game moves, etc. Once a problem has been turned into a graph, we can use the few same recipe for graphs. And with these graphs, the amount of work can explode with a careless algorithm.

2. Graph Vocabulary and Modeling

Before any recipe, we need a shared vocabulary. Most of these words are everyday ideas with a precise name.

Word	Meaning	Everyday picture
Dot (vertex)	One "thing" in the graph	A city, a person, a task
Line (edge)	A connection between two dots	A road, a friendship, a hyperlink
Neighbours	Two dots joined directly by a line	Two friends

Word	Meaning	Everyday picture
Undirected	Lines work in both directions	A friendship, a two-way road
Directed	Every line is an arrow that goes only one way	"Follows" on social media, a one-way street, "must come before"
Weighted	Every line carries a number: a weight	Road length, travel time, ticket price
Degree	How many lines touch a dot (for arrows: how many go *in* and how many go *out*)	How many friends a person has
Path	A trip from one dot to another along lines. Its length is the number of lines, or the total weight in a weighted graph	A route on a map
Cycle	A path that comes back to the dot where it started	A loop you can walk around
Connected	You can get from every dot to every other dot	A country with no cut-off islands
Component	One separate "island" of a graph that is not connected	A group of friends no one else knows
Tree	A connected graph with no cycles	A family tree, a folder structure
DAG	A **d**irected graph with no cycles (acyclic graph)	Course prerequisites
Bipartite	The dots can be split into two teams so that every line joins one dot from each team	Students and the projects they could do

Representing Graphs

A computer cannot look at a drawing. We must store the dots and lines in memory in a way that lets our recipes ask questions quickly: Which dots are the neighbours of this one? Is there a line between these two? There are three common ways to store a graph. For example, for the graph below, we will take a look of the three representations.

Adjacency Matrix (A Grid)

Make a table with one row and one column for every dot. Put a 1 in the cell where a line joins the row dot to the column dot, and a 0 everywhere else.

```
  S  A  B  C  D  E  F
S 0  1  1  0  0  0  0
A 1  0  0  1  0  0  0
B 1  0  0  0  1  0  0
C 0  1  0  0  1  1  0
D 0  0  1  1  0  0  1
E 0  0  0  1  0  0  0
F 0  0  0  0  1  0  0
```

Adjacency List (A Short List per Dot)

Give every dot its own list of neighbors.

```
S : A, B
A : S, C
B : S, D
C : A, D, E
D : B, C, F
E : C
F : D
```

The Edge List (Just The Lines)

Keep one long list with a single entry per line.

```
(S,A) (S,B) (A,C) (B,D)
(C,D) (C,E) (D,F)
```

Situation	Good Choice	Why
A sparse graph (most real graphs)	Adjacency list	Small memory, exploring neighbours is quick
A dense graph, or a graph with only a few dozen dots	Adjacency matrix	Memory is no problem, and ""is there a line?"" is instant
Many questions of the type ""is X directly joined to Y?"	Adjacency matrix	One look per question
You are not sure	Adjacency list	It is the safe default for real-world data

Turning Real Problem into Graphs

Modeling is the most important skill in this chapter. To turn a problem into a graph, answer four questions:

- What are the **dots**?
- What are the **lines**?
- Do the lines have a **direction**?
- Do the lines carry a **number** (a weight)?

Example

If we want to find a driving route, then the dots of the graph are the **junctions**, whilst the lines of the graph are the **road sections**. it is a **directed** graph (if there are one-way streets) and the **weights** are the travel time or length.

If we want to create a graph of **university courses**, the dots are the courses, whilst the lines are the "must be taken before" relationship. It's a directed graph with no weights

If we want to assign students to projects, the dots are the **students and projects** whilst the lines are the *"this student could do this project"* relationship. Not a weighted graph, and it doesnt have weights (it's bipartite)

If we want to create a graph of **water** flowing through city pipes, the dots are the **junctions**, the lines are the **pipes**. It's a directed graph and the weights are the **capacity** of each pipe.

Performance

Time Complexity

For graphs there are two sizes to track, so we use two letters:

- V is the number of dots (V for vertices).
- E is the number of lines (E for edges).

Label	Plain-English meaning	Work for 1,000 dots and 5,000 lines
`O(V + E)`	Look at every dot and every line a fixed number of times	about 6,000
`O(E log V)`	Every line costs a little ""halving"" work, like searching sorted data	"about 50,000
`O(V × E)`	Every dot triggers a full pass over all the lines	about 5,000,000

Label	Plain-English meaning	Work for 1,000 dots and 5,000 lines
$O(V^3)$	Three nested loops over the dots	1,000,000,000 (a billion)

For each graph representations:

Question or action	Adjacency matrix	Adjacency list	Edge list
Memory used	One cell per pair: V x V	One entry per dot plus one per line end: about V + 2E	One entry per line: E
Add a line	1 step	1 step	1 step
Remove a line	1 step	Search the lists	Search the list
Growth of the memory	O(V^2)	O(V + E)	O(E)

1.9 Specialized Data Structures

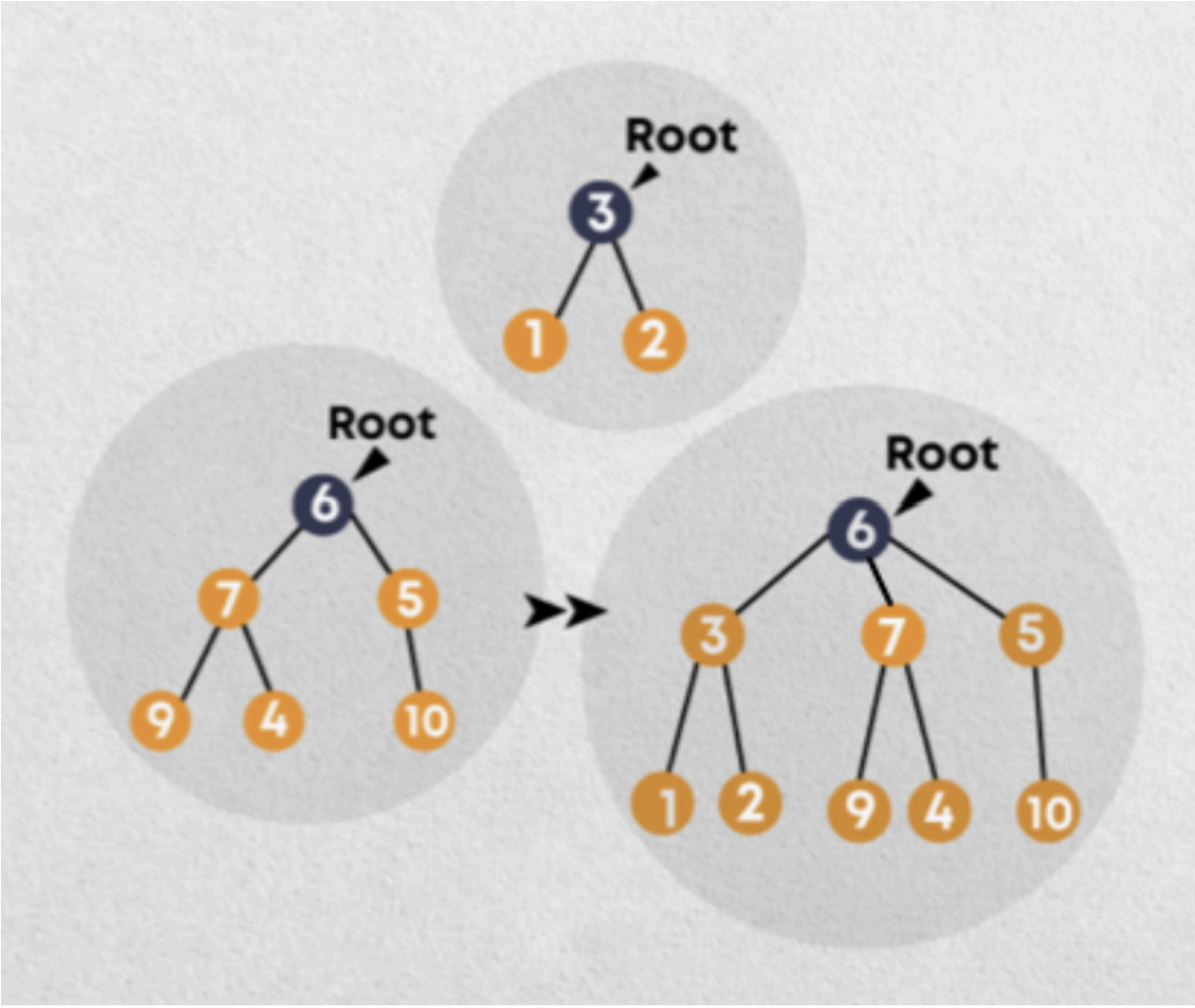
What are Specialized Data Structures?

Regular data structures works for lots of programming cases and usage, but they aren't always the most efficient. Specialized data structures are designed to solve certain problems more efficiently than regular structures such as arrays, linked lists, stacks, queues, and others. They organize and store data according to specific requirements, making certain operations be performed faster or with less memory usage.

Disjoint Set Union (DSU)

Disjoint Set Unions (DSUs) are used to find which elements belong to the same group. This data structure enables the merging of separate sets and addressing different inquiries concerning them, such as determining if elements a and b belong to the same set and finding out the size of the set.

Two sets are called disjoint sets if they don't have any element in common. The disjoint set data structure is used to store such sets. DSU maintains a collection of separate sets, or groups which are represented by trees. Each group has a representative element called its root or representative.



In this example, we can see two separate Disjoint Set Union (DSU) trees being merged into a single set. At the top, one tree has root node 3 with child nodes 1 and 2. On the bottom left, another tree has root node 6, with child nodes 7 and 5, where 7 further branches into 9 and 4, and 5 branches into 10.

The arrow shows the union operation combining these two sets. In the resulting tree on the bottom right, root node 6 absorbs the entire tree rooted at 3 as its direct child, successfully merging the two disjoint sets into a single component.

Advantages and Disadvantages

Advantages:

- **Fast Operations:** With optimizations, operations take almost zero time `O(1)`.
- **Great for Groups:** Perfect for tracking connected groups, like finding if two items belong to the same network.
- **Short Code:** Simple to write in code.

Disadvantages:

- **Hard to Split:** Once you merge two sets, it's very hard to split them back apart or undo a union.
- **Not for Sorting:** It can't sort items or help you search for a specific value inside a group.

Operations

Operation	Description
find	Locates the representative element (root) of a disjoint set by following parent pointers.
union	Merges two separate disjoint sets into a single combined set.

Optimisations can help DSU to be more efficient:

- Path compression, when finding an element's root, make the elements encountered point directly to that root. For example, D->C->B->A can be shortened to just D->A.
- Union by size or rank, When merging two groups, attach the smaller or shallower tree to the larger or taller one. This helps prevent unnecessarily long chains.

Performance

Space Complexity

A disjoint set union `O(N)`, as it stores each of the n elements in separate parent and rank/size arrays.

Time Complexity

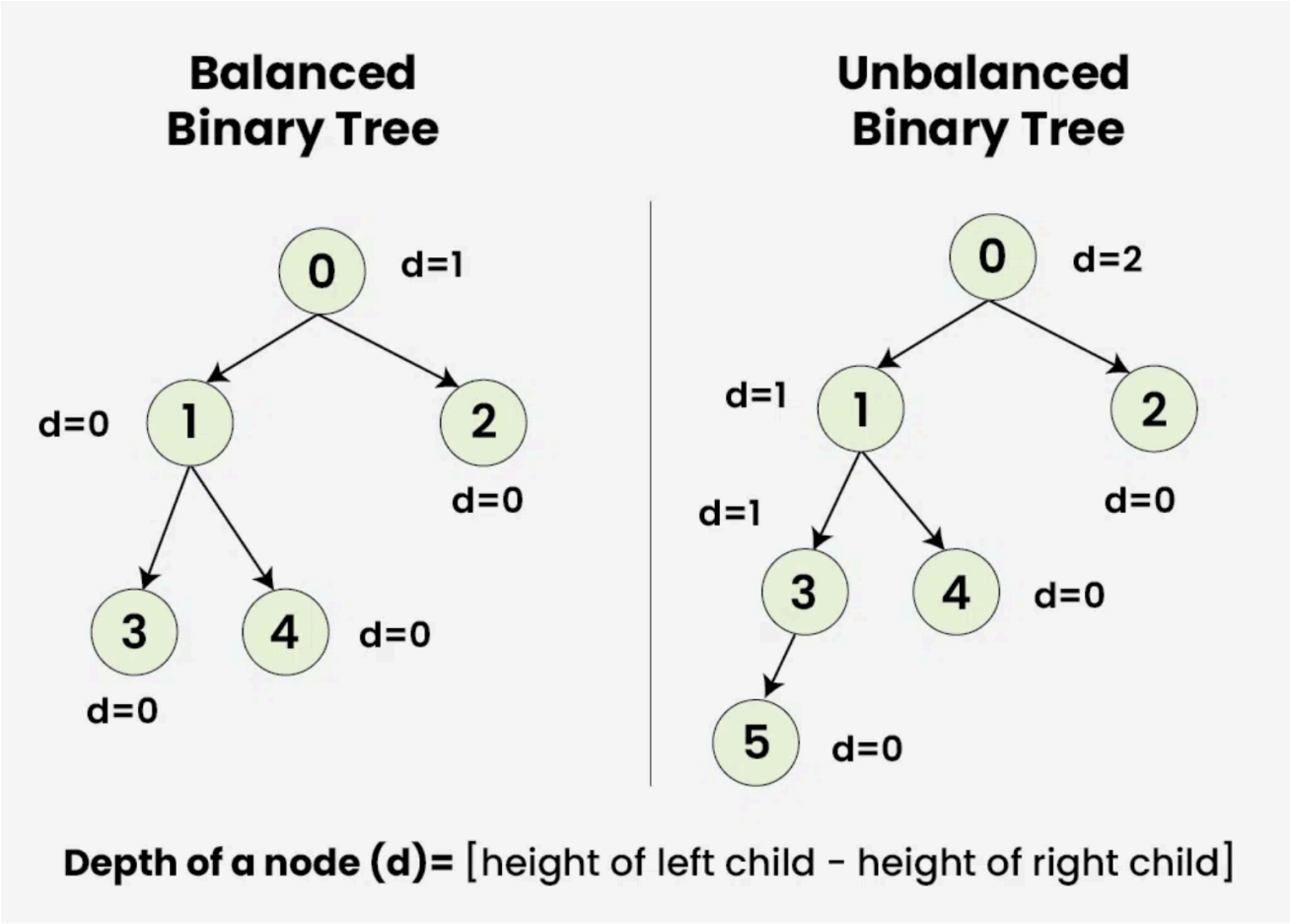
Operation	Best Case	Worst Case	Average
Find	`O(1)`	`O(α(N))`	`O(α(N))`
Union	`O(1)`	`O(α(N))`	`O(α(N))`

Binary Search Trees (BST)

Binary Search Trees (BSTs) organizes values based on three rules:

- Values smaller than a node go to its left.
- Values larger than a node go to its right.
- Each node has at most two children.

However, BSTs aren't always well-shaped or divided properly. Unbalanced BSTs can make search times much longer than it has to be, that is why Balanced Search Trees tries to keep the tree's height small. When a tree becomes unbalanced, it rearranges its nodes while still following the BST rules.



In the image, we compare a balanced binary tree with an unbalanced binary tree. We get the depth of a node (d) by calculating the height difference between its left and right subtrees. On the right, the tree is unbalanced because a left-heavy skew causes the root 0 to have a depth of d=2, as the left branch grows significantly deeper than the right.

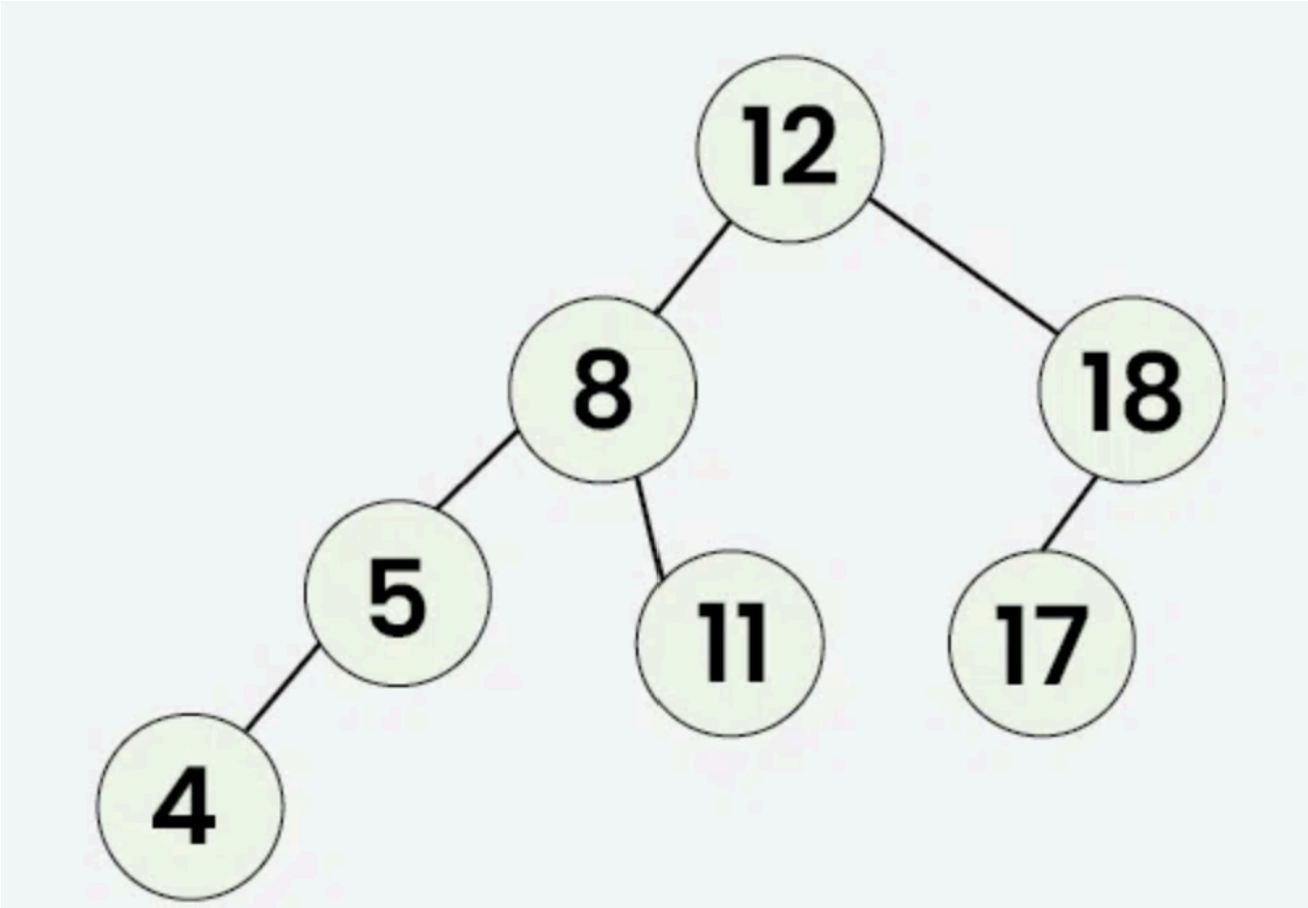
There are three conditions to check if a Binary Search Tree is balanced:

- The absolute difference between heights of left and right subtrees at any node should be less than or equal to 1.
- For each node, its left subtree should be a balanced binary tree.
- For each node, its right subtree should be a balanced binary tree.

There are two well-known self-balancing BSTs:

AVL Trees

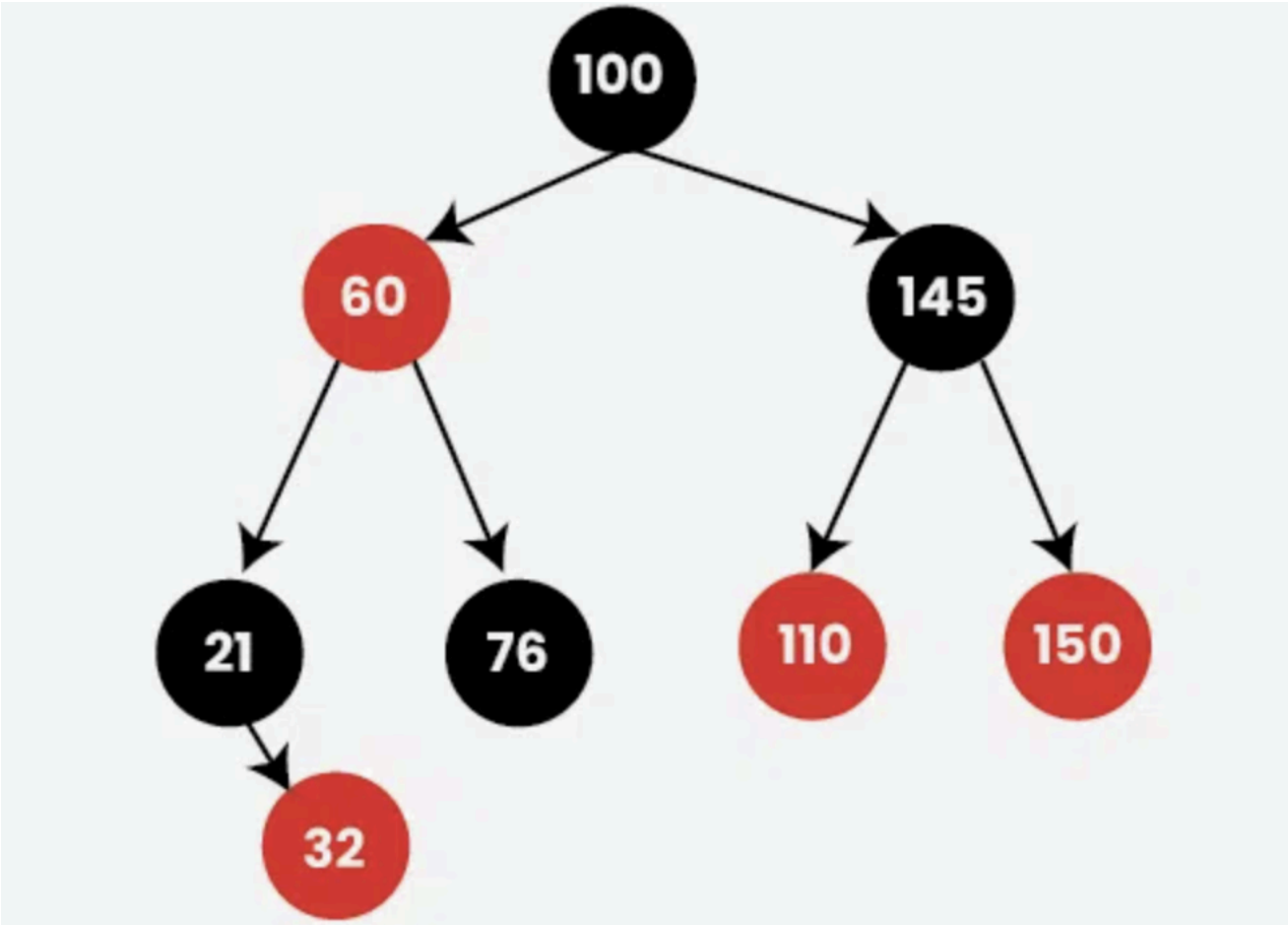
A self-balancing binary search tree where the difference between heights of left and right subtrees cannot be more than one for all nodes.



Red-Black Trees

A self-balancing binary search tree where each node has an additional attribute: a color, which can be either red or black. The primary objective of these trees is to maintain balance during insertions and deletions, ensuring efficient data retrieval and manipulation. Red-black trees have 5 special properties to ensure the tree is balanced:

- Every node is colored either red or black.
- The root of the tree is always black.
- Every leaf node (nodes without children) that are represented with NIl is black.
- If a node is red, then both of its children must be black.
- Every single path or branch from node x down to any of its descendant NIL leaves must contain the same number of black nodes, and you count the nodes on the path excluding the parent node x itself.



Advantages and Disadvantages

AVL Tree

Advantages

- **Maximum Search Speed:** Strict balancing rules guarantee the absolute fastest lookup and search times.

Disadvantages

- **Slower Writes:** Strict balance rules trigger frequent rotations during insertions and deletions, making write operations slower.

Red Black Tree

Advantages

- **Faster Inserts and Deletes:** Looser balancing requires fewer rotations, making write operations much faster.

- **Slightly Slower Lookups:** Less strict balancing can mean taking a few more steps during searches compared to AVL trees.
- **Extra Memory:** Every single node requires an extra bit of storage just to keep track of its color attribute.

Operations

Operations are the same as normal binary search trees (BSTs).

Performance

Space Complexity

A binary search tree O(n), as it stores n nodes, each containing data and pointers to its child nodes.

Time Complexity

AVL Tree

Operation	Best Case	Worst Case	Average
Access	`O(logN)`	`O(logN)`	`O(logN)`
Search	`O(logN)`	`O(logN)`	`O(logN)`
Insertion	`O(logN)`	`O(logN)`	`O(logN)`
Deletion	`O(logN)`	`O(logN)`	`O(logN)`

Red Black Tree

Operation	Best Case	Worst Case	Average
Access	`O(logN)`	`O(logN)`	`O(logN)`
Search	`O(logN)`	`O(logN)`	`O(logN)`
Insertion	`O(logN)`	`O(logN)`	`O(logN)`
Deletion	`O(logN)`	`O(logN)`	`O(logN)`

References:

- https://www.geeksforgeeks.org/dsa/introduction-to-disjoint-set-data-structure-or-union-find-algorithm/
- https://www.w3schools.com/dsa/dsa_data_binarytrees.php
- https://www.geeksforgeeks.org/dsa/balanced-binary-tree/

2.1 Foundations of Algorithms

What is an Algorithm?

An algorithm is a step-by-step computational procedure designed to transform a given set of inputs into a predictable outputs. It acts as a systematic tool for solving specific computational problems by detailing the exact operations a computer must perform.

Key characteristics of an algorithm include:

- **Input:** Zero or more defined values or datasets provided to the algorithm.
- **Output:** One or more results corresponding to the calculated inputs.
- **Definiteness:** Every step must be clear, unambiguous, and precisely specified.
- **Finiteness:** The procedure must terminate after a finite number of steps for all valid inputs to avoid endless loop.
- **Effectiveness:** Each operation must be basic, practical, and feasible enough to be executed using available resources.
- **Generality:** It should be able to solve an entire class of problems with varied inputs, rather than just one specific case.
- **Language Independence:** An algorithm is a conceptual plan; it is independent of any specific programming language and can be written in pseudocode, flowcharts, or plain prose.

Formal Problem Statements and Problem Instances

To analyze an algorithm mathematically, a computational problem must be formally stated in terms of its required input and desired output properties.

Example: The Sorting Problem

- **Input:** A sequence of n numbers represented as:

$$\langle a_1, a_2, \ldots, a_n \rangle$$

- **Output:** A reordering (permutation) of the input sequence such that the elements satisfy the ordering relationship:

$$a'_1 \leq a'_2 \leq \cdots \leq a'_n$$

Instance of a Problem

A **problem instance** refers to a specific set of input data that satisfies all constraints defined by the problem statement.

- **Example Input Instance:**

$$\langle 31, 41, 59, 26, 41, 58 \rangle$$

- **Expected Output Instance:**

$$\langle 26, 31, 41, 41, 58, 59 \rangle$$

Correctness and Efficiency

Correctness

Correctness

An algorithm is considered correct if, for every valid input instance, it eventually halts and produces the exact specified output.

- **Incorrect Algorithms:** Algorithms that either fail to terminate (enter an infinite loop) or halt with an invalid result.
- **Controlled Error Rates:** In specific specialized domains (such as probabilistic primality testing or randomized search), algorithms with small, mathematically bounded error rates can still be practical and useful.

Efficiency

Algorithm efficiency measures how well an algorithm utilizes computational resources—primarily time and memory—as the scale of the problem increases. An algorithm must be efficient enough to execute within acceptable time and resource limits. When selecting or analyzing an algorithm, efficiency is governed by several core factors such as input size (n) and the volume of data being processed. As n grows, the execution time or memory footprint changes according to the algorithm's growth rate. There is also initial structures which also affect the extent to which the data is already organized or formatted prior to processing. For example, sorting an already sorted array takes far fewer operations than sorting a completely randomized array.

Selecting the Right Algorithm

Choosing the optimal algorithm for a given task depends on several trade-offs:

- **Input Size:** The volume of data (n) being processed.
- **Initial Structure:** The extent to which the data is already organized or partially sorted.
- **Hardware Constraints:** Available main memory (RAM), storage bandwidth, and CPU architecture.

Modern Computing and Parallelism

Historically, performance improvements relied on increasing single-core processor clock speeds. Due to physical power and thermal limitations, modern processor architectures rely heavily on **multi-core designs**.

To maximize computational performance on multi-core hardware, algorithms must be designed with **parallelism** in mind, breaking down tasks into independent threads that execute concurrently across multiple processing units.

2.2 Correctness Complexity

What is Algorithm Correctness?

An algorithm is correct if it produces the expected output for every valid input.
This breaks down into two levels:

- **Partial Correctness:** If the algorithm finishes running, the output is guaranteed to be correct.
- **Total Correctness:** The algorithm is partially correct and it is mathematically guaranteed to teminate (it will nver get stuck in an infinite loop).

To prove correctness, computer scientists use structural logic:

- **Preconditions and Postconditions:** The state that must exist before the algorithm runs and the state guaranteed after it finishes must be true.
- **Loop Invariants:** A property that remains true before, during, and after every iteration of a loop. Proving an invariant usually relies on mathematical induction.
- **Variants:** A quantitative measure that strictly decreases during every loop iteration. When it reaches a baseline (like zero), the loop stops, guaranteeing termination.

Structural Correctness in Data Structures

Think of an invariant like a strict rule a data structure must always follow to stay correct. For example, imagine a line of people ordered from shortest to tallest. If a short person cuts into the middle, the rule is temporarily broken, and you must swap them until the correct order is restored. Similarly, inserting or deleting items in a data structure disrupts its internal rules. The algorithm must always fix these violations before finishing, or else the data structure becomes corrupted and loses its efficiency guarantees.

What is algorithm complexity?

Complexity abstracts away specific hardware speeds to measure how an algorithm's resource consumption scales as the input size n grows.

- **Time Complexity:** The number of basic operations (comparisons, assignments, arithmetic) executed.
- **Space Complexity:** The amount of auxiliary memory required beyond the input itself.

Asymptotic Complexity Classes

Different algorithmic paradigms and structures yield vastly different growth rates:

- **Constant Time** - `O(1)`: Operations take the same time regardless of input size (e.g., accessing an array element by index, hash table lookup).

- **Logarithmic Time** - `O(logN)`: The problem size is halved at each step (e.g., Binary Search). Analogy: Finding a word in a dictionary by repeatedly opening it to the middle.
- **Linear Time** - `O(N)`: Execution time grows directly in proportion to the input size (e.g., finding the maximum element in an unsorted array via a single pass).
- **Linearithmic Time** - `O(NlogN)`: Common in efficient comparison-based sorting algorithms like Merge Sort and Quicksort, which repeatedly partition data and merge results.
- **Quadratic Time** - `O(N^2)`: Typically arises from nested loops where every element is compared against every other element (e.g., Bubble Sort, Insertion Sort, or checking all pairs in a matrix).
- **Exponential Time** - `O(2^N)`: Explodes rapidly with input size, often seen in naive recursive solutions like finding the n-th Fibonacci number without memoization.
- **Factorial Time** - `O(N!)`: The ultimate bottleneck, occurring in brute-force solutions that generate all possible permutations, such as the naive Traveling Salesperson Problem (e.g., visiting n cities in every possible order).

> n is the number of elements or nodes depending on the usage.

Asymptotic Notation (Big O, Big Omega, Big Theta)

We use Asymptotic Notation to categorize this scaling into bounds:

- **Big O** (O): The asymptotic upper bound (worst-case scenario). It guarantees that the algorithm will not exceed this growth rate.
- **Big Omega** (Ω): The asymptotic lower bound (best-case scenario). It establishes the minimum time or space required.
- **Big Theta** (Θ): A tight bound where the upper and lower bounds match, meaning the function grows at a precise predictable rate from both above and below.
- **Little o** (o) and Little omega (ω): Strict upper and lower bounds (where growth is strictly less than or greater than, without equality).

This graph below show the best-case and worst-case based on the complexity for data structures and algorithms:



Role of Data Structures

Algorithms and data structures are inseparable because the layout of the data directly dictates the complexity bounds.

Complexity Trade-offs

- If you need `O(1)` constant time lookups, you might choose a Hash Table, but you sacrifice memory to get it.
- If you need memory efficiency and strictly ordered data, you might use a Binary Search Tree, which slows lookups to `O(log N)`.

2.3 Searching and Sorting

Almost every program works with collections of data: contacts, products, scores, messages. Two jobs come up again and again: finding a particular item, and putting items in order. This chapter is about those two jobs and about the step-by-step recipes (called algorithms) that computers use to do them.

What is Searching?

Searching means looking through a collection to find one particular item, or to discover that it is not there. The item we are looking for is called the target.

This is similar to finding a friend's number in your phone, looking up a word in a dictionary, spotting your suitcase on an airport belt, or pressing "Find" inside a document.

Every search ends in one of two ways:

- **Found**: we learn where the target is (its position in the collection).
- **Not found**: we learn for certain that the target is not in the collection.

What is Sorting?

Sorting means arranging items in a chosen order: smallest to largest (ascending), largest to smallest (descending), A to Z, oldest to newest. The detail used to decide the order, such as a price, a name or a date, is called the key.

```
For example:
Before sorting: [42 7 19 3 25]
After sorting: [3 7 19 25 42] (ascending: smallest to largest)
```

This is similar to a leaderboard ranked by score, an online shop showing "price: low to high", contacts in alphabetical order, or e-mails with the newest first.

Searching Algorithms

A searching algorithm answers one question: "*Is the target in this collection, and if so, where?*" There are two classic recipes. The first works on any list. The second needs the list to be sorted, and in return it is far faster.

Linear Search

If we have a list in no particular order and want to know whether a target is in it, and where. To solve it, we want to check the items one by one until you find it or run out.

This is what you do when you look for your keys in a messy drawer, or run a finger down an unsorted guest list to find a name. Because the order tells us nothing, there is no shortcut, and any item could be the one.

How it works:

- Start at the first item.
- Compare it with the target. If they match, stop: found.
- If not, move on to the next item and repeat step 2.
- If you reach the end of the list without a match, stop: not found.

Example

With the list `8 3 12 5 9 1`, find the target `5`.

```
Look 1:  [8]  3   12   5   9   1   8 is not 5  -> keep going
Look 2:   8  [3]  12   5   9   1   3 is not 5  -> keep going
Look 3:   8   3  [12]  5   9   1   12 is not 5 -> keep going
Look 4:   8   3   12  [5]  9   1   5 is 5      -> FOUND at position 4
```

If the target were `7` (which is not in the list), we would have to look at all six items before we could say "not found".

Performance

- **Best-case time complexity:** `Ω(1)` (Constant time, when the target is the first item)
- **Worst-case time complexity:** `O(N)` (Linear time, when the target is last or not found)
- **Average-case time complexity:** `Θ(N)` (Linear time, when the target is in the middle)
- **Space complexity:** `O(1)` (Constant space, as no extra memory is needed)

Binary Search

Imagine we have a list that is already sorted and want to find a target fast, even if the list is enormous. The core idea is that with every look should throw away half of what is left.

You may have played this game: "I'm thinking of a number between 1 and 100." You do not guess 1, then 2, then 3. You guess 50, hear "higher" or "lower", and instantly eliminate half of the numbers. A paper dictionary works the same way: you open it near the middle, see whether your word comes before or after that page, and continue in the correct half.

How it works:

- Look at the middle item of the part of the list that is still possible.
- If it matches the target, stop.

- If the target is smaller than the middle item, it can only be in the left half. Throw away the middle item and everything to its right.
- If the target is bigger, it can only be in the right half. Throw away the middle item and everything to its left.
- Repeat on the half that remains. If nothing remains, then the target is not in the list. (If the remaining part has an even number of items, there are two "middles". Either one works, ust pick the same one every time.)

Why the data must be sorted?
Steps 3 and 4 are conclusions drawn from the order: "the middle is 46 and we want 30, so 30 can only be on the left, because everything on the right is 46 or bigger". If the list is not sorted, that conclusion is false, and the algorithm may throw away the very item it is looking for.

```
Unsorted list:   20   50  [10]  40   30   target = 20, middle = 10
20 is bigger than 10, so binary search throws away the LEFT half ...... which is ex
```

Binary search is only correct on sorted data. The sorting is what earns us the right to throw half of the list away on every look.

Example

Imagine we are given a sorted list of 15 numbers, we need to find `58`. Try to predict the output based on the instructions on how to do binary search.

```
Round 1:   3   8   14  21  27  33  39 [46] 52  58  64  71  79  85  93
                                          ^ middle = 46; 58 is bigger -> keep the R

Round 2:                                   52  58  64 [71] 79  85  93
                                                       ^ middle = 71; 58 is small

Round 3:                                   52 [58] 64
                                                 ^ middle = 58 -> FOUND
```

Only 3 looks, where linear search would have needed 10. What if the target is `60`, which is not in the list? Round 3 looks at `58`, `60` is bigger, so keep the right side, which is just `64`. Round 4 looks at `64`, `60` is smaller, so keep the left side, which is now empty. Nothing is left to check, so the answer is not found, after 4 looks.

Performance

- **Best-case time complexity:** $\Omega(1)$ (Constant time, when the middle element is the target on the first check)

- **Worst-case time complexity:** `O(logN)` (Logarithmic time, when the search space is reduced to 1 or the element is not found)
- **Average-case time complexity:** `O(N)` (Logarithmic time, when the target is found after dividing the search space)
- **Space complexity:** `O(1)` (Constant space for iterative implementation, or `O(logN)` for recursive due to the call stack)

Linear Search vs. Binary Search

Aspects	Linear Search	Binary Search
Needs sorted data?	No	Yes
Core move	Look at items one by one	Look at the middle, throw away half
Worst case, 1,000,000 items	1,000,000 looks	about 20 looks
Growth	Proportional	Halving
Best when	The list is small or unsorted, or you search once	The list is big and searched many times

Is Sorting Worth It?

Binary search is fast, but sorting has a price. Suppose a list has 1,000 items, sorting it costs roughly 10,000 steps, a linear search costs up to 1,000 looks, and a binary search costs up to 10 looks. (These are rough step counts for illustration, not exact timings.)

Number of Searches	Linear Search Only	Sort Once, then Binary Search
1	1,000	10,000 + 10 = 10,010
10	10,000	10,000 + 100 = 10,100
100	100,000	10,000 + 1,000 = 11,000
1,000	1,000,000	10,000 + 10,000 = 20,000

For a single search, linear search wins. After roughly ten searches the two break even, and from then on sorting first pays off more and more. If the data changes constantly, other structures built for fast lookups (such as hash tables or search trees) are often a better fit, and you will meet them later in your DSA journey.

Learning Check

> Which search would you use in each case?
> (a) Checking once whether a name appears in a list of 20 names that arrived today.
> (b) A website that must answer "is this username taken?" millions of times, against a list of 5 million names that rarely changes.
>
> **Solution.**
> (a) **Linear search.** The list is tiny, you search once, and sorting would cost more than it saves.
> (b) **Sort once, then binary search.** Each check needs at most about 23 looks instead of up to 5 million, and the one-time sorting cost is quickly repaid.

Sorting Algorithms

Sorting sounds simple, yet there are dozens of different recipes for it. Each one makes different trade-offs between **speed**, **memory** and **behavior on tricky input**. Before meeting them, here are the words we will use to judge them.

Term	Meaning
Comparison	Asking "which of these two items should come first?"
Swap	Exchanging the places of two items
Pass	One full sweep through the list (or through the part that is still being worked on)
In place	Sorting inside the list itself, using only a tiny amount of scratch space (no big second copy)
Stable	Items with **equal keys** keep the same order relative to each other as before the sort

Why does stability matter?
Imagine students already listed alphabetically, which you then sort by grade. Two students have the same grade. A stable sort keeps them in alphabetical order. An unstable sort may shuffle them.

Example

```
Before:                          Alice (B)   Bob (A)   Carol (B)   Dan (A)

After a STABLE sort by grade:    Bob (A)   Dan (A)   Alice (B)   Carol (B)     <- ti
After an UNSTABLE sort, maybe:   Dan (A)   Bob (A)   Carol (B)   Alice (B)     <- ti
```

The algorithms in this section fall into two families:

- **Simple sorts** (bubble, selection, insertion): easy to understand, but the work grows like the **square** of the list size, so they are only comfortable on small lists.

- **Efficient sorts** (merge, quick, heap): cleverer, with work growing only a little faster than the list size `O(NlogN)`. They are the ones used on large data.

These algorithms can be analogized as follows:

Algorithm	Everyday picture
Bubble sort	Bubbles rising: big items drift to the end as neighbours are fixed
Selection sort	Lining people up by height: repeatedly pick the shortest and send them to the front
Insertion sort	Sorting playing cards in your hand: slide each new card into its place
Merge sort	Zipping two sorted piles of papers into one
Quick sort	Picking a reference person: shorter people go left, taller people go right
Heapsort	A tournament: the champion is taken off the top, again and again

The following three simple sorts all use the same list so that you can compare them: `5 3 8 1 4`, sorted in **ascending** order.

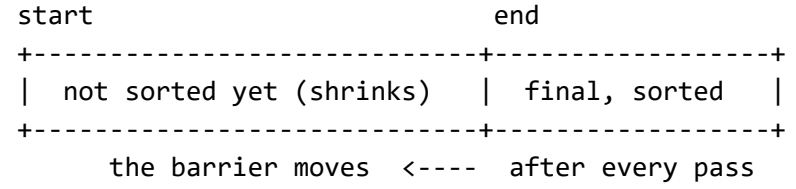
Bubble Sort

If we want to put a list in order using the simplest idea possible, with no extra memory. The core idea is to keep fixing out-of-order neighbours, and the biggest items "bubble up" to the end. Think of bubbles in a fizzy drink: the bigger ones rise fastest. Here, the largest item keeps getting carried to the right until it reaches the end. We never look at the whole list at once. We only ever ask one tiny question: "are these two neighbours in the right order?"

Comparing and Swapping Adjacent Elements

One pass works like this:

- Start at the left end and compare the first two items.
- If the left one is bigger, swap them. Otherwise, leave them alone.
- Slide one step to the right and compare the next pair. Keep going to the end of the part that is still unsorted. After one pass, the largest item has been carried all the way to the end: it is in its final place. The next pass does the same for the second largest, and so on. If a whole pass makes no swaps at all, the list is already in order and we can stop early.



> **Key idea** is that each swap fixes exactly one pair of neighbours that were in the wrong order. Bubble sort is nothing more than "repeat that until no pair is wrong".

Example

```
Start:                      5  3  8  1  4

Pass 1
  compare 5,3  -> swap      3  5  8  1  4
  compare 5,8  -> fine      3  5  8  1  4
  compare 8,1  -> swap      3  5  1  8  4
  compare 8,4  -> swap      3  5  1  4  8        <- 8 is now in its final place

Pass 2
  compare 3,5  -> fine      3  5  1  4  8
  compare 5,1  -> swap      3  1  5  4  8
  compare 5,4  -> swap      3  1  4  5  8        <- 5 is final

Pass 3
  compare 3,1  -> swap      1  3  4  5  8
  compare 3,4  -> fine      1  3  4  5  8        <- 4 is final

Pass 4
  compare 1,3  -> fine      1  3  4  5  8        no swaps this pass -> DONE
```

Performance

- **Best-case time complexity:** `O(N)` (when the list is already sorted, detected in a single pass using a "swapped" boolean flag).
- **Worst-case time complexity:** `O(N^2)` (quadratic time, when the list is in reverse order).
- **Average-case time complexity:** `Θ(N)` (quadratic time, for randomly ordered elements).
- **Space complexity:** `O(1)` (constant space, as it rearranges elements in place using only a temporary variable for swaps).
- **Stability:** Stable (equal elements never pass each other since swaps only occur for strictly out-of-order elements)

Strengths and limits

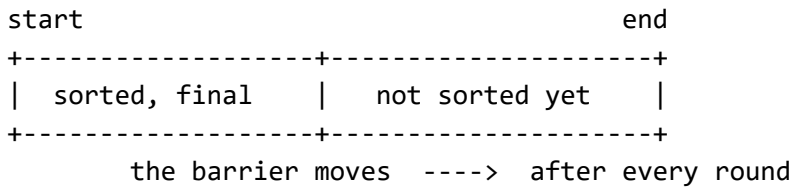
- Very easy to understand and to check, stable, in place.
- Each swap only moves an item one step, so it needs a huge number of swaps on big or scrambled lists.
- It is mostly used for **teaching**. In his classic book on sorting, Donald Knuth judged that it has very little to recommend it beyond its catchy name, because other simple sorts do the same job better.

Selection Sort

Imagine we want to sort a list while moving items around as little as possible. The core idea is to repeatedly pick the smallest remaining item and put it next in line. Picture lining up students by height. You look at the whole group, pick the shortest person, and send them to the front. Then you look at everyone who is left, pick the shortest again, and put them second. And so on.

Finding and Placing the Smallest Element

- Split the list, in your mind, into a sorted part on the left (empty at first) and an unsorted part on the right.
- Scan the whole unsorted part to find its smallest item.
- Swap that item with the first item of the unsorted part. It is now in its final place.
- The sorted part has grown by one. Repeat until only one item is left (it must already be in the right place).



> **Key idea** is that selection sort *searches* for the right item to put in each position, instead of fixing neighbours as bubble sort does. It makes **at most one swap per round**.

Example

```
Start:       5  3  8  1  4
Round 1:    [1] 3  8  5  4        smallest of all five is 1 -> swap it with the fir
Round 2:    [1  3] 8  5  4        smallest of (3 8 5 4) is 3 -> already in place
Round 3:    [1  3  4] 5  8        smallest of (8 5 4) is 4 -> swap it with 8
Round 4:    [1  3  4  5] 8        smallest of (5 8) is 5 -> already in place
Done:        1  3  4  5  8
```

> The brackets mark the sorted part.

Performance

- **Best-case time complexity:** `O(N^2)` (quadratic time, as it scans the remaining unsorted portion to find the minimum element regardless of initial order).
- **Worst-case time complexity:** `O(N^2)` (quadratic time, for unsorted elements).
- **Average-case time complexity:** `Θ(N)` (quadratic time)
- **Swaps:** At most `O(N)` swaps total (`N-1` swaps maximum), making it useful when memory writes are exceptionally expensive.

- **Space complexity:** `O(1)` (constant space, performing all operations in place).
- **Stability:** Unstable (long-distance swaps can jump an element past an identical twin, altering their relative order).

Strengths and limits

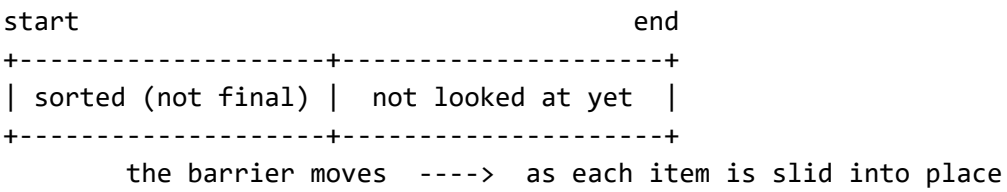
- Predictable and simple, and it moves items very few times. That helps when **moving** an item is expensive (for example, big records, or memory that wears out when written).
- It is **not faster on sorted input**, it is not stable, and it is slow on large lists.
- **A connection to heapsort** Heapsort (below) is often described as an *improved selection sort*: it also repeatedly extracts the largest item, but it uses a heap to find that item quickly instead of scanning the whole unsorted part.

Insertion Sort

We want to sort a list that is built gradually, or that is already almost in order, without wasted effort. The core idea is to take the next item and slide it into the right spot among the items already sorted. This is exactly how most people sort playing cards in their hand. The cards in your left hand are already in order. You pick up the next card with your right hand and slide it in between the cards where it belongs.

Building a Sorted Portion One Element at a Time

- Treat the first item as a sorted portion of size one.
- Take the next item from the unsorted part.
- Compare it with the sorted items, from right to left. Each item that is bigger is shifted one place to the right, which opens a gap.
- Drop the new item into the gap. The sorted portion has grown by one.
- Repeat until the unsorted part is empty.



> **Key idea** is unlike selection sort, the sorted part is **not final**: a later item may still slide in front of items that are already there. But it is always **in order among itself**.

Example

```
Start:      5 | 3  8  1  4              (sorted part | unsorted part)
Take 3:     3  5 | 8  1  4              3 is smaller than 5: shift 5 right, dro
Take 8:     3  5  8 | 1  4              8 is bigger than 5: nothing to shift, i
Take 1:     1  3  5  8 | 4              shift 8, 5 and 3 right; 1 goes to the f
Take 4:     1  3  4  5  8 |             shift 8 and 5 right; 4 drops in after 3
```

Performance

- **Best-case time complexity:** `O(N)` (when the list is already sorted, requiring only one comparison per element).
- **Worst-case time complexity:** `O(N^2)` (quadratic time, when the list is in reverse order).
- **Average-case time complexity:** `O(N^2)` (quadratic time, for randomly ordered elements).
- **Space complexity:** `O(1)` (constant space, as it works in place).
- **Stability:** Stable (equal elements never jump past each other).

Strengths and limits

- Simple, stable, in place, and very fast on small or nearly sorted lists. It can also sort items as they arrive.
- Slow on large, scrambled lists.
- It is used inside real libraries: Python's built-in sort, **Timsort**, is a hybrid that uses insertion sort on short stretches and merging for the rest.

The Simple Sorts at A Glance

	Bubble sort	**Selection sort**	**Insertion sort**
Core move	Swap out-of-order neighbours	Pick the smallest, place it	Slide the next item into the sorted part
Already sorted list	Fast (with the early stop)	No faster	Very fast
Nearly sorted list	So-so	No faster	Very fast
Moves of items	Very many	Very few	Many (shifts)
Stable?	Yes	No	Yes
Verdict	Mostly for teaching	When moving items is costly	The best of the three in practice

Heapsort

If we want to sort an array in place, with a guaranteed `O(NlogN)` running time, no extra memory and no recursion. Each classic sort gives up at least one of these:

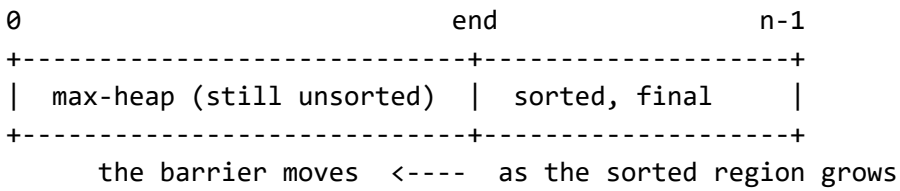
Algorithm	Worst-case time	Extra space	Stable?
Insertion sort	`O(N^2)`	`O(1)`	yes
Merge sort	`O(N\logN)`	`0(n)`	yes
Quicksort	`O(N^2)`	`O(logN)` stack (typical)	no
Heapsort	`0(N\logN)`	`O(1)`	no

Heapsort was published by J. W. J. Williams in 1964, and R. W. Floyd quickly improved the build phase to linear time (Section D). It is one of the few comparison sorts that offers all three guarantees at once.

Repeatedly Move the Maximum into its Final Position

A max-heap keeps the largest item at the root. Sorting in ascending order means the largest item belongs in the last position. So:

- Turn the array into a max-heap.
- Swap the root with the last item of the heap. That item (the maximum) is now in its final position.
- Shrink the heap by one slot (the final item is now outside it), sift-down the new root to repair the heap, and repeat.



> **Why a max-heap for ascending order?**
> Because each extraction leaves the maximum at the end of the array. A **min**-heap would put the minimum there and give a **descending** result.

Stability: Heapsort is Not Stable

Tag equal keys as 2a, 2b with input [2a, 2b, 1]. This is already a valid **max-heap**. Swap root and last: [1, 2b, 2a]. Heap is [1, 2b], so sift-down swaps: [2b, 1, 2a]. Swap root and index 1: [1, 2b, 2a]. The output is 1, 2b, 2a, so the equal keys were **reversed**. The long-distance swaps (root to end) are what destroy **relative order**, and there is no cheap fix.

So choose heapsort when you want sorting to be **reliable** and **predictable**, even if it isn't the fastest option in practice. Even in the worst possible situation, heapsort takes `O(N\logN)` time. It won't suddenly

become extremely slow like basic Quicksort can. Heapsort is mainly used in embedded and real-time systems, kernels (the Linux kernel's `sort()` is a heapsort variant for this reason), or as the fallback inside introsort. Avoid it when stability matters (use merge sort, TimSort), when speed on typical data matters most (use quicksort or introsort), or when input is nearly sorted (use insertion sort or TimSort).

Which One Should I use?

Situation	Good choice	Why
A very small list, or one that is nearly sorted	Insertion sort	Simple, and close to n steps on nearly sorted data
Memory is tight and you need a worst-case guarantee	Heapsort	In place, always `O(N\logN)`
Everyday programming	Your language's built-in sort	Tuned hybrids. Python's built-in sort (Timsort) mixes merge and insertion sort, and Python guarantees its sorts are stable

> Bubble sort and selection sort are valuable for *understanding* how sorting works, but they are rarely the right tool for real work.

Heap-based Usage Patterns

Top-K Elements

Given n items, find the k largest (or smallest). The items may sit in an array or arrive as a stream you can't store. Examples: top three leaderboards, "trending now" lists, search engines keeping the best k documents per query.

The key idea is to create a heap where the **weakest item sets the admission threshold**. Think of a sports team that admits only k people, and the rule is "the k best so far." The coach needs to know one thing, who is the **weakest** person currently inside? A newcomer is only worth considering if they **beat** that person. If they do, the weakest person is shown out and the newcomer takes their place.

Worked Example

```
Heap holds the current top 3 (min-heap, threshold at the root)

          [7]   <- weakest member = threshold
         /   \
       [8]   [9]

New item 5:  5 < 7  -> rejected, nothing changes   (1 comparison)
New item 10: 10 > 7 -> 7 is evicted, 10 takes its place,
                       then settles into position  (a few steps)
```

K-way Merge

You have k lists, each already sorted, with N items in total, and you want one sorted list. For example, sort a file too large for memory by sorting chunks, writing them out as sorted "runs," then merging the runs or databases, where compaction merges several sorted files into one.

The key idea is that **only the fronts matter**. Because each list is sorted, its smallest item is always at the front, so we just need to track the fronts. How? Keep a min-heap of the current fronts. Take the smallest one, and replace it with the next item from the same list.

Worked Example

```
L0: [1]  4  7         heap of fronts:
L1: [2]  5  8   -->      (1 from L0)
L2: [3]  6  9          /           \
                  (2 from L1)   (3 from L2)

Output 1 -> L0's next front (4) joins the heap
Output 2 -> L1's next front (5) joins
Output 3 -> L2's next front (6) joins
Output 4 -> L0's next front (7) joins
... a list that runs out simply stops contributing; the heap shrinks
```

Calculating Median (Two Heaps)

Numbers arrive one at a time. After each arrival, report the median of everything seen so far. The median is the middle value of the sorted data. With an odd count it is the single middle item, and with an even count it is conventionally the average of the two middle items.

If we keep the data sorted everytime a new item comes in, the cost will be expensive, so the core idea is that **you only need the boundary** (the middle). So split the data into a lower half and upper half, the median depends only on the largest item of the lower half and the smallest item of the upper half. For example:

```
Sorted order:    1    3    5   ||   7    8    9   15
                 \____________/     \_____________/
                   lower half          upper half
                   MAX-heap            MIN-heap
                   root = 5            root = 7
                         ^_____________^
                  the two roots = the boundary
```

This can be extended into **any percentile** (just adjust the lower and upper heap bounds). Some real-world use includes streaming dashboards and monitoring, median filters on sensor and image data, etc.

Sorting for Efficient Processing

When data is in no particular order, almost every question forces you to look at everything: "What is the lowest value?", "What is the middle value?", "Is this item here?", "Are there repeats?".

The core idea is to **sort once, then let the order answer the questions**

Once the data is sorted, useful facts become visible just from the positions:

- The smallest item is the first, and the largest is the last.
- The middle value (the median) sits in the middle.
- Equal or similar items sit next to each other.
- All items between two values form one continuous block.
- **Binary search** becomes possible

Worked Example

```
Exam scores, unsorted:  72   45   91   45   60   88   72

Exam scores, sorted:    45   45   60   72   72   88   91
                        ^              ^              ^
                  lowest (1st)         |      highest (last)
                             median: 4th of 7 = 72

The equal scores (45, 45) and (72, 72) now sit side by side.
```

There are certain trade offs for this method. Sorting is not free but it will pay off when you will ask many questions about the same data. Also, you may not need a full sort (perhaps you only need the largest few items, middle value, etc where complete sort is wasteful).

Finding Duplicates

Does a collection contain **repeated values**? Examples: the same e-mail address registered twice, a repeated payment, an ID that should be unique but is not. The direct method is to compare **every item**

with every other item. For 1,000 items that is about 500,000 comparisons, and it grows with the **square** of the list size.

The core idea is that after sorting, equal items **sit side by side**. If two values are equal, nothing can come between them in sorted order. So after sorting, we only need to compare **each item with the next** one. No other pairs matter.

Worked Example

```
Unsorted:   9  4  7  4  1  9  4
Sorted:     1  4  4  4  7  9  9

Compare each item with the next one:

  1 | 4    different
  4 | 4    SAME       -> 4 is a duplicate
  4 | 4    SAME       -> still the 4s
  4 | 7    different
  7 | 9    different
  9 | 9    SAME       -> 9 is a duplicate

Result: 4 appears 3 times, 9 appears 2 times, 1 and 7 appear once.
```

Sorting first turns a "squared" problem into an `O(NlogN)` one. The sorting step is the expensive part: the scan afterwards is only about n steps.

Finding Elements in a Range

Find all items between two limits. Examples: products priced between 20 and 60, students scoring from 60 to 80, log entries between 9:00 and 10:00.

On unsorted data, every **question** means checking **every item**. If you have a million items and a thousand such questions, that is a **billion** checks. The core idea is that in sorted data, the items in a range form **one block**, so only the two **edges** need to be found.

In a sorted list, everything between two limits sits in one continuous block. So we do not need to look at every item: we only need to find **where the block starts** and **where it ends**. Binary search is perfect for finding both edges.

Worked example

```
position:    1    2    3    4    5    6    7    8    9    10
price:       5    12   18   25   30   42   55   70   85   99
                          ^                   ^
                     START: first price  END: first price
                     that is 20 or more  that is bigger than 60
                     |<--- the answer: positions 4 to 7 --->|
                          25, 30, 42, 55   (4 items)
```

2.4 Recursion and Divide-and-Conquer

What is Recursion?

Imagine we are playing the “higher or lower” guessing game. Your friend thinks of a number from 1 to 100. You guess 50. Your friend says “higher”. Now you only need to look at 51–100, so you guess 75. Each guess cuts the possibilities in half, and you repeat the same idea each time.

Recursion means solving a task by repeating the same steps on a smaller version of the same task, until the task is small enough to answer directly. In other words, it is a procedure that refers to itself to solve a smaller version of the problem.

Recursion vs Loops A loop finishes a set of steps and then repeats it. Recursion repeats the steps as a subtask of itself. The situation that is small enough to answer directly, so the repeating stops. For example, a list with no items, or a list with only one item.

What is Divide-and-Conquer

Imagine we are sorting a pile of exam papers: We would want to split the pile into two smaller piles. Ask two friends to sort one pile each. Join the two sorted piles into one: look at the top paper of each pile, take the one that comes first, and repeat. This is exactly how divide-and-conquer works.

Divide-and-Conquer is a way to solve a big problem by splitting it into smaller problems of the same kind, solving each small one, and then joining the answers.

- **Divide:** split the problem into smaller parts that look like the original.
- **Conquer:** solve each part by using the same method again (recursion). If a part is small enough, just solve it directly.
- **Combine:** put the small answers together into the answer to the big problem.

Example

Recursion

We want to find `27` in this sorted list: `3, 8, 12, 15, 21, 27, 34, 40`.

- If the list is empty → the number is not here. Stop.
- Otherwise, look at the middle number. (If the list has an even number of entries, the middle is the first entry of the second half.)
    - If it matches → found it. Stop.
    - If your number is smaller → search the first half, using these same steps.
    - If your number is bigger → search the second half, using these same steps.

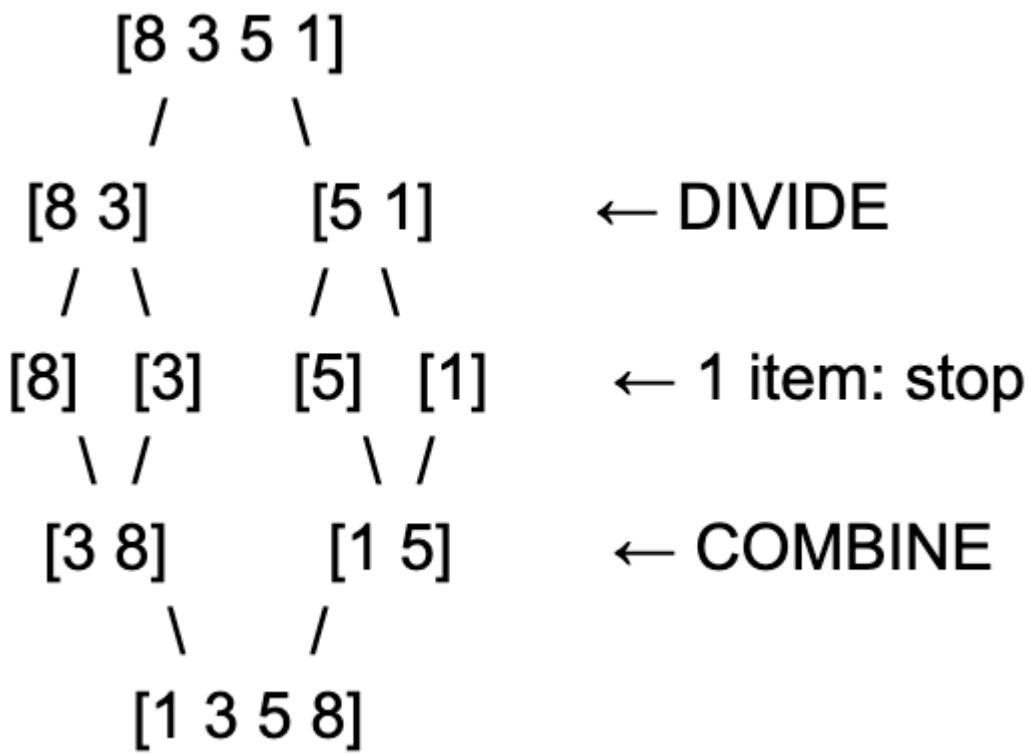
Step	List	Middle	What Happens
1	3,8,12,15,21,27,34,40	21	27 is bigger, so keep the right half
2	27,34,40	34	27 is smaller, so keep the left half
3	27	27	Found it!

Divide-and-Conquer

Imagine we want to sort `8, 5, 3, 1` using divide-and-conquer.

Step	What Happens	Result
Divide	Split the list in half	[8, 3] and [5, 1]
Divide	Split again until each part has 1 item	[8], [3], [5], [1]
Conquer	One item is already sorted, so stop splitting	
Combine	Merge [8] and [3]	[3, 8]
Combine	Merge [5] and [1]	[1, 5]
Combine	Merge [3, 8] and [1, 5]	[1, 3, 5, 8]

- Compare 3 and 1. Take 1.
- Compare 3 and 5. Take 3.
- Compare 8 and 5. Take 5.
- Only 8 are left. Take 8.



Performance

Merging:

- **Best-case time complexity:** `O(NlogN)` (guaranteed splitting and merging)
- **Worst-case time complexity:** `O(NlogN)` (consistent performance regardless of initial order)
- **Average-case time complexity:** `O(NlogN)`
- **Space complexity:** `O(N)` (requires auxiliary memory for merging)

Sorting:

- **Best-case time complexity:** `O(NlogN)` (balanced partitions)
- **Worst-case time complexity:** `O(N^2)` (unbalanced partitions from poor pivot selection)
- **Average-case time complexity:** `O(NlogN)`
- **Space complexity:** `O(logN)` (recursion call stack)

Strengths and Limits

- **Merging:** Guaranteed `O(NlogN)` performance and stability, making it ideal for linked lists and external sorting, though it requires `O(N)` extra auxiliary memory for the merging process.
- **Sorting:** Fast average speed with minimal in-place space overhead (`O(logN)` stack space), but it is unstable and can degrade to `O(N^2)` time if poor pivot selections occur.

2.5 Algorithm Design Paradigm

We learned before about step-by-step recipes (called algorithms) for two jobs: searching and sorting. Looking back, some of those recipes shared the same way of thinking. Linear search simply tried everything. Selection sort kept grabbing "the best item right now". Merge sort and quick sort split a big job into smaller copies of itself. A reusable way of thinking like this is called a design paradigm: a general strategy for inventing algorithms. This note is about those strategies, how to tell whether a recipe built from one of them is correct, and how to choose between them.

What is a Design Paradigm?

An algorithm solves one problem. A paradigm is a plan of attack that works on many problems. Think of cooking, where roasting, boiling and frying are techniques, and a recipe for one particular dish uses one or more of them. Once you know the techniques, you can cook dishes you have never seen before.

Paradigm	Everyday Picture	The Idea in One Phrase
Brute force	Trying every key on a big key ring	Try every possibility
Backtracking	Exploring a maze and walking back from dead ends	Build the answer step by step, undo a step when stuck
Greedy	Grabbing the biggest banknote first	Take the best-looking choice now and never look back
Dynamic Programming	Keeping a notebook of answers you already worked out	Solve each small piece once, then reuse the answer
Divide-and-Conquer	Splitting a pile of exams between helpers	Split, solve the pieces separately, combine (see merge sort and quick sort in note 2.3)

Design and proof toolkit

Before meeting the strategies themselves, we need a few **habits** that apply to every strategy. They answer two questions: "*How do I design a recipe on purpose?*" and "*How do I know it is right?*"

Step	The question to ask	Example: "find the largest number in a list"
1. Specify	Exactly what goes in, and what must come out?	In: a list with at least one number. Out: a number from the list that is at least as big as all the others
2. Design	Which strategy? What are the steps?	Look at the numbers one by one and remember the biggest seen so far

Step	The question to ask	Example: "find the largest number in a list"
3. Prove	Why is the output right for every input, not just my examples?	The "biggest so far" is always the biggest of the numbers looked at so far (see Tool 1 below)
4. Analyze	How much work and memory does it need?	One look per number, one number remembered: work in proportion to n

Tool 1: Loop Invariant (A Promise That Stays True Every Round)

A loop repeats the **same** steps again and again. Each repetition is a **round** (a *pass*, in the language of note 2.3). A loop invariant is a statement about the **work done** so far that

- is true **before** the first round,
- is still true **after every round**, and
- when the loop stops, **proves the answer is right**. ("Invariant" means "does not change": the *statement* stays true even though the data keeps changing.)

Tool 2: Induction (The Domino Argument)

Many recipes are recursive: they hand **smaller** copies of the same job back to themselves (as merge sort does). To prove such a recipe correct, we use induction, which works like a line of dominoes.

- **Base case:** show the recipe works for the smallest input. (The first domino falls.)
- **Step:** show that *if* the recipe works for all smaller inputs, then it works for this one (Each domino knocks over the next). Then it works for **every** size.

```
[first domino falls] -> [2nd] -> [3rd] -> [4th] -> ... all fall
base case each one knocks over the next (the step)
```

Tool 3: Exchange Argument (Swapping Toward The Recipe's Answer)

This is the main tool for proving that a greedy recipe is right (section C uses it a lot). The plan:

- Take any **best** answer, even one produced by somebody else's recipe.
- Show that you can **swap one piece** of it for the piece *our* recipe would choose, **without making it worse**.
- Repeat. Step by step, the best answer turns into our recipe's answer, so ours is just as good.

Example: serving customers one at a time. Customer A needs 5 minutes, customer B needs 2 minutes. Everyone wants to be finished as early as possible (shortest waiting + processing time), so we add up the moments at which the customers finish.

Order	A finishes at	B finishes at	Total
A, then B	5	7	12

Order	A finishes at	B finishes at	Total
B, then A	7	2	9

Tool 4: Counterexample (One Input That Breaks The Recipe)

To show a recipe is wrong, a single failing input is enough. That input is called a counterexample.

- Claim: "to find the smallest item, look only at the first and last items." Counterexample: `5 1 9`. The recipe says `5`, but the smallest item is `1`.

Brute Force (Exhaustive Search)

Find an answer by examining **all possible candidates** and keeping the one (s) that follow the rules (or the best one). For example, which of these numbers add up to exactly 12? In which order should a van visit five houses so that the trip is shortest? Which groups of friends can share one car?

The core idea is to generate every **candidate**, check **each** one. This is what you do when you try every key on a big key ring. It is always correct, as long as you truly cover every candidate. The only question is *how many candidates there are*.

The main disadvantage of this method is that its slow. Grouping using trees require 2^n steps, whilst orderings (three choices for first place, two choice for second, one choice for the third place) requires n! steps. With about 30 items, groups take about 1.07 billion steps, whilst orderings take about 265 followed by 30 zeros.

Worked example: which numbers add up to 12?

The numbers are `3 5 7 9`. Brute force looks at all 2 x 2 x 2 x 2 = 16 groups and adds each one up:

```
{} 0 {3,5} 8 {3,5,7} 15 {3,5,7,9} 24
{3} 3 {3,7} 10 {3,5,9} 17
{5} 5 {3,9} 12 <-- {3,7,9} 19
{7} 7 {5,7} 12 <-- {5,7,9} 21
{9} 9 {5,9} 14
{7,9} 16
```

Backtracking

Brute force wastes huge effort on candidates that were doomed after their first few choices. What if we build the answer one choice at a time, check the rules after every choice, and walk back the moment a path is hopeless.

- **Make** the next choice, which extends the partial answer.

- **Check the rules.** If the partial answer already breaks one, **prune**: undo the choice and try the next option for this step.
- If all choices are made and the rules still hold, you have found a **solution**. Record it (and stop, or keep going to find all of them).
- If a step has no options left, undo the **previous** choice and try *its* next option.

Worked example

Suppose you need to choose 2 numbers from `1, 2, 3`, and their sum must equal 4. We can build our answer one number at a time:

```
Start
├── Choose 1
│   ├── Choose 1 → 1 + 1 = 2 (not enough)
│   ├── Choose 2 → 1 + 2 = 3 (not enough)
│   └── Choose 3 → 1 + 3 = 4 ✓ SOLUTION
│
├── Choose 2
│   └── Choose 2 → 2 + 2 = 4 ✓ SOLUTION
│
└── Choose 3
    └── No valid combination (the 3 + 1 is already found, and 3 + 2 is too big)
```

Worst case scenario, it can stay as bad as brute force, such as `O(N!)` or `O(2^N)`, because for some inputs, pruning cuts very little. In practise, the real work is often thousands or millions of times smaller. It also only remembers the current path, so the memory grows with the number of **choices**, not **candidates**.

Greedy

Greedy is the **simplest** and **fastest** strategy in this note, and also the most **dangerous** one. It is simple because it never looks back. It is dangerous because it often looks right and is sometimes **wrong**. That is why the first section's proof habits matter so much here.

We must make a series of choices to get the **best overall result** (the most, the fewest, the cheapest, the shortest). Checking every combination would take far too long. Can we simply decide step by step? The core idea is that for every step, take the choice that **looks best right now**, and never change your mind.

Picture a cashier giving change by always handing over the biggest coin that still fits. To pay back 63 cents with coins of 25, 10, 5 and 1 cent, the cashier hands over 25 + 25 + 10 + 1 + 1 + 1: six coins, which is also the fewest possible. Greedy **worked**. Now change the coins. With coins of 1, 3 and 4 and an amount of 6, the cashier hands over 4 + 1 + 1: three coins. But 3 + 3 needs only **two**. The very same idea gave a **wrong answer**.

> **Key idea:** Greedy is **fast** and **simple**, but it is **not always right**. Every time you use it, you owe the reader either a **proof** or an honest search for a counterexample.

When does greedy **work**? Two properties

- **Greedy-choice property:** there is always a best **overall** answer that starts with the **greedy** choice. In other words, taking the best-looking option now never closes the door on a **best answer**.
- **Optimal substructure:** after the choice is made, what remains is a smaller copy of the same problem, and a best answer to the **leftover** problem, together with our choice, is a **best** answer to the whole problem.

Argument	The plan, in plain words	Picture
Exchange argument	Take any best answer. Show that swapping in the greedy choice does not make it worse. Repeat until the best answer *is* the greedy answer.	Swapping neighbours until a line is in the right order
Greedy stays ahead	Show that after every step, the greedy result so far is at least as good as what any other recipe has after the same number of steps. So at the end it is at least as good.	A runner who is ahead at every checkpoint wins the race

Interval Scheduling (Choosing The Most Meetings)

The problem. One meeting room, and many booking requests, each with a start time and an end time. Accept **as many requests as possible** so that no two accepted meetings overlap (A meeting that ends at 11 and one that starts at 11 do not overlap). The core idea is to ***always pick the request that finishes earliest***. The meeting that ends first leaves the **most room** for everything that comes after it.

Worked example

```
hour:  8 9 10 11 12 13 14 15 16 17
A 8-15 [==================)
B 9-11   [======)
C 10-13    [=========)
D 11-14       [==========)
E 13-16             [==========)
F 15-17                   [======)
```

The result is **B, D, F**: three meetings, and no schedule can fit **four**. Compare the wrong rules, earliest start. It **picks A (8-15) first**. Only **F (15-17)** still fits afterwards, so it finds just 2 meetings.

Huffman Coding (Compressing Text)

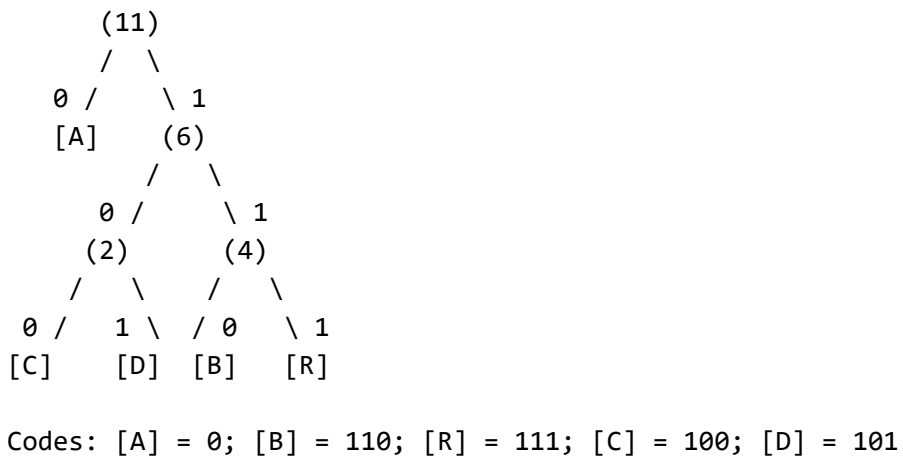
Computers write every letter as a **string of bits (0s and 1s)**, usually the same number of **bits** per letter. But some letters are used **far more** than others. If *frequent* letters get **short** codes and rare letters get **long** codes, the whole message becomes **shorter**. There is one rule: no code may be the beginning of another code (such a set is called prefix-free). Otherwise the receiver could not tell where one letter **ends** and the next **begins**.

Our example message is ABRACADABRA (11 letters): A appears 5 times, B appears 2 times, R appears 2 times, C appear once and D appear once. Five different letters need **3 bits** each if all codes have the same length (2 bits give only 4 codes), so the message costs 11 x 3 = 33 bits. The core idea is to repeatedly join the two **rarest** items into one **combined** item.

How it works

- Treat every letter as an item whose weight is how often it appears.
- Take the two lightest items and join them under a new combined item whose weight is their sum.
- Repeat step 2 until only one item is left. This builds a tree.
- Read each letter's code from the top of the tree down to the letter: 0 for a left branch and 1 for a right branch. The rarest letters end up deepest in the tree, so they get the longest codes.

Step	Items (weights)	The two lightest	Joined into
1	A 5, B 2, R 2, C 1, D 1	C 1 and D 1	CD 2
2	A 5, B 2, R 2, CD 2	B 2 and R 2 (three items weigh 2: any two will do)	BR 4
3	A 5, CD 2, BR 4	CD 2 and BR 4	CDBR 6
4	A 5, CDBR 6	A 5 and CDBR 6	the root 11




```
Codes: [A] = 0; [B] = 110; [R] = 111; [C] = 100; [D] = 101
```

The message ABRACADABRA becomes:

```
[A] [B] [R] [A] [C] [A] [D] [A] [B] [R] [A]
 0  110 111  0  100  0  101  0  110 111  0  -> 23 bits instead of 33
```

Reading the bits from the left, the receiver can always tell where a letter ends: as soon as the bits read so far match a code, that letter is done (0 is A, 110 is B, and so on).

Problem	Greedy rule	Works	Why
Coins of 25, 10, 5, 1	Largest coin that fits	Yes	It happens to work for these coins (needs its own proof)
Coins of 1, 3, 4	Largest coin that fits	No	Counterexample: pay 6 (greedy 3 coins, best 2)
Interval scheduling	Earliest finish	Yes	Exchange argument, greedy stays ahead
Interval scheduling	Earliest start, or shortest	No	Counterexamples above
Huffman coding	Join the two rarest	Yes	Exchange argument

Dynamic Programming I

Brute force is always right but far too slow. Greedy is fast but sometimes wrong. Dynamic programming (DP) takes a middle road: it considers **all** the options (so it is always right), but it never works out the same small question twice (so it is fast).

Dynamic programming solves a big problem by breaking it into smaller versions of itself, called **subproblems**. It solves each subproblem **once**, writes the answer down, and looks it up whenever the same subproblem comes up again.

The problem is that the same question asked again and again. For example, the **Fibonacci numbers** are `1, 1, 2, 3, 5, 8, 13, 21, ...`. Each number is the **sum of the two before it**. This suggests a natural recursive recipe: "*to get the n-th number, get the (n - 1)-th and the (n - 2)-th, and add them.*"

The core idea ist to never solve the same small question twice: write the answer down and look it up next time.

When Does Dynamic Programming Apply?

Two ingredients must be present:

- **Overlapping subproblems:** the **same** subproblems come up **again** and again. (Fibonacci: **yes**. Merge sort: **no**. Its halves are different pieces that never repeat, so a notebook would help nothing, and plain divide and conquer is enough).
- **Optimal substructure:** a best answer to the whole problem is built from best answers to **smaller** subproblems (the same property you met for greedy in section C).

	Memoization (top-down)	Tabulation (bottom-up)
How it starts	From the big question	From the smallest subproblems
How it works	Keep the natural recursive recipe, but keep a notebook (a "memo"): before solving a	Fill a table from the smallest subproblems upward, in an order

	Memoization (top-down)	Tabulation (bottom-up)
	subproblem, look in the notebook; after solving it, write the answer there	such that the ingredients of every entry are already there
Subproblems solved	Only the ones actually needed	All the ones in the table
Strength	Very easy to write once you have the recursive idea	No recursion, and easy to save memory (section E)
Cost	Some overhead from the calls and the lookups	You must work out a correct filling order

Memoization on `fib(5)`:

```
fib(5) -> asks fib(4) -> asks fib(3) -> asks fib(2) and fib(1) (base cases: answere
                         fib(3) finished: written in the notebook
          fib(4) asks fib(2): found in the notebook
          fib(4) finished: written in the notebook
fib(5) asks fib(3): found in the notebook

Computed: fib(5), fib(4), fib(3), fib(2), fib(1) = 5 times. Everything else was a l
```

Coin Change (The Fewest Coins)

Given coin values and an amount, pay **exactly** that amount using the **fewest** coins (you have as many coins of each value as you like). Greedy failed for coins 1, 3, 4 and an amount of 6 (it found 4 + 1 + 1, three coins).

We divide it into subproblems: What is the fewest number of coins that makes the amount a?" for a = `0, 1, 2, ...`. Then try each coin that is not bigger than a as the last coin paid. The fewest for a is 1 plus the smallest of "fewest for a-1 coin".

Amount	Last coin 1	Last coin 3	Last coin 4	Fewest	Best last coin
0	-	-	-	0	-
1	1 + fewest(0) = 1	-	-	1	1
2	1 + fewest(1) = 2	-	-	2	1
3	1 + fewest(2) = 3	1 + fewest(0) = 1	-	1	3
4	1 + fewest(3) = 2	1 + fewest(1) = 2	1 + fewest(0) = 1	1	4
5	1 + fewest(4) = 2	1 + fewest(2) = 3	1 + fewest(1) = 2	2	1 (or 4)
6	1 + fewest(5) = 3	1 + fewest(3) = 2	1 + fewest(2) = 3	**2**	3

Rod Cutting (The Best Way To Cut A Rod)

A rod of length n can be cut into pieces of whole-number lengths. Each length has a price. Cut the rod so that the **total price of the pieces is as high as possible**. (Not cutting at all is allowed: the whole rod is one piece).

A rod of length n has $n - 1$ places where it could be cut, and each place is either cut or not, so brute force would examine 2^{n-1} ways. That is the explosion from section B again: for a rod of length 50 it is about 563 trillion ways.

Worked example

Prices: length 1 earns 2, length 2 earns 5, length 3 earns 9, length 4 earns 10, length 5 earns 12.

Rod length	Options (first piece k: price + best for the rest)	Best	First piece
0	-	0	-
1	1: 2 + 0 = 2	2	1
2	1: 2 + 2 = 4, 2: 5 + 0 = 5	5	2
3	1: 2 + 5 = 7, 2: 5 + 2 = 7, 3: 9 + 0 = 9	9	3
4	1: 2 + 9 = 11, 2: 5 + 5 = 10, 3: 9 + 2 = 11, 4: 10 + 0 = 10	11	1 (or 3)
5	1: 2 + 11 = 13, 2: 5 + 9 = 14, 3: 9 + 5 = 14, 4: 10 + 2 = 12, 5: 12 + 0 = 12	**14**	2 (or 3)

The best price for a rod of length 5 is **14**, by cutting it into pieces of lengths 2 and 3 (prices 5 + 9). Notice that the whole rod (price 12) is **not** the best choice.

Dynamic programming II

This section shows the classic problems that every programmer should be able to recognize.

Longest Increasing Subsequence (LIS)

A **subsequence** of a list keeps some of its items in the **same order**, but the items need not be next to each other: 2 3 6 9 is a subsequence of 5 2 8 6 3 6 9 7. A subsequence is increasing when every item is **bigger** than the one before it. Find the **longest** increasing subsequence. Think of finding the longest steadily rising trend hidden in noisy measurements, when you are allowed to ignore some of the points.

The core idea is that for each item, ask: what is the longest increasing subsequence that ENDS exactly at this item? This "ending at" trick keeps the subproblems small and lets them build on each other.

Worked example

```
position:             1 2 3 4 5 6 7 8
value:                5 2 8 6 3 6 9 7
longest ending here:  1 1 2 2 2 3 4 4
came from (position): - - 1 1 2 5 6 6
```

For position 7 (the value 9): the **earlier** smaller items are at positions 1 to 6, and the best one is position 6 (length 3), so the length is 3 + 1 = 4. The largest entry is **4**. To find the subsequence itself, follow the "came from" row backwards: position 7 9, from 6 6, from 5 3, from 2 2. Read forwards: 2 3 6 9.

Longest Common Subsequence (LCS)

Given two texts, find the longest subsequence that appears in **both** (same order, gaps allowed). This is the idea behind tools that show what changed between two versions of a file, and behind comparing DNA strands. **Example:** KITTEN and SITTING share the subsequence ITTN (length 4). The core idea is to compare the beginnings of the two texts, **one letter** at a time.

```
         (none)   S  I  T  T  I  N  G
(none)     0      0  0  0  0  0  0  0
  K        0      0  0  0  0  0  0  0
  I        0      0  1  1  1  1  1  1
  T        0      0  1  2  2  2  2  2
  T        0      0  1  2  3  3  3  3
  E        0      0  1  2  3  3  3  3
  N        0      0  1  2  3  3  4  4
```

The bottom-right cell says **4**. To read off *which* letters, walk back from that cell:

```
Bottom-right cell (N, G): the letters differ; the cell on the left holds the same v
Cell (N, N): the letters are equal -> take N, move diagonally up-left
Cell (E, I): the letters differ; above and left both hold 3 -> move up (either work
Cell (T, I): the letters differ; the cell on the left (3) beats the one above (2) -
Cell (T, T): equal -> take T, up-left Cell (T, T): equal -> take T, up-left
Cell (I, I): equal -> take I, up-left Reached a zero -> stop.
Letters taken, read backwards: N T T I -> forwards: I T T N
```

Paths On A One-Way Map (DP on a DAG)

Because there are no loops, the places can be lined up in an order in which every arrow points forward (called a **topological order**). Then the best answer for a place depends only on places earlier in the line, which is exactly what dynamic programming needs.

Worked example

From	To	Cost	From	To	Cost
S	A	2	B	C	2
S	B	5	B	D	7
A	B	1	C	D	1
A	C	6	C	T	8
			D	T	2

The subproblem is to find the cheapest cost to reach each place from S. The solution is to look at every arrow coming into the place; for each, take the cheapest cost of the place it comes from plus the arrow's cost; the answer is the smallest.

Place	Arrows into it (cost so far + arrow)	Cheapest	Came from
S	-	0	-
A	S: 0 + 2 = 2	2	S
B	S: 0 + 5 = 5, A: 2 + 1 = 3	3	A
C	A: 2 + 6 = 8, B: 3 + 2 = 5	5	B
D	B: 3 + 7 = 10, C: 5 + 1 = 6	6	C
T	C: 5 + 8 = 13, D: 6 + 2 = 8	**8**	D

The cheapest cost is **8**. Following "came from" backwards `T, D, C, B, A, S` gives the route `S, A, B, C, D, T` with a cost of 2 + 1 + 2 + 1 + 2 = 8.

The Problems at A Glance

Problem	Subproblem in words	Table size	Time (about)
LIS	Longest rising run ending at item i	n	n x n
LCS	LCS of the first i letters and the first j letters	m x n	m x n
Edit distance	Fewest edits from the first i letters to the first j	m x n	m x n
Cheapest DAG path	Cheapest cost to reach place v	number of places	places + arrows

Dynamic programming III

This section shows two more shapes of subproblem: a **stretch** in the middle of a sequence (called **interval DP**) and a **branch of a tree**.

Matrix-Chain Multiplication

Multiplying a long chain of grids can be done in **any grouping** and the final result is the same, but the **amount of work is not**. We do not actually multiply anything here: we only decide **how to group** the chain so that the work is smallest.

Example with three grids: `A` is 10 by 30, `B` is 30 by 5 and `C` is 5 by 60.

Grouping	Work	Total
(`A` `B`) `C`	`A` x `B`: 10 x 30 x 5 = 1,500 (gives 10 by 5), then x `C`: 10 x 5 x 60 = 3,000	4,500
`A` (`B` `C`)	`B` x `C`: 30 x 5 x 60 = 9,000 (gives 30 by 60), then `A` x: 10 x 30 x 60 = 18,000	27,000

The second grouping needs **six times** the work. Trying every grouping is another explosion: a chain of 4 grids has 5 groupings, 5 grids have 14, 10 grids have 4,862, and 20 grids have 1,767,263,190. The core idea is that the last multiplication splits the chain into a left stretch and a right stretch. Try every split point.

Worked example

Four grids: `A1` is 10 by 30, `A2` is 30 by 5, `A3` is 5 by 60 and `A4` is 60 by 10.

```
stretch best cost best split (after grid number ...)
1..2 1,500 1
2..3 9,000 2
3..4 3,000 3
1..3 4,500 2 (A1   A2)  A3
2..4 4,500 2  A2  (A3   A4)
1..4 5,000 2 (A1   A2) (A3   A4)
```

For the whole chain (1..4) there are three possible last splits:

Split after	Left stretch	Right stretch	Final multiplication	Total
grid 1	0	4,500	10 x 30 x 10 = 3,000	7,500
grid 2	1,500	3,000	10 x 5 x 10 = 500	**5,000**
grid 3	4,500	0	10 x 60 x 10 = 6,000	10,500

The best grouping is (`A1` `A2`) (`A3` `A4`) with **5,000** multiplications. The five possible groupings cost 10,500, 33,000, 5,000, 30,000 and 7,500, so the best one is more than six times cheaper than the worst. The "best

split" column is a parent pointer, following it recovers the grouping.

Optimal Binary Search Tree

A binary search tree keeps sorted keys so that a search starts at the top and goes left (smaller) or right (bigger) at every step, like binary search. The number of looks for a key is its depth (the top key takes 1 look). If some keys are searched far more often than others, the tree's shape matters: **popular** keys belong near the **top**.

Given the keys in order and how often each is searched, build the tree with the **fewest total looks**. (To keep things simple we count only searches that find a key.)

Shape of the tree	Looks for A, B, C	Total looks per 100 searches
B on top, A left, C right (the balanced tree)	2, 1, 2	50 x 2 + 10 x 1 + 40 x 2 = 190
A on top, then C, then B below C	1, 3, 2	50 x 1 + 10 x 3 + 40 x 2 = **160**
A on top, B below, C below B (a chain)	1, 2, 3	50 + 20 + 120 = 190
C on top, then A, then B below A	2, 3, 1	100 + 30 + 40 = 170
C on top, then B, then A below B	3, 2, 1	150 + 20 + 40 = 210

The **balanced tree is not the best**: the popular key **A** should sit on top. The core idea is to try every key of a stretch as its root, and combine the best trees for the keys on each side.

Stretch of keys	Options (root: cost)	Best	Best root
A	-	50	A
B	-	10	B
C	-	40	C
A, B (60 searches)	root A: 60 + 0 + 10 = 70, root B: 60 + 50 + 0 = 110	70	A
B, C (50 searches)	root B: 50 + 0 + 40 = 90, root C: 50 + 10 + 0 = 60	60	C
A, B, C (100 searches)	root A: 100 + 0 + 60 = 160, root B: 100 + 50 + 40 = 190, root C: 100 + 70 + 0 = 170	**160**	A

The best roots tell us the shape: A on top, then C (best root for the stretch B, C), then B below C. Notice how similar this is to matrix-chain multiplication: both try every split of a stretch.

Choosing a paradigm

Knowing the strategies is only half of the skill. The other half is choosing one for a new problem. This section gives a checklist and then looks at the same problem through several strategies.

The Decision Checklist

Work through the questions in order, from the cheapest strategy to the most elaborate one. It is a guide, not a law.

- Write the problem precisely (section A) and ask whether it is a known problem with a known recipe (sorting, searching, shortest routes, ...). If so, use the known recipe or your language's built-in tool.
- Is the input small enough to try everything? (up to about 20 items for groups, about 10 for orderings.) If so, brute force is enough, and it is also the best referee for testing anything cleverer.
- Does a "best choice right now" survive tiny tests, and can you prove it? (Try to break it first; then use an exchange argument or "greedy stays ahead".) If so, use greedy: it is the fastest.
- Do smaller copies of the problem repeat, and do best answers to them combine into a best answer? If so, use dynamic programming.
- Does the problem split into smaller copies that are independent and easy to combine? If so, use divide and conquer (like merge sort and quick sort).
- Is it a puzzle of choices with rules to satisfy? Use backtracking with pruning.
- Nothing fits and the input is huge? The problem may be a genuinely hard one.

Common mistakes when choosing

- Using **greedy without a proof** (or without even trying to break it).
- Using **dynamic programming when the subproblems do not overlap**. It then spends memory and gains nothing (merge sort is the classic example).
- Using **brute force on a big input** without first counting the candidates (section B).
- Choosing subproblems that **do not carry enough information** to build the answer.
- Not keeping a **brute-force referee** to test a clever recipe on small inputs.

The Paradigms Side By Side

Paradigm	Core move	Always correct?	Typical speed	What you must check
Brute force	Try every candidate	Yes	Exponential or worse (2n,n!)	Only that no candidate is missed
Backtracking	Build step by step, prune dead ends	Yes, if pruning is safe	Often far better than brute force, same worst case	Pruning never cuts a valid answer
Greedy	Take the best-looking choice now	Only when proven	Very fast: a sort plus one pass	Exchange argument or "stays ahead"; hunt for counterexamples
Dynamic programming	Solve each subproblem once and reuse	Yes, if the recurrence is right	(subproblems) x (work each): polynomial	The recurrence (induction)

Paradigm	Core move	Always correct?	Typical speed	What you must check
Divide-and-Conquer	Split into independent pieces, then combine	Yes, if the combine step is right	Often `O(NlogN)`	Induction on the size

Quick Learning Check

Choose a strategy for each problem and say why.

(a) Fit as many lectures as possible into one hall, given each lecture's start and end time.

(b) Cut a long log into pieces so that the pieces' total selling price is as high as possible (each length has its own price).

(c) Seat 12 guests around a table so that certain pairs of guests are not next to each other.

(d) Sort 10 million records.

Solution

(a) Greedy (earliest finish first), provable with an exchange argument.

(b) Dynamic programming: this is rod cutting, where subproblems (shorter logs) repeat and a best cutting is built from best cuttings of shorter logs.

(c) Backtracking with pruning: 12 guests have 12! = 479,001,600 seatings, and placing one guest at a time lets us abandon a seating as soon as two guests who must be apart are neighbours.

(d) Divide and conquer (merge sort or quick sort), as in note 2.3.

2.6 Graph Algorithms

What are Graph Algorithms?

Graph algorithms are procedures used to solve problems involving relationships between objects or by representing object as nodes (vertices) and relationships as edges. Graph algorithms use these relationships or connections to answer questions such as whether two nodes (vertices) are connected, what the shortest route between them is, and how a network can be connected at minimum cost.

In this study guide we will be discussing 5 major types of graph algorithms:

- Graph Traversal
- Connectivity and Cycle Detection
- Shortest Path Algorithms
- Minimum Spanning Tree (MST)
- Topological Sorting

Graph Traversal

Graph traversal is the process of visiting all the vertices or edges in a graph.

There are two primary methods for graph traversal:

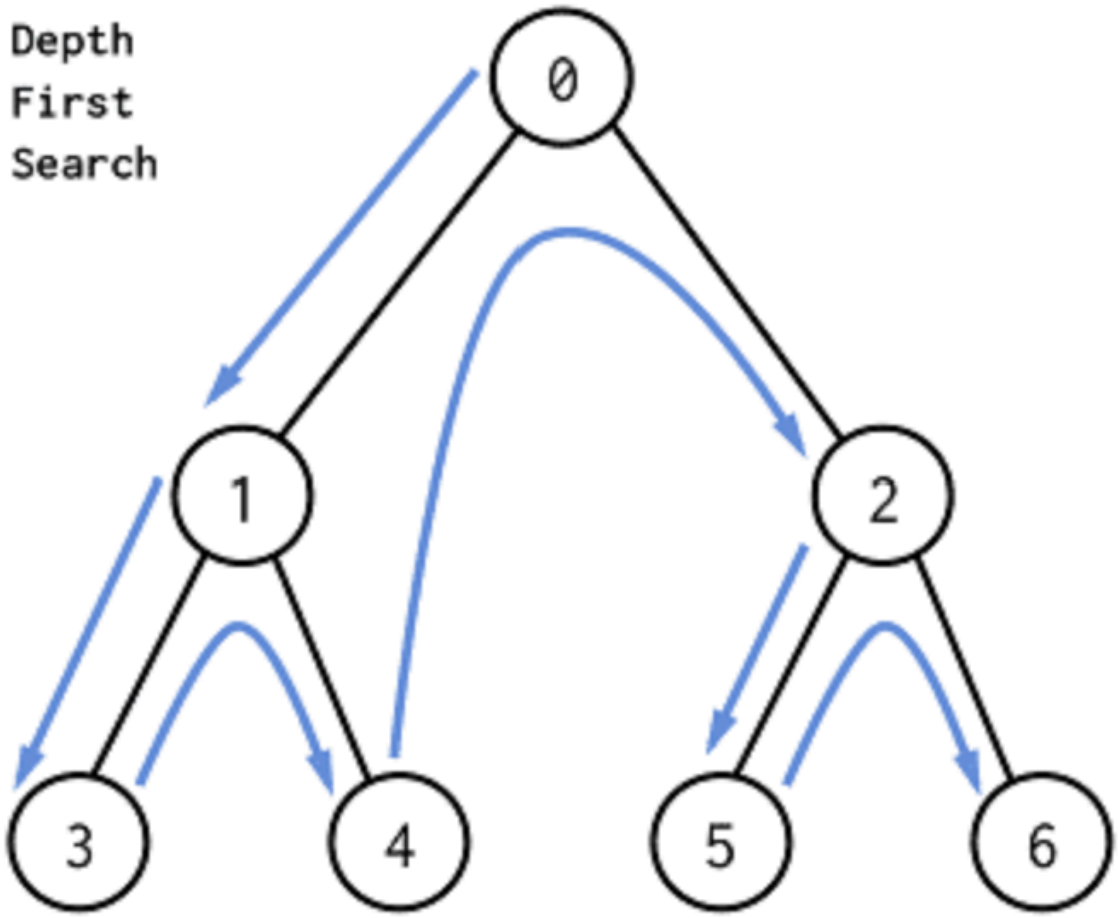
- Depth-First Search (DFS)
- Breadth-First Search (BFS)

Depth-First Search (DFS)

Depth-First Search (DFS) starts at a source vertex and explores until it hits a dead end before backtracking. DFS prioritizes exploring deeper into the graph instead of fully exploring each level. It can be implemented using recursion or an explicit stack. DFS is used for tasks like cycle detection and topological sorting.

How DFS works:

- Start at a vertex and mark it as visited.
- Process the vertex.
- Visit an unvisited neighbor, repeating the same procedure.
- When no unvisited neighbors remain, backtrack to the previous vertex.
- Continue until all reachable vertices have been explored.



In this illustration, traversal starts at 0, moves to its neighbour 1, then goes as deep as possible to 3. After reaching a dead end, it backtracks to 1 and explores 4. Then, it backtracks all the way up, exploring 2 and repeating the same process for 5 and 6. In this example, the traversal follows the sequence of: 0->1->3->4->2->5->6.

This sequence can be visualized with the following table.

Step	Visited	Stack
0	-	{0}
1	0	{1, 2}
2	1	{3, 4, 2}
3	3	{4, 2}
4	4	{2}
5	2	{5, 6}
6	5	{6}
7	6	{}

With an adjacency list, DFS has a time complexity of `O(V + E)`, where this means the time it takes to finish DFS depends on how many vertices `V` and edges `E` are in the graph. DFS also has an auxiliary (supplementary) space complexity of `O(V)`, meaning the extra memory DFS uses to track where it has visited. The space is used by the visited information and the recursion stack or explicit stack.

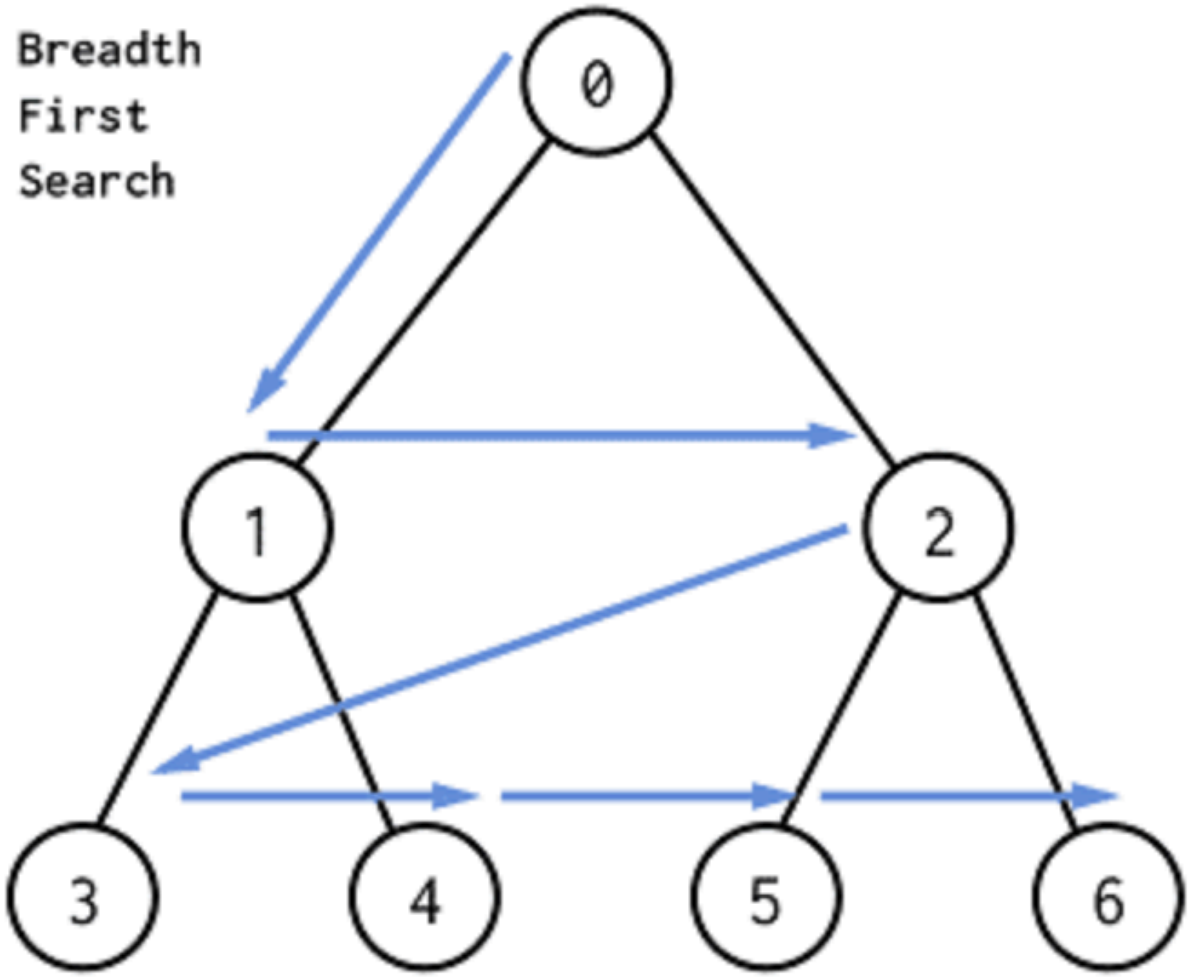
For very deep graphs, recursive DFS may exceed a programming language's recursion limit. An explicit stack can be used instead.

Breadth-First Search (BFS)

Breadth-First Search (BFS) starts at a source vertex and explores all vertices at its level before moving on to the next level. BFS uses a queue, which follows the First In, First Out (FIFO) principle. The first vertex added to the queue is the first one processed. BFS can be used for finding the shortest path in unweighted graphs.

How BFS works:

- Mark the starting vertex as visited and add it to the queue.
- Remove the front vertex from the queue and process it.
- Examine its neighbors. For each unvisited neighbor, mark it as visited and add it to the queue.
- Repeat until the queue is empty.



In this illustration, traversal starts at 0, moves to its neighbour 1, then explores the adjacent vertex of 2. After exploring every vertex at its level, it starts exploring the next level, starting with 3. It repeats the process for 4, 5, and 6 until all vertices have been visited. In this example, the traversal follows the sequence of: 0->1->2->3->4->5->6.

This sequence can be visualized with the following table.

Step	Visited	Queue
0	-	{0}
1	0	{1, 2}
2	1	{2, 3}
3	2	{3, 4}
4	3	{4, 5}
5	4	{5, 6}
6	5	{6}

Step	Visited	Queue
7	6	{}

With an adjacency list, BFS has a time complexity of `O(V + E)`, where `V` is the number of vertices and `E` is the number of edges. Each vertex and edge is examined a bounded number of times.

Its auxiliary space complexity is `O(V)` because the visited information and queue may store up to a number of entries proportional to the vertices.

Connectivity and Cycle Detection

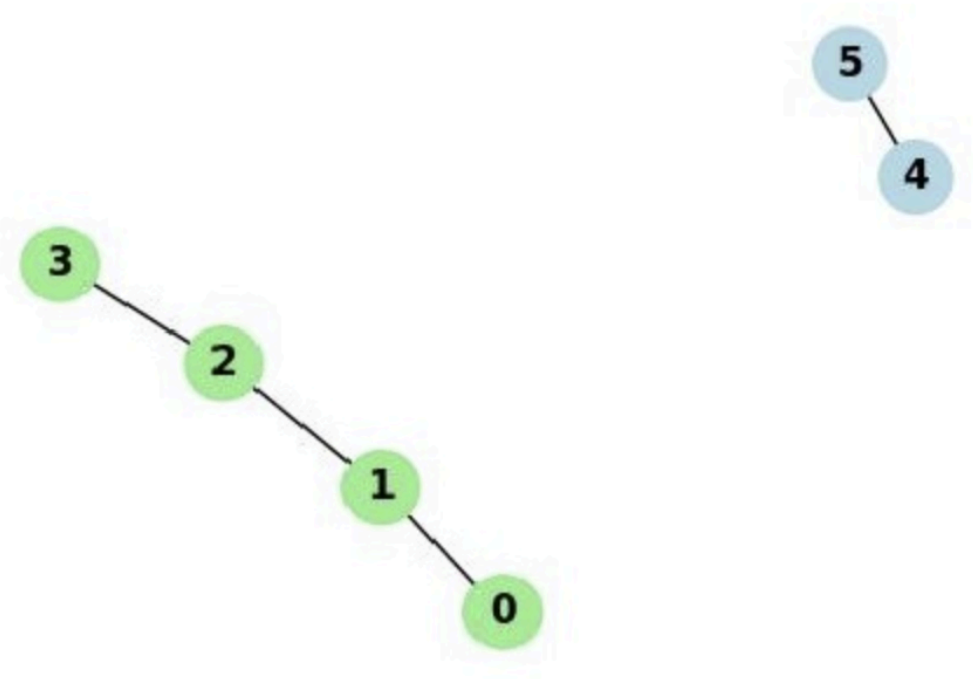
Connectivity

Connectivity describes whether vertices can reach one another through paths. In an undirected graph, a graph is connected if every vertex can be reached from every other vertex. If this is not true, the graph consists of multiple connected components. A connected component is a group of vertices in which every pair of vertices is connected by a path. To find connected components, the two algorithms that are used are BFS and DFS.

How does this work:

- Mark every vertex as unvisited.
- Choose an unvisited vertex and do BFS or DFS from it.
- Assign all vertices reached during that traversal to the same component.
- Repeat with another unvisited vertex until every vertex has been visited.

With an adjacency list, this process takes `O(V + E)` time.



In the illustration, DFS identifies two connected components: {0, 1, 2, 3} and {4, 5}.

In directed graphs, connectivity has additional rules because edges can only be followed in their specified directions. This divides the level of connectivity into two, strongly connected and weakly connected. A strongly connected component (SCC) is a group of vertices in which every vertex can reach every other vertex by following directed edges. A weakly connected component is a group of vertices that is connected when edge directions are ignored. Connectivity in directed graphs uses different algorithms that won't be further discussed such as Kosaraju's and Tarjan's can identify strongly connected components.

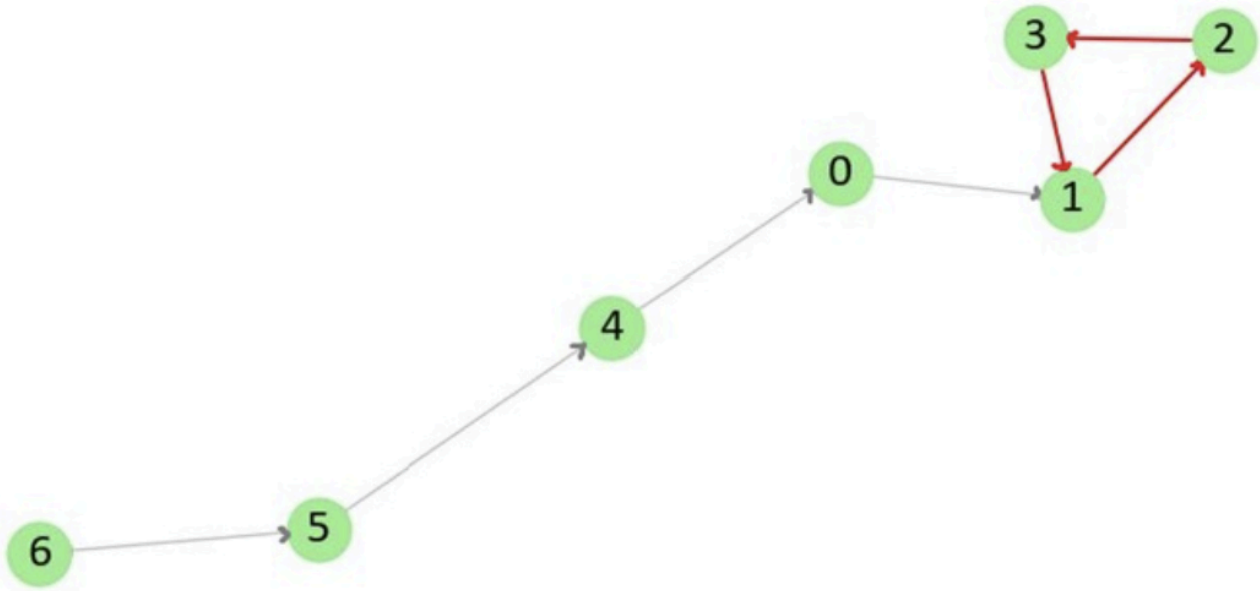
Cycle Detection

Cycle detection is the process of finding cycles (if any) in a graph. It is important in tasks such as checking dependencies in directed graphs and detecting infinite loops. The two major methods for cycle detection are:

DFS-Based Cycle Detection
This method uses DFS to explore the graph and keep track of the visited vertices. If a vertex is encountered that is already in the recursion stack, a cycle is detected.

- In an undirected graph, DFS can detect a cycle by finding a visited neighbor that is not the current vertex's parent until it meets the edge of the vertex's parent.
- In a directed graph, DFS can detect a cycle by finding an edge to a vertex that is currently on the active recursion path. A common implementation tracks three states:
    - Unvisited: the vertex has not been explored.
    - Visiting: the vertex is currently being explored.

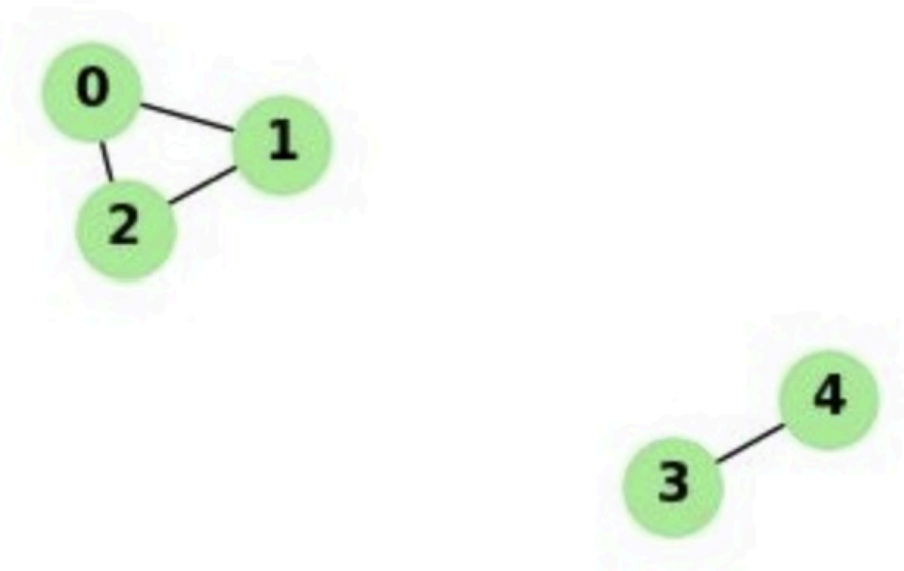
- Visited: the vertex and its outgoing edges have been fully explored. An edge to a vertex in the visiting state indicates a directed cycle.



In this graph, a cycle is detected involving vertices 1, 2, and 3.

Union-Find Cycle Detection

This method uses a union-find data structure to keep track of connected components and detect cycles in an undirected graph. If adding an edge connects two vertices that are already in the same component, a cycle is detected.



In this graph, a cycle is detected with vertices 0, 1, and 2.

Shortest Path Algorithms

A shortest-path algorithm finds a path between vertices that minimizes the total path cost. Depending on the graph, the cost may be the number of edges or the sum of edge weights. The appropriate algorithm depends on whether edges have weights, whether those weights can be negative, and whether a single source or all pairs of vertices must be considered.

Three major algorithms to find the shortest paths:

BFS
BFS finds shortest paths in an unweighted graph or when every edge represents the same cost because it explores vertices in increasing order of their distance, measured in edges, from the source. During traversal, the algorithm records the distance of each newly discovered vertex as one more than the distance of the vertex from which it was reached.

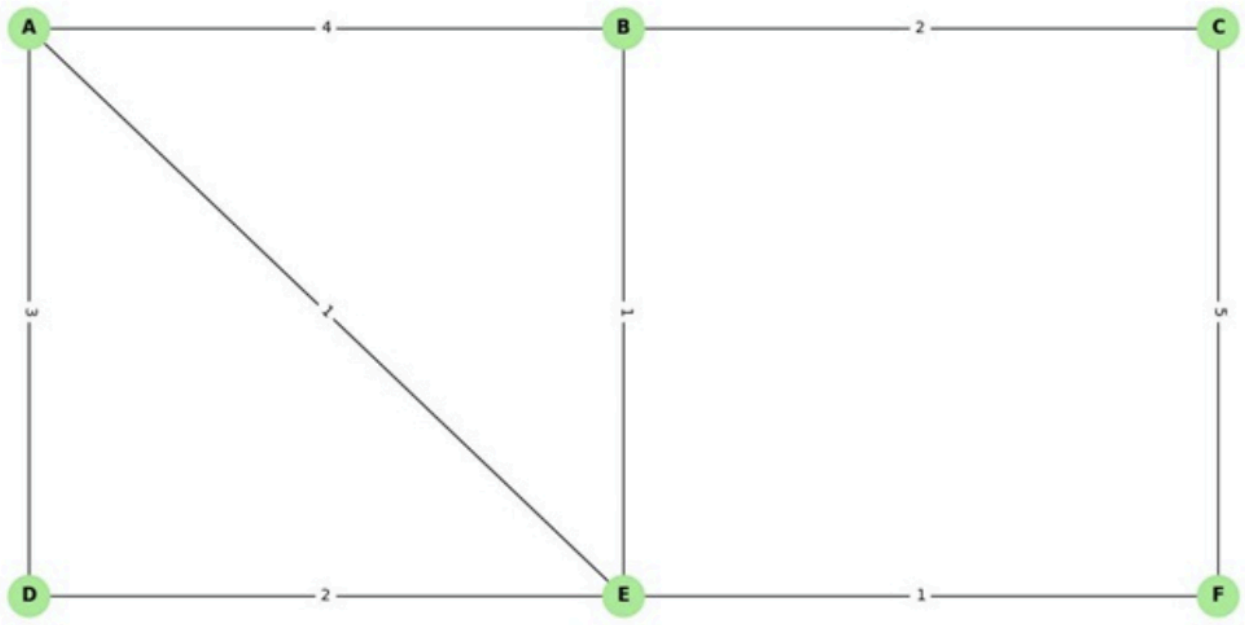
Its time complexity is `O(V + E)` with an adjacency list.

Dijkstra's Algorithm
Dijkstra's algorithm finds the shortest paths from one source vertex to all reachable vertices in a weighted graph when all edge weights are non-negative. Dijkstra's Algorithm uses a priority queue to select the vertex with the minimum distance at each step.

How does this work:

- Label all other vertices estimated to be infinity (∞), except the source vertex which we know is 0 cost.
- Explore neighbours with the least estimated cost following a priority queue.
- Evaluate all adjacent unvisited neighbours to calculate costs. If the newly calculated path is shorter than the previous one, update the vertex's cost. This improvement step is called relaxation.
- Move to the next lowest cost estimate vertex, while keeping track of the shortest paths taken.
- Repeat step 2 through 4 until the priority queue is empty or the designated end point has been reached.
- Organize the lowest cost path based on found estimates.

Relaxation can be modelled where for an edge from u to v with weight w, it checks: *distance[u]* + w < *distance[v]*. Its time complexity is `O((V + E) logV)`.



The shortest path from `A` to `F` using Dijkstra's algorithm is `A -> E -> F` with a total weight of 2.

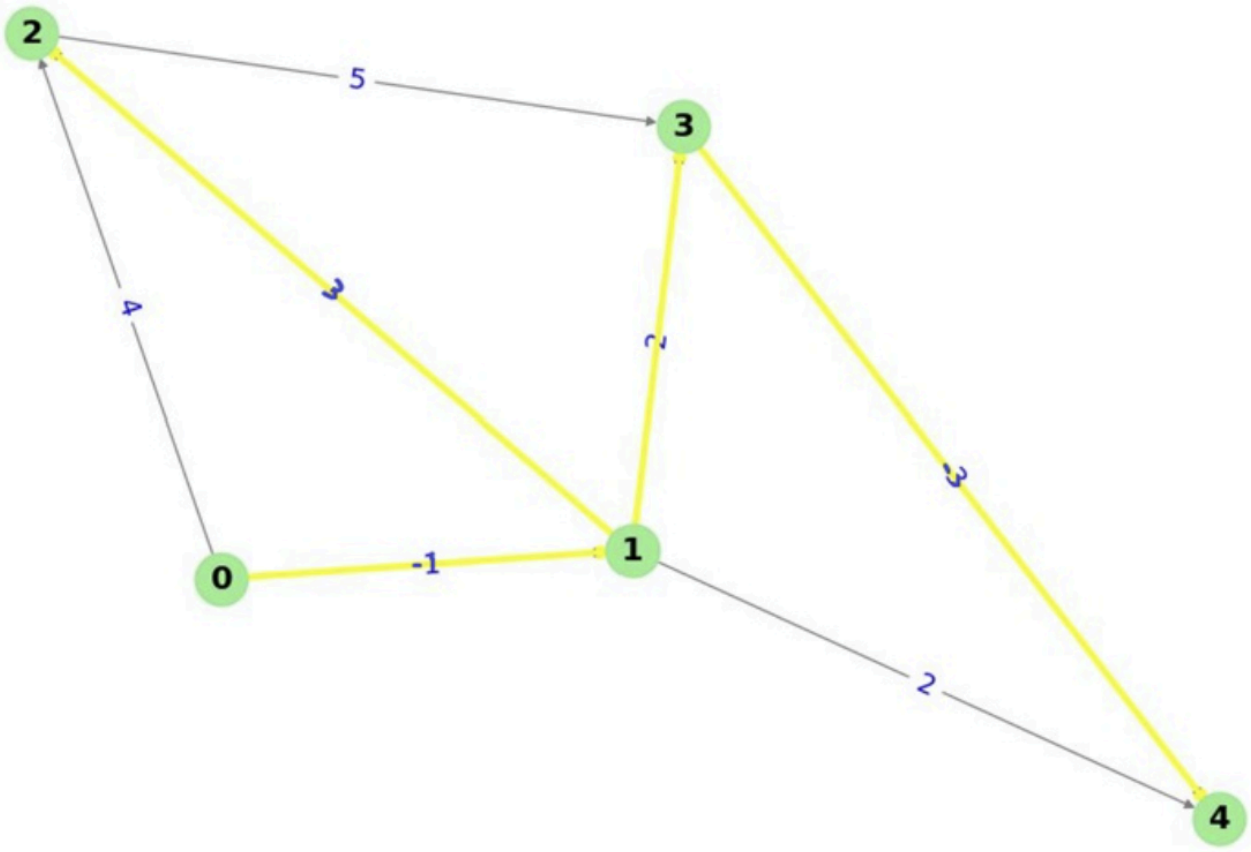
Bellman–Ford Algorithm

Bellman–Ford Algorithm also finds shortest paths from a single source, but it supports negative edge weights. However, this algorithm does not work if the graph contains a loop whose total edge weights are less than 0. If a negative cycle exists, a path could loop through it infinitely to keep reducing the distance, making finding a true shortest path impossible. Unlike Dijkstra's Algorithm which also doesn't work with negative cycles, Bellman-Ford Algorithm can detect negative cycles.

How does this work:

- Label all other vertices estimated to be infinity (∞), except the source vertex which we know is 0 cost.
- Relax every edge (u, v) with the cost w, evaluate if the path to v through u is shorter than the previously calculated cost for v. If it is, update the vertex's cost and record u as its predecessor.
- Repeat step 2 for a total of $V - 1$ iterations or times (where V is the number of vertices). In a graph without negative cycles, a shortest path contains at most $V - 1$ edges. Optimal or shortest paths can be found sooner than $V - 1$, but $V - 1$ iterations will guarantee shortest paths.
- If the graph keeps relaxing after $V - 1$ iterations, that means the graph has a negative cycle.
- Organize the lowest cost path based on predecessors.

Its time complexity is `O(V x E)`.

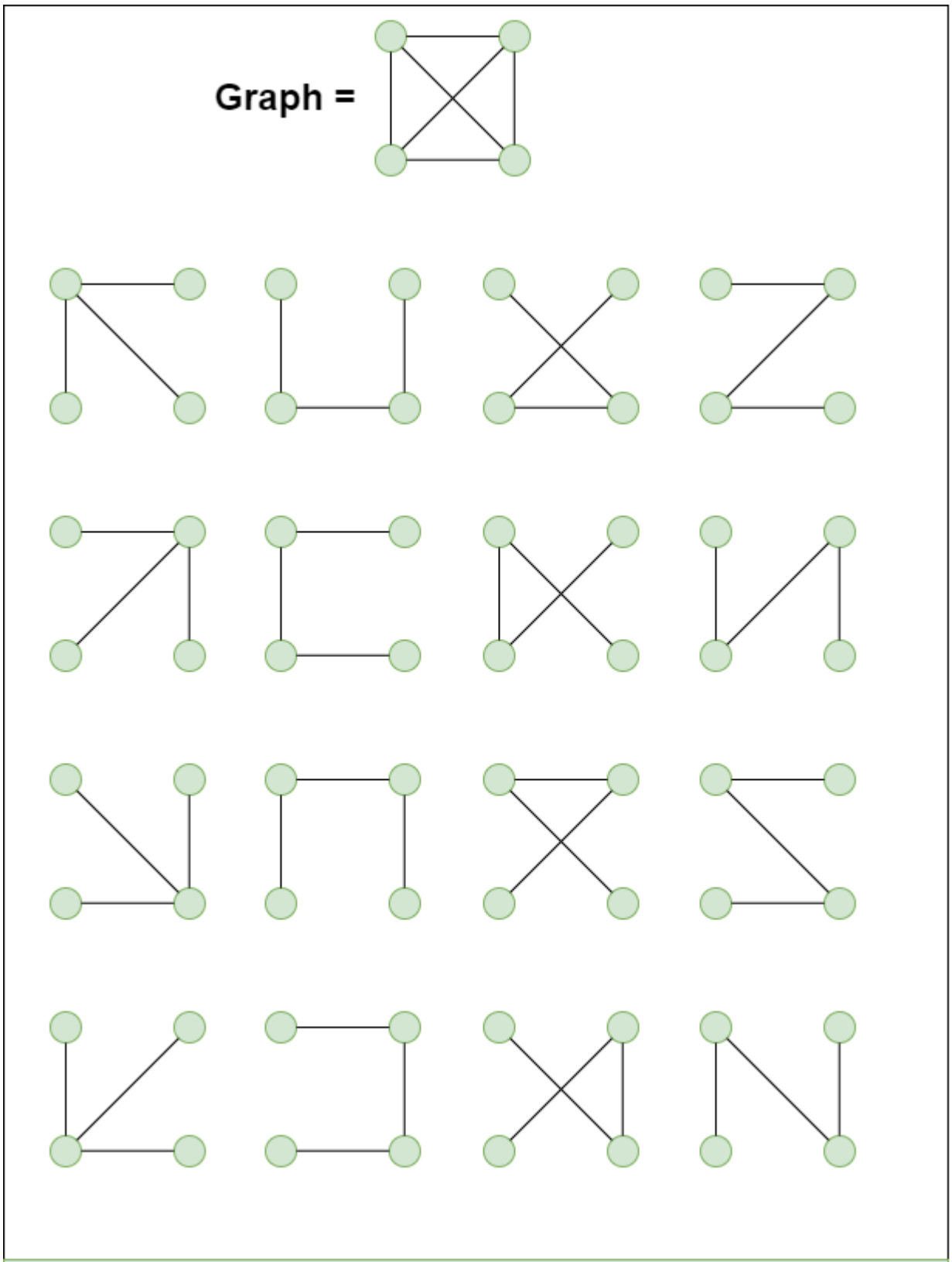


- Vertex 0: The distance to itself is 0, as no travel is needed.
- Vertex 1: The shortest distance is -1, achieved via the edge from vertex 0 to vertex 1 with a weight of -1
- Vertex 2: The shortest distance is 2, found by traveling from vertex 0 to vertex 1 (distance -1), and then from vertex 1 to vertex 2 (distance 3), resulting in a total distance of -1 + 3 = 2.Vertex 3: The shortest distance is 1, obtained by traveling from vertex 0 to vertex 1 (distance -1), and then from vertex 1 to vertex 3 (distance 2), resulting in a total distance of -1 + 2 = 1.
- Vertex 4: The shortest distance is -2, found by traveling from vertex 0 to vertex 1 (distance -1), from vertex 1 to vertex 3 (distance 2), and then from vertex 3 to vertex 4 (distance -3), resulting in a total distance of -1 + 2 + (-3) = -2.

Hence, shortest distances from vertex 0: {0: 0, 1: -1, 2: 2, 3: 1, 4: -2}.

Minimum Spanning Tree (MST)

A Minimum Spanning Tree (MST) is a subgraph of an undirected, weighted graph that connects all the vertices together with the minimum total edge weight, without any cycles.



All Possible Spanning Trees of the Graph

There are two commonly used algorithms to find an MST:

- Kriskal's Algorithm
- Prim's Algorithm

Kruskal's Algorithm
Kruskal's algorithm is a greedy algorithm used to find the MST of a graph using an edge-based approach. It works by sorting the edges by weight and adding edges to the MST in increasing order of weight, ensuring no cycles are formed.

How it works:

- Sort all edges from lowest to highest weight.
- Start with an empty spanning tree by choosing the lowest-weight edge.
- Examine the next lowest-weight edge and connect it if it does not create a cycle.
- Repeat until MST contains V-1 edges.

Its time complexity is `O(E log E)`.

Prim's Algorithm
Prim's algorithm is also a greedy algorithm used to find the MST of a graph using a vertex-based approach. It starts with any vertex and grows the minimum spanning tree (MST) by adding one vertex at a time. It always adds the smallest edge that connects a vertex already in the tree to a vertex not yet in the tree. A priority queue is used to select the smallest edge at each step.

How it works:

- Choose any starting vertex.
- Examine the edges connecting the current tree to vertices outside it.
- Select the lowest-weight edge that connects the tree to a new vertex.
- Add that edge and vertex to the tree.
- Repeat until all vertices have been included.

Its time complexity is `O((V + E) logV)`.

Topological Sorting

Topological sorting arranges each vertice in order so vertices with no dependencies are ordered first. A topological sorting is not unique, meaning as long as the order is valid, the topological sorting is also valid. This type of sorting is also only possible for a directed acyclic graph (DAG).

There are two methods to do topological sorting:

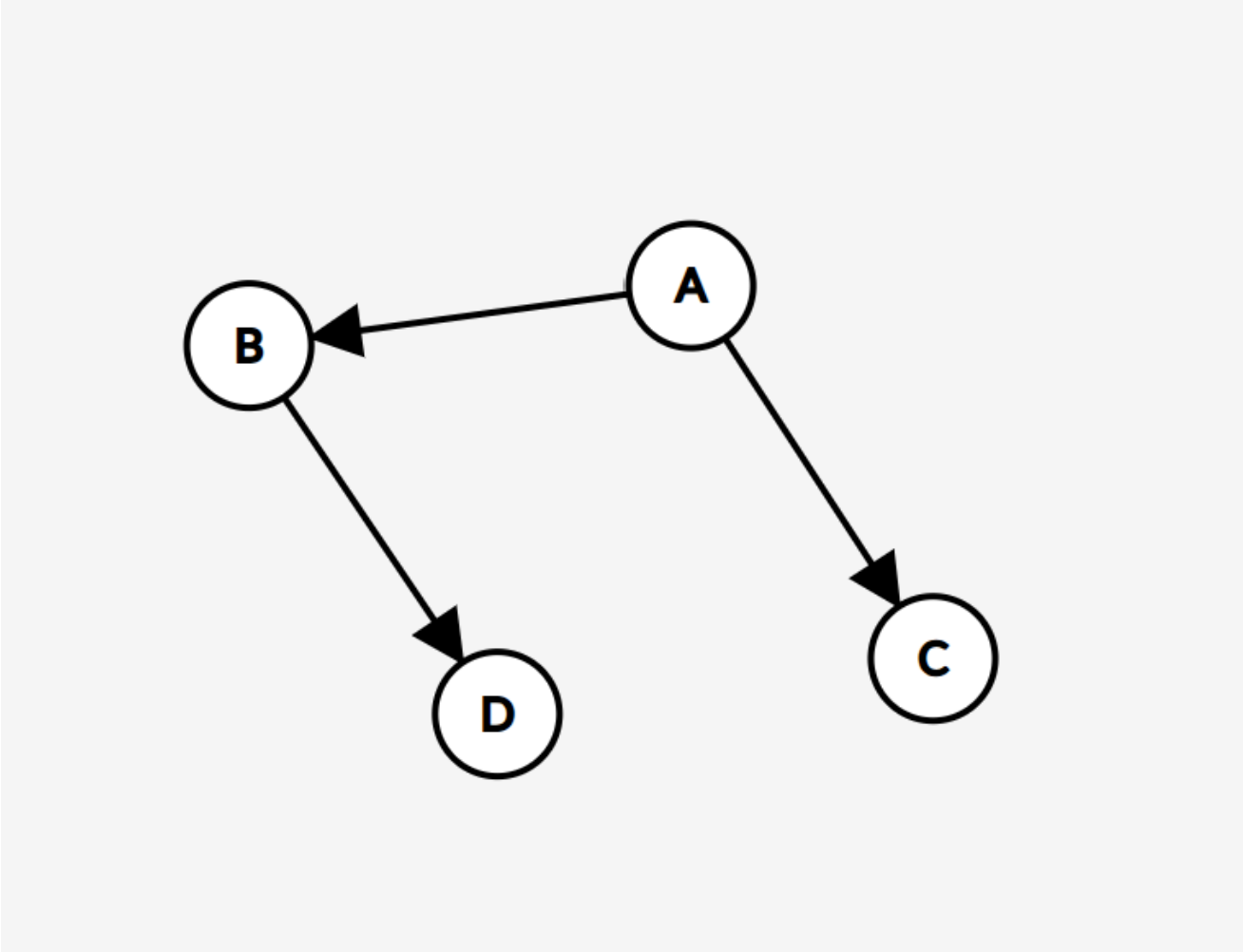
- Depth-First Search (DFS)
- Breadth-First Search (BFS) using Kahn's Algorithm

Depth-First Search (DFS) Topological sorting uses DFS by placing each vertex into a list after all its reachable unvisited neighbors have been processed.

How it works:

- Mark every vertex as unvisited.

- Start DFS from an unvisited vertex.
- Explore unvisited neighbours deeper until hitting a dead end.
- Add the vertex to the list and backtrack to check other paths.
- Repeat for any remaining unvisited vertices.
- Reverse the result list to obtain the topological ordering.



Using the DFS, we get the order of `D -> B -> C -> A`. We reverse the list to get the topological order, which is `A -> C -> B -> D`.

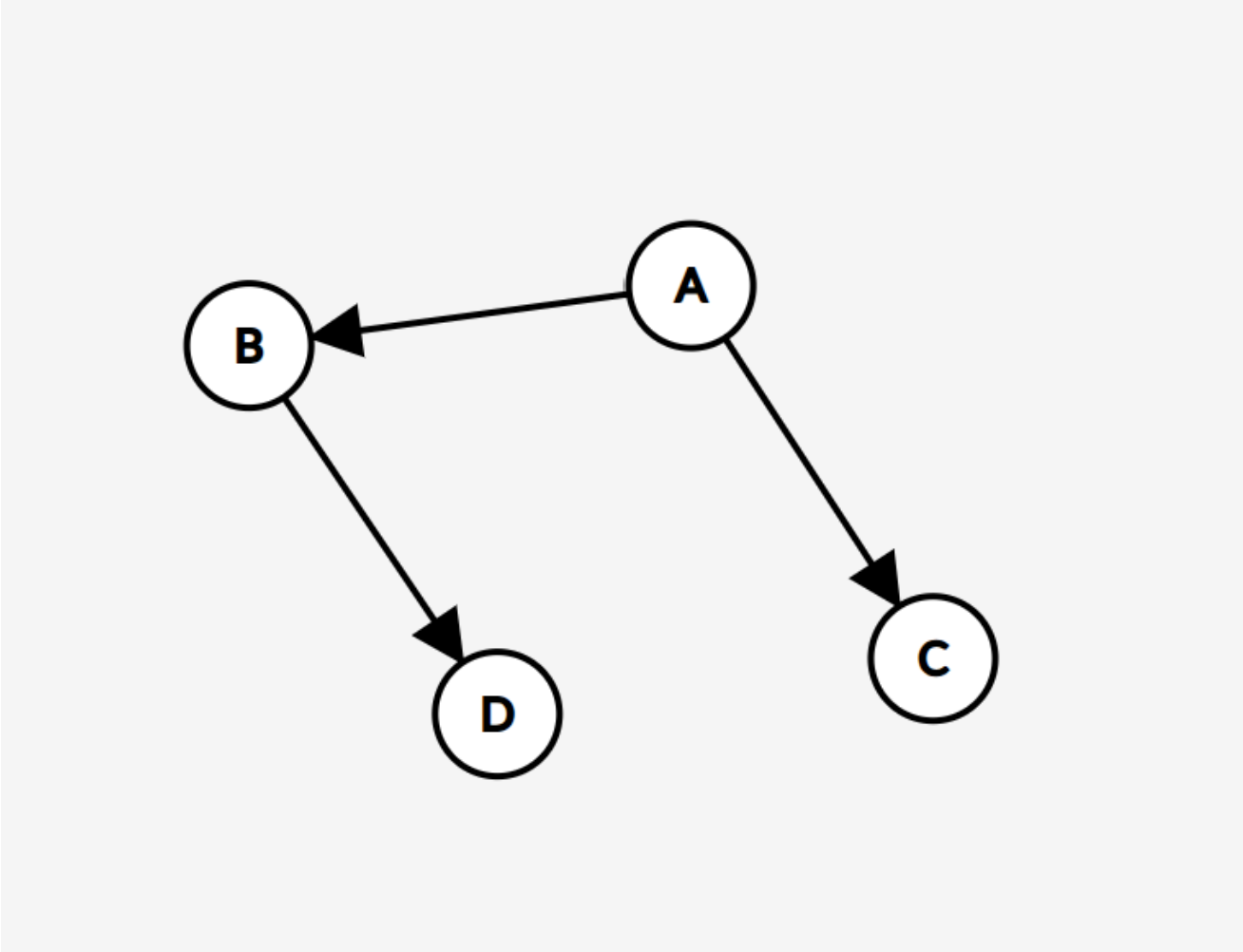
BFS using Kahn's Algorithm

Kahn's algorithm uses in-degree, which is the number of incoming edges or dependencies a vertex has. As long as the vertex has no remaining dependencies, it can be placed in the queue.

How it works:

- Calculate the in-degree of every vertex.
- Add every vertex with in-degree zero to a queue.
- Remove a vertex from the queue and add it to the ordering list.
- Reduce each vertex's in-degree by one.
- Whenever a neighbour's in-degree becomes zero, add it to the queue.
- Continue until the queue is empty.

- If the ordering doesn't include all vertices, the graph contains a cycle.



Vertex	In-degree
A	0
B	1
C	1
D	2

Sorting using Kahn's Algorithm, we get the topological order of `A -> B -> C -> D`.

References:

- https://aiknowledgehub.in/solving-a-maze-using-bfs-and-dfs/
- https://www.tutorialspoint.com/graph_theory/graph_theory_graph_algorithms.htm
- https://neo4j.com/blog/graph-data-science/graph-algorithms/#h-graph-traversal
- https://www.puppygraph.com/blog/graph-algorithms
- https://www.w3schools.com/dsa/dsa_algo_graphs_traversal.php
- https://www.geeksforgeeks.org/dsa/graph-data-structure-and-algorithms/
- https://www.youtube.com/watch?v=EFg3u_E6eHU&t=467

- https://youtu.be/j0OUwduDOS0?si=fC5Z0ACYLtzva-Rw
- https://youtu.be/TtQi1LVVOUI?si=Aw0sQw2AldHw4fxR
- https://youtu.be/9PHkk0UavIM?si=1SaYD_QVCmuN2K5f
- https://www.geeksforgeeks.org/dsa/spanning-tree/

3.1 Foundation of Connectionist AI

What is Artificial Intelligence (AI)?

Artificial Intelligence (AI) is a field of computer science focused on enabling computers and machines to do tasks that are commonly associated with intelligent beings. This includes, but not limited to, processing thoughts, decision-making, and creativity. In the 21st century, AI has become increasingly integrated into everyday life. Examples include AI-generated images, enemy behavior in video games, recommendation systems, and Large Language Models (LLMs) such as ChatGPT and Claude. These examples show how AI can be used across entertainment, communication, and many other fields.

Components of Artificial Intelligence:

Machine Learning

Machine Learning is a subset of AI that focuses on making machines or algorithms "learn" patterns from training data and use them to make predictions or decisions. This "training" enables machine learning models to make decisions or predictions without explicit, hard-coded instructions. The core idea behind machine learning is that if you optimize a model's performance on a dataset of tasks that resemble the real-world problems it will be used for, through a process called model training, the model can make accurate predictions on the new data it is used for.

Examples:

- Taobao Recommendations: Recommendation systems analyze, user behavior, inputs like browsing history, and other interactions to suggest products users might be interested in.
- Spam Detection: Machine learning models learn patterns from examples of spam and legitimate emails to filter incoming messages.

Main types of machine learning algorithms:
Supervised Learning
Supervised Learning is when a model is trained on "Labelled Datasets" or in simpler terms, datasets that are categorized. The raw data that these algorithms use already have a predetermined label attached to them. For example, a model trained on labeled images of cats and dogs can learn patterns that help it classify a previously unknown image.

Supervised learning has two main categories:

- **Classification**, predicts categorical outputs, meaning it assigns data into predefined classes like spam/non-spam emails or disease risk categories. These algorithms learn to map input features to discrete labels.
- **Regression**, predicts continuous values, such as house prices or product sales. It learns the relationship between input features and a numerical target variable.

Unsupervised Learning
Unsupervised learning is when a model is trained on unlabelled data. The algorithm has to find any hidden patterns, groups or relationships within the data on its own. It's mainly used for clustering, dimensionality reduction and data visualization. For example, a model analyzing customer purchasing behavior may identify groups of customers with similar shopping habits without being given predetermined customer categories.

Unsupervised learning has three main categories :

- **Clustering**, groups data points into clusters based on their similarity. This technique is useful for identifying patterns and relationships in data without the need for labeled examples.
- **Dimensionality Reduction Techniques**, reduces the number of features while preserving important information.
- **Association Rule Learning**, technique for discovering relationships between items in a dataset. It identifies rules that indicate the presence of one item implies the presence of another item with a specific probability.

Reinforcement Learning
Reinforcement learning is when a model is trained to make a sequence of decisions through trial and error. The system or we call the agent, interacts with the environment, receives feedback in the form of rewards or penalties and learns optimal actions over time. For example, an AI agent learning to play chess gets positive feedback for good moves and negative for poor ones. Over time, it improves its strategies through repeated games.

Types of Reinforcement Learning :

- Positive Reinforcement, rewarding desired behavior (e.g., giving points for correct answers).
- Negative Reinforcement, removing negative outcomes to encourage good actions (e.g., turning off a buzzer after the right move).

Deep Learning

Deep learning is a subset of machine learning that builds multilayered neural networks, called deep neural networks, that more closely simulate the complex decision-making power of the human brain.

Neural networks consist of interconnected artificial neurons organized into layers. In a deep neural network, information passes through multiple layers, where mathematical transformations allow the model to learn increasingly complex patterns.

The deep learning process consists of three layers, the input layer receives data, the hidden layers transform the information, and the output layer generates a prediction or other result. Through training, the network adjusts itself to improve its performance.

What is Connectionist AI?

Connectionist AI is an approach to artificial intelligence that models human behavior through networks of interconnected artificial neurons or processing units we call nodes. Instead of relying on hard-coded rules or instructions, connectionist models learn patterns from data by adjusting the strengths of connections between neurons.

Symbolic vs Connectionist AI

Symbolic AI and Connectionist AI are two approaches to developing artificial intelligence. They differ mainly in how they represent information and solve problems. Symbolic AI uses rules and logical structures to solve problems based on the principle of warrantable knowledge. Connectionist, or neural network AI is based on the structure of the human brain, processes data and adapts easily to new data as it relies on patterns.

For example, when trying to recognize a cat using symbolic AI, hard-coded rules will be used to give categorisations or characteristics to a cat, like a cat has four legs, two ears, or other feline characteristics. If these conditions are fulfilled, the AI will recognise it as a cat. In contrast, connectionist AI can only recognize a cat once it has been trained sufficiently to distinguish what is a cat. It has to be trained on labeled cat and non-cat images, learning patterns in the data that help it distinguish cats from other objects. This is just one among the many differences between these two.

n present day, hybrid forms of these AI have been created, called Neuro-Symbolic AI. This combines neural networks with symbolic reasoning to tackle tasks like visual question answering, where understanding complex scenes and answering questions requires both pattern recognition and logical reasoning.

To get a clearer understanding of the differences, we can refer to tthe table below.

Aspect	**Symbolic AI**	**Connectionist AI**
Knowledge Representation	Uses explicit symbols and rules	Uses connections and weights in networks
Transparency	High, interpretable and traceable	Low, hard to interpret
Reasoning Ability	Excels in logical reasoning and problem-solving	Excels in pattern recognition and prediction
Scalability	Struggles with large, complex datasets	Scales well with large datasets
Flexibility	Less flexible and adaptable	Highly adaptable through training
Resource Requirements	Less resource-intensive	Requires significant computational resources
Risk of Overfitting	Lower risk	Higher risk

> Overfitting = Cases when an AI model fits training data too closely, instead of learning the underlying patterns behind the data.

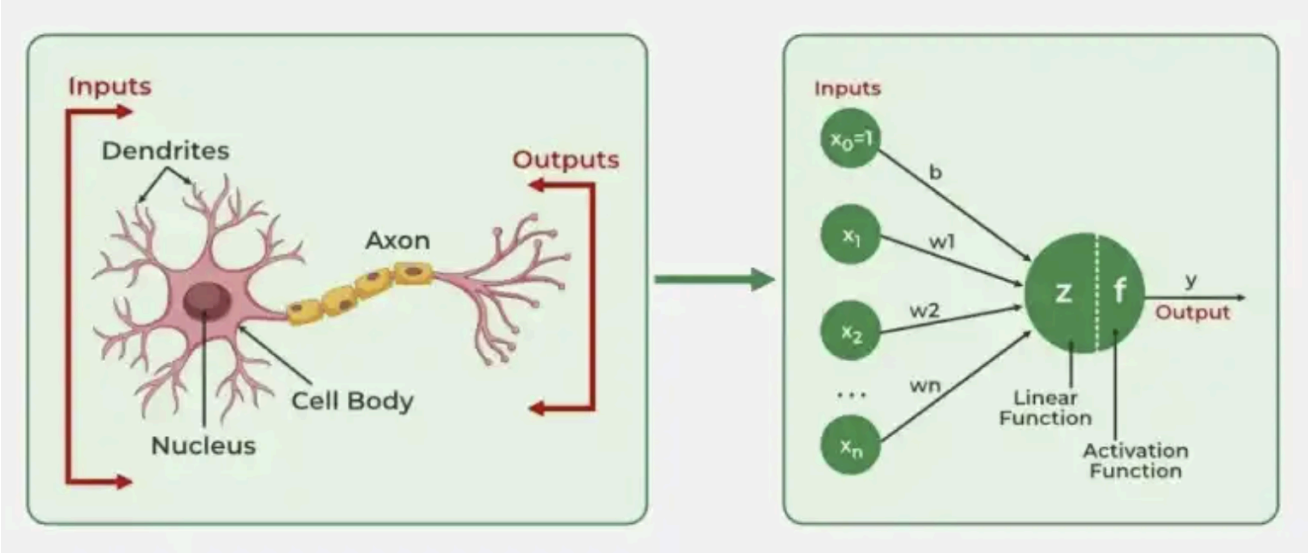
References:

- https://www.ibm.com/think/topics/artificial-intelligence
- https://www.geeksforgeeks.org/machine-learning/types-of-machine-learning/
- https://www.geeksforgeeks.org/deep-learning/introduction-deep-learning/
- https://www.geeksforgeeks.org/deep-learning/neural-networks-a-beginners-guide/
- https://www.geeksforgeeks.org/artificial-intelligence/difference-between-symbolic-and-connectionist-ai/

3.2 The Artificial Neuron

What is Artifical Neuron?

Artificial neurons serve as the fundamental building blocks of connectionist AI. When linked together, they form Artificial Neural Networks (ANNs) that are the foundation of modern deep learning. Structurally, an ANN consists of these interconnected neurons organized into distinct layers. Each neuron receives inputs, combines them using weights and a bias, and applies an activation function to produce an output.



The idea behind neural networks themselves comes from human biology and is inspired by the brain. This structure can be compared to biological neurons, with dendrites receiving signals as inputs, the cell body processing them, and the axon transmitting signals to other neurons. Similarly, artificial neurons receive inputs, process them, and pass their outputs to other neurons. However, artificial neural networks are only simplified mathematical models and do not reproduce the full complexity of biological neurons.

How does It Work?

We can think of an artificial neuron as a small decision-maker. It receives information, processes it, and produces an output. For example, when identifying an image, a neuron might respond more strongly to certain patterns in the image.

Each neuron receives inputs, such as numbers or pixel values, from the input data or other neurons. Each input has a weight, which determines how strongly it influences the neuron's result. The neuron combines these weighted inputs with a value called a bias, then applies an activation function to determine its output.

When this concept is translated into a mathematical model it becomes:

$$z = \sum_{i=1}^{n} w_i x_i + b \quad \text{and} \quad a = \sigma(z)$$

Where:
x_i = input values,
w_i = weights of each input,
b = bias,
z = weighted sum,
σ or f = activation function,
a or y = output,

In simpler terms, this basically means the neuron calculates the weighted sum before activation by multiplying each input with their weights and adds them together. Then a bias is added and the value is passed through the activation function.

Example

Imagine we are supposed to calculate weighted sum and the output. We are given:

- x_1 = 0.8
- x_2 = 0.5
- w_1 = 2.0
- w_2 = -1.0
- b = -0.5

Calculating weighted sum:

Step	Result
1	$z = (w_1 x_1 + w_2 x_2) + b$
2	$z = (2.0 \times 0.8) + (-1.0 \times 0.5) + (-0.5)$
3	$z = 1.6 - 0.5 - 0.5$
4	$z = 0.6$

Calculating output using ReLu:

Step	Result
1	$f(z) = max(0, z)$
2	$f(0.6) = max(0, 0.6)$
3	$y = 0.6$

Using the ReLu (Rectified Linear Unit) function as the activation function, we get that the final output is 0.6. The ReLu function outputs zero for negative inputs and retains positive inputs. We will learn more

about these concepts in the next chapter.

References:

- https://www.ibm.com/think/topics/artificial-intelligence
- https://www.geeksforgeeks.org/machine-learning/types-of-machine-learning/
- https://www.geeksforgeeks.org/deep-learning/introduction-deep-learning/
- https://www.geeksforgeeks.org/deep-learning/neural-networks-a-beginners-guide/
- https://www.geeksforgeeks.org/artificial-intelligence/difference-between-symbolic-and-connectionist-ai/
- https://towardsdatascience.com/explain-like-im-five-artificial-neurons-b7c475b56189/

3.3 Neural Network Architecture

A neural network takes input data and passes it through layers of connected nodes to make a prediction.

Neurons are organized into layers, each with a different role:

- **Input Layer:** Receives the original data.
- **Hidden Layers:** Process the information and learn patterns.
- **Output Layer:** Produces the final result.

Information passes from one layer to the next, with each layer transforming the information it receives. You can think of this process as gradually assembling puzzle pieces. Early layers may identify simple patterns, while later layers combine them into more complex features. Through training, the network adjusts its weights and biases to improve its predictions.

Input Layer

The input layer is where data enters the network. It receives raw feature values from a dataset. For example, to predict house prices using three features: House size in square feet Number of bedrooms Age of the house in years

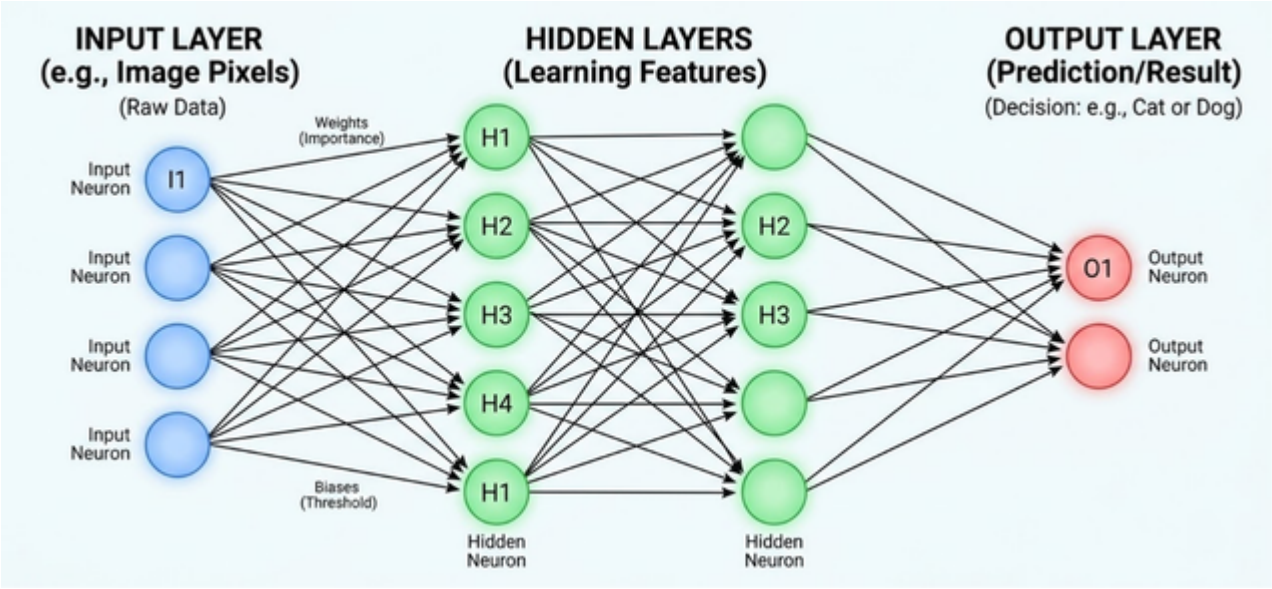
Hidden Layer

Hidden layers sit between the input and output layers. They perform math steps to find patterns between features. Instead of looking at size and age separately, a hidden node can combine them to measure something deeper, like overall wear and tear or space efficiency, before making a final prediction.

Output Layer

The output layer gives the final calculated result. The type of result depends on the task: Regression Tasks: Predicts a continuous number (estimating a house price of $250,000). Classification Tasks: Predicts a category probability (a 90% chance of being a cat and a 10% chance of being a dog).

The following is a simple illustration on neural network architecture can be visualised.



In this example, the input layer receives the data of the image pixels. The hidden layer then processes the information to learn the patterns from the data. This can include learning patterns of cat or dog features, such as having two ears, their teeth, and other characteristics. Finally, the output layer predicts that based on the input data, the result is a cat or a dog.

Other Components of the Neural Network:

Weights and Bias

Weights are a number that multiplies an input to show its importance. A higher weight gives an input a bigger impact on the final answer.

Bias is an extra value added to the calculation that helps shift the output so the network can fit the data better.

Imagine deciding whether to buy a jacket. Weights represent how much importance you give to each factor. For instance, cold weather gets a high weight (very important), while matching your shoes gets a low weight. Bias is your baseline tendency, if you get cold easily, you have a high positive bias, making you much more inclined to buy the jacket regardless of other factors.

Activation Functions

Activation function are like decision-makers sitting inside a neuron. After the network multiplies the inputs by the weights and adds the bias, the activation function decides whether that neuron should "fire" (pass the message forward) and how strong that signal should be. This matters because without activation functions, a neural network could only do basic straight-line math, no matter how many layers it has. Activation functions add non-linearity (curves and bends) to the math, allowing the network to learn complex patterns like recognizing a face, understanding human speech, or driving a car.

Think of it like a light switch with a dimmer: If the incoming signal is too weak, the switch stays completely off (output is 0). If the signal passes a certain threshold, the switch turns on, and the activation function decides how bright the light should shine. Common examples include ReLU (which simply blocks negative signals and lets positive ones pass through) and Sigmoid (which squashes values between 0 and 1, often used for yes/no probabilities).

Two common activation Functions:

- **ReLU** is like a thermometer that can't read below freezing. If the temperature drops below 0, it just says 0. If it's warmer than 0, it shows the actual number (like 5, 15, or 30). the function returns zero when the input is negative and returns the input itself when it is positive.

$$ReLU(z) = max(0, z)$$

- **Sigmoid** is like a sliding scale from 0 to 1 (or 0% to 100%). It is used for yes/no questions, like "Is this a picture of a cat?". It takes any score and turns it into a clear probability—like 0.9 (90% sure it's a cat) or 0.1 (10% sure it's a cat). The sigmoid function converts any real-valued input into a value strictly between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

3.4 How Neural Networks Learn

Training is the process of updating weights and biases so the model's guesses get closer to the real values. In simple terms, this means the model improving itself so it gets “smarter” or better suited for its purpose.

Training Step-by-Step

Forward Propagation

Forward propagation is the first step, where input data travels from the input layer through the hidden layers until it reaches the output layer. As the data passes through each neuron, the network performs calculations using its weights and biases to produce a prediction.

For example, imagine we want a neural network to predict the price of a house. We provide information such as the house's size, location, and number of bedrooms. The network processes this information and predicts that the house costs $300,000. At this stage, the network is simply making its best guess based on what it has learned so far. It has not checked whether the prediction is correct yet.

Loss Function Calculation

After making a prediction, the network needs to measure how accurate it is. This is done using a loss function, which calculates the difference between the predicted value and the actual value.

For example:

- Predicted house price: $300,000
- Actual house price: $250,000
- Prediction error: $50,000

The loss function uses this difference to calculate how wrong the prediction is. A higher loss means the prediction is further from the correct answer, while a lower loss means the prediction is more accurate. The main goal of training is to reduce this loss as much as possible so that the network can make better predictions in the future.

Backpropagation

Once the network knows how much error it has made, it needs to figure out how to improve. This is where backpropagation comes in. Backpropagation is the process of sending information about the error backward through the network, starting from the output layer and moving toward the earlier layers. It calculates how much each weight contributed to the final error. In simple terms, it helps the network understand which internal values need to change to make a better prediction.

For example, if the network predicts a house price that is too high, backpropagation calculates how the weights connected to house size, location, and other inputs affected that prediction. It then provides

information that helps the network adjust those weights.

Gradient Descent and Learning Rate

After backpropagation calculates the information needed to improve the network, gradient descent uses it to update the weights and biases. Gradient descent is an optimization method that helps the network find values that produce a lower loss. It does this by gradually adjusting the weights in the direction that reduces the error. The size of each adjustment is controlled by something called the learning rate. The learning rate determines how big or small each update should be.

There are two common problems with choosing the learning rate:

- Too Large Learning Rate: The network makes very large adjustments to its weights. It may jump over the best values and struggle to find the lowest error, making training unstable.
- Too Small Learning Rate: The network makes very small adjustments. Although it may eventually improve, training becomes very slow and requires many updates to reach a good result.

3.5 Evaluating Learning

After training, you want test how well the network works on new, unseen data. This ensures the model is truly correct and learns actual patterns from the data. This whole process prevents the model from accidentally picking up too many worthless patterns and matching too closely with the training data (overfitting).

Dataset Splitting

To keep testing honest, the dataset is split into three separate parts:

- **Training Set (~70%):** Used directly to train weights and biases.
- **Validation Set (~15%):** Used during training to tweak settings and prevent the model from memorizing.
- **Test Set (~15%):** Saved for the very end to check real-world accuracy.

These are used together to make sure the model is trained for all cases of its usage.

Common Training Mistakes

While training, there are two very common mistakes that happen, if testing is not done properly.

- **Overfitting:** The model gets high accuracy on training data, but low accuracy on test data because it memorized noise instead of learning main trends.
- **Underfitting:** The model gets poor accuracy on both training and test data because it is too simple to learn anything useful.

Classification Metrics

To measure classification performance, these metrics look at correct and incorrect predictions:

- **Accuracy:** The percentage of total correct predictions.
- **Precision:** Out of all positive predictions, how many were actually correct.
- **Recall:** Out of all real positive items, how many did the model find.
- **F1-Score:** The balance between precision and recall, best used when data classes are uneven (detecting rare fraud cases).

3.6 Major Neural Network Types

Connectionist AI relies on Artificial Neural Networks (ANNs), which model cognition using layers of interconnected rocessing units (artificial neurons) that adjust weighted connections through data. Within these, there are also many different types of the neural networks.

Feed Forward Networks (FNNs)

To put it simply Feed forward Network is like an asembly line. It is the most basic type of neural network. Where information only moves in one direction, forward. Data enters through an "input layer", passes through one or more "hidden layers" that process the information, and exits as a final decision through an "output layer". Feedforward Neural Network are best for: Simple tasks like identifying if an email is spam or predicting a price based on a spreadsheet of basic data.

Convolutional Neural Networks (CNNs)

To put it simply Convolutional Neural Networks is like scanning a picture with a magnifying glass. CNNs are designed specifically to look at grids of data, like images by scanning small chunks of the image piece by piece. It looks for basic edges and shapes first, then pieces them together. Convulutional Neural Networks are best for: Facial recognition, identifying diseases in medical x-ray, and computer vision for self driving cars.

Recurrent Neural Networks (RNNs)

To put it simply Recurrent Neural Networks is like reading a book and remembering the last sentence you read. Standard networks forget everything they just did the moment they started a new job ask. RNN solves this problem problem by using a built in "memory" loop that lets them remember previous inputs, so they can understand context in a sequence of data over time. Recurrent Neural Networks are best for: Speech recognition (like alexa, siri, or google), translating sentences between languages, and predicting the weather.

Transformers

To put it simply Transformers is like looking at a whole paragraph at once and instantly understanding how every word relates to the other. Transformers are the modern heavyweight of the ai world. Transformers use an "attention mechanism." This allows the networks to look at an entire sequence of data simultaneously and figure out which part are the most important, making them incredibly fast and smart with language. Transformers are best for Large Language Models (LLMs) like Chat GPT, Gemini, and advanced text generation.

Generative Adversarial Networks (GANs)

To put it simply Generative Adversarial Networks is like an art forger and a police detective. GANs are actually two networks competing against eachother. Where one network (The forger) tries to generate fake data, like a realistic picture of a human face and the other one (the detective) tries to spot whether the iamhe is real or fake. They constantly learn from eachother until the forger creates images the detective couldn't tell the difference between its real or fake. Since its so lifelike. Generative Adversarial Networks are best for: Creating AI art, deepfakes, generating realistic video game assets, and enhancing low resolution images.

Autoencoders

To put it simply Autoencoders is like packing and unpacking a perfectly organized suitcase. An autoencoders learns how to compress (encode) data down to its most essential, bare bones information to save space, and then learns how to rebuild (decode) it back to its original form. Autoencoders are best for: Removing background noises from audio or grainy spots from photos, and detecting anomalies (like credit card fraud) by noticing when data doesn't "unpack" normally.

Long Short-Term Memory Networks (LSTMs)

To put it simply Long Short-Term Memory Networks is like taking detailed notes so you don't forget the beginning of a long lecture. LSTMs are an upgraded, highly popular version of RNNs. LSTMs have a special internal "gates" that actively decide what information is important enough to keep in long-term memory and what to throw away. Long Short-Term Memory Network are best for: Complex language translation, generating text, and analyzing time sensitive data like stock market trends.

Self-Organizing Maps (SOMs)

To put it simply Self-Organizing Maps is like sorting a massive, messy pile of laundry by putting similar colors and fabrics next to eachother. SOMs are given chaotic, unlabeled data and left alone to figure it out. They organize the data into a 2D map where similar items are grouped close together. Self-Organizing Maps are best for: Customer segmentation (grouping similar types pf shoppers together), identifying weather patterns, and visualizing complex data.

Spiking Neural Networks (SNNs)

To put it simply Spiking Neural Networks is like sending a message using flashes of flashlight in the dark. SNNs are the closest networks to how biological human brains actually work. Instead of processing numbers constantly, SNNs only send a signal (a "spike") when they gather enough information to reach a certain threshold. Because they aren't "on" all the time. Spiking Neural Networks are best for: Extremely low power AI chips, robotics, and advanced brain computer interference.

Modular Neural Networks

To put it simply Modular Neural Networks is like a large company specialized departments. Instead of building one massive network to solve a huge problem, a modular network links several smaller separate networks together. Each small network is an expert at one spesific task, and a "manager" network combines their findings to make a final decision. Modular Neural Networks are best for: Highly complex systems like self-driving cars (where one module watches for pedestrians, another reads signs, and another controls the steering wheel).

3.7 Practical Implementation

Connectionist AI is everywhere in our lives. It learns by finding patterns in data rather then following rigid, pre-programmed rules. Connectionist AI could be found in many of our daily lives items, like cars, on your phone, etc. Here's how Connectionist AI a practically implemented in things you likely use everyday.

Computer vision & image processing

Connectionist AI allows computers to "see", usually using Convolutional Neural Networks (CNNs). For examples:

- **Self-Driving Cars:** Neural Networks processes the live video feed from the car's camera to make split second driving decisions. Like when there's a pedestrian crossing the roads, the car's camera "see" there's someone/something infront of the cars so its makes a split-second decision to stop the car smoothly.
- **Facial Recognition & Photo Apps:** Usually we unlock our phones using face ID, because its more convenient then inputting your password everytime you want to unlock your phones, or when a photo app automatically groups a pictures of you god together. Those activity are made possible with Connectionist AI by identifying the unique visual patterns of a face or an object.
- **Medical Imaging:** Connectionist AI are also very usefull in healthcare. Neural networks are trained on thousands of x-ray and MRI scans. They can now analyze new scans to spot early signs of diseases or tumors much faster than human doctors.

Natural Language Processing (NLP) & Speech

Understanding human language that is full of slang, accents, and context is very hard for ordinary computers. With Connectionist AI, particulary Recurrent Neural Networks (RNNs) and transformers solves this problem.

For examples:

- **Digital Assistants:** When you speak to Siri, Alexa, and other voice assistants, neural networks are doing 2 different things: first, they transcribe your spoken audio into text, and second, they figure out what you actually mean so they can respond accurately.
- **Live Translation:** When you point your smartphone at a menu trying to translate the content into your native language to make it easier to understand, google translate changes the text in real time. What you're seeing are neural networks processes both image and the language simultaneously.
- **Large Language Models (LLMs):** Chat GPT, Gemini, and other similiar LLMs uses transformers networks to understand the contect of entire paragraphs at once, allowing them to write essays, answer questions, write computer codes, etc.

Predictive Analytics & Recommendation Engines

Conectionnist AI are excellent in looking at massive ammounts of historical data and predicting what'll happen next.

For example:

- **Content Recommendations:** Netflix, Youtube, Spotify and other similars apps uses neural networks to analyze your past viewing or listening habits, like what you like to watch or listen. Then, compare them to millions of other users, and accurately predict what movie or music you'll like.
- **Fraud Detection:** Banks and credit card companies uses neural networks like Autoencoders to monitor your spending habits. If your cards is suddenly used to buy expensive items in another country, the network will recognize that this patter doesnt match your usual spending habit and will immediately flags the transaction as fraud.
- **Stock Market & Finance:** Financial institution uses complex networks like LSTMs to analyze years of stock market trends, news headlines, and economic data to predict whether a stock's price will go up or down.

3.8 Applications, Limitations, and Responsible AI

Connectionist AI, has completely transformed how computers interact with the real world, but they are not perfect for every situation. Because they learn from massive ammounts of data rather than explicit programming rukes, their strengths and weaknesses are very different from traditional softwares.

Applications

Neural Networks excels at handling "messy" datas. Like facial fecognition, translating languages, or making predictions based on massive histories. Instead of using a rulebook like traditional programming, Connectionist AI are designed to learn patterns from examples. Because connectionist AI learned the general pattern rather than memorizing a rigid rule, it can now look at brand new, messy picure of an object and accurately determine the object, even if its blurred, low-light or other things. This ability to find aptterns in chaos is why neural networks can translate human slang or predict stock market. They dont need the world to be perfectly organized to understand it.

Limitations

Despite their incredible power, Connectionist AI have significant drawbacks that make them unsuitable for certain tasks.

For example:

- **The “Black Box" problem:** A neural network might succesfully identify a disease or a tumor from an x-ray, but it could not easily explain why it made the decision. It's hard for human to trace the logic, since it's filled with a giant web of numbers and probabilities. This lack of transparency is a major issue in healthcare, law, and finance.
- **Data Dependency:** Connectionist AI is basically useless without a massive ammount of data. A neural network needs to process thousand of labeled images of an boject before it could reliably recognize one. If the training is biased, incomplete, or low-quality, the output's will be too.
- **Computational Expense:** Training a powerful neural network, require millions of dollar worth of GPU running constantly for months that consumes a massive ammount of electricity and limits the development of top tier AI to large tech corporations.
- **Lack of Common Sense:** An AI might write beautifully structured essay about history, but it is just predicting tge next logical word based on patterns it saw during training, it doesnt actually comprehend the facts and actually understand the real world. This is why AI can confidently give completely wrong answers (Hallucinations).

Responsible AI

Responsible AI refers to the development and use of AI systems in ways that are ethical, fair, safe, transparent, and beneficial to society. Since neural networks learn from data and make predictions based

on patterns, their development and use must be carefully managed to minimize potential harm.

There are few important principles to responsible AI:

Fairness and Bias

AI systems should treat individuals fairly and avoid discrimination based on characteristics such as gender, race, or age. Since AI models learn from training data, biased data may cause them to reproduce or reinforce existing discrimination. For example, a recruitment program primarily on resumes from male applicants may unfairly favor male candidates over equally qualified female candidates.

Transparency and Explainability

AI systems should provide understandable information about how they make decisions. Users should be able to understand when AI is being used and, where possible, why a particular result or decision was produced. For example, a medical imaging program that detects illnesses should provide reasons for the outputted conclusion.

Privacy and Data Protection

AI systems should protect personal and sensitive information collected during their development and operation. Personal data should be collected, stored, and used responsibly, with appropriate security measures to prevent unauthorized access and misuse. For example, any AI chatbot should protect users information and personal data unless the user specifically consented to it being shared.

Accountability

Developers and organizations should take responsibility for the outcomes of AI systems. They should establish clear responsibilities and mechanisms to monitor AI systems, investigate problems, and address harmful consequences. For example, if a self-driving car causes an accident due to system failure, all related organisations should address the problem.

Safety and Reliability

AI systems should be tested and monitored to ensure that they operate safely, consistently, and reliably. They should be designed to minimize errors and prevent foreseeable harm, especially in high-risk applications. For example, AI chatbots should be ensured to not encourage harmful activites or offer medical advice towards individuals.

How to Use AI Responsibly

The following are a few practices that we can adopt to use AI responsibly.

- **Verify all AI generated information**, AI can produce inaccurate or misleading information that sound logically plausible even though it is all blatant misinformation (hallucination), so important

outputs should be checked against reliable sources.

- **Do not overshare**, avoid entering passwords, financial information, or other sensitive information into AI systems without understanding how the information will be managed.
- **Avoid overdependence**, AI should be used as a tool to support learning, creativity, and decision-making rather than replacing human thinking.

Connectionist AI enables computers to learn complex patterns and perform tasks such as image recognition, language processing, and prediction. However, neural networks also have limitations, including their dependence on training data, computational requirements, and difficulty of interpretation. Therefore, responsible development and use are essential to ensure that these systems remain fair, safe, reliable, and beneficial to society.